

Computer Science (H446)

NEA Programming Project

FlightDeck

Candidate Name: **Jake Aiden Spowart**

Candidate Number: **3310**

Centre Name: **Heanor Gate Spencer Academy (HGSA)**

Centre Number: **23218**

There are 189 pages in this document.

Table of Contents

Table of Contents	2
Analysis of the Problem	6
Problem Definition and Resolutions	6
Potential Challenges	6
Suiting to a Computational Solution	7
Computational Methods and Approaches	8
Thinking Ahead	8
Abstraction	9
Decomposition Diagram	10
Heuristics	12
Identifying Stakeholders	13
Secondary Research - Existing Solutions	14
Existing Solution 1: FlightRadar24	14
Existing Solution 2: FlightAware	18
Existing Solution 3: Veryon	21
Primary Research - Consumer Questionnaire	25
Language Choice	30
Solution Requirements	31
List	31
Self-Sought Requirements:	31
Secondary Research-Driven Requirements:	32
Primary Research-Driven Requirements:	33
Table (Any Further Details)	33
Hardware and Software Requirements	35
Solution Limitations	35
Success Criteria	37
Design of the Solution	45
Decomposition	45
Graphical UI Designs	45
Architecture	48
Architecture	48
Programming Paradigms	48
User Navigation	49
Key Components	50
Referenced in Analysis of the Problem > Solution Requirements > List / Table (Any Further Details).	50
Data Flow	51
Where Data Comes From	51
How Data is Processed	51
Where is Data Stored	51
Structure	52
Use of Data Structures	52
Key Variables	52

Validation Involving Key Variables	54
Database Design (Data Table)	55
Database Design (ERD's)	55
Post-Development (Function and Robustness) Test Plan	56
Usability Test Plan	59
Algorithms	61
Pseudocode Interpretations	61
Account Creation and Separate Sign In	61
Account Creation Validation and Error Messages	62
Sign In Validation	63
Database Integration	64
Database Commitment with Validation	64
API Integration	66
API Queries	66
Usability	67
GUI Designs	67
Usability Features	67
Simplified GUI Elements and Back-end Functions	67
Data Entry	67
Buttons and Navigation	68
Confirmations and Data Re-entries	68
Usability Features from Research	68
Development of the Solution	69
3.1 Library and SDK Imports and Database Integrations	69
3.2 "Create Account" Interface	71
3.3 "Create Account" Execution, Validation / Verification and DB Commitment	76
Personal and Commercial Aspects	76
Commercial Only Aspects	81
3.4 "Sign-In" Interface	83
3.5 Main Program Interface	87
3.6 "Search for Information" Tab	91
General GUI Development	91
API Integration, Query and SQL Functions	96
Validation Triggers	103
GUI Elements	103
3.7 "Manage Alerts" Tab	106
General GUI Development	106
API Integration, Query and SQL Functions	108
3.8 "Notifications" Tab	111
General GUI Development	111
API Integration, Query and SQL Functions	112
Code Appendix (Selection Statements)	116
Key Branches, Conditions and Logic Statements:	119
Test Tables	121

T1 - Configured validation error messages only showing on "Personal Create Account" tab not "Commercial Create Account" tab, should be both.	121
T2 - Validation error messages showing on the "Commercial Create Account" tab even when validation criteria met.	122
T3 - Validation error message ("Passwords must match") showing on the "Personal Create Account" tab when validation criteria are not met, but the data is still committed to the database.	125
T4 - Validation error message ("Passwords must match") showing on the "Commercial Create Account" tab when validation criteria are not met, but the data is still committed to the database.	128
T5 - Upon entering valid credentials into the sign-in window (window 2), the main window (window3) doesn't open and a very long list of errors are returned.	130
T6 - A button does not appear under the main window under the "Your Alerts" tab.	131
T7 - The "Search for Flight" and "Search for Airport" do not disappear as expected when the tab is switched.	132
T8 - "DateFrom" and "DateTo" inputs for the "get_searchFLT()" API query, is not recognised by the "datetime" library.	134
T9 - API "get_searchAPT()" query does not recognise the entered airport code, which should be valid.	134
T10 - Difficulty outputting (graphically) API query returns, without having to create a separate element for each attribute.	135
T11 - Difficulty outputting (graphically) API flight query returns and finding the correct attribute name.	137
T12 - Additional stats (attributes) don't appear when the "Show Stats For Nerds" button is pressed, after searching for a flight.	138
T13 - "email_address_per" and "email_address_co" variables (used in the main window, to validate flight alert creation) are inaccessible, as they are outside the scope of which they were set / defined.	140
T14 - Transactions into the "alerts" table within the "flightdeckdb" database (for flight alerts), are not committing the email address attribute.	141
T15 - "list index out of range" for "ExistAlertResult3[4]" / "ExistAlertResult3[3]" post-SQL-query.	141
Evaluation	143
Post-Development Test Plan (Evidence and Annotations)	143
Usability Test Plan (Evidence and Annotations)	146
Test Plan Evidence Appendix	148
For Post-Development Test Plan	148
For Usability Test Plan	158
Success-Criteria Testing	162
Future Address of Partially/Mostly Met and Unmet Success Criteria	162
Future Address of Partially/Mostly Met and Unmet Functionality	162
Maintenance Issues and General Limitations	164
Maintenance Limitations	164
Libraries And Other Import Limitations	164
Database Management Limitations	164
General Limitations	165
API Sandbox and Scalability Limitations	165

Database Scalability Limitations_____	165
Development Complexity / Time Limitations_____	165
User Feedback / Use_____	165
Future Developments To Meet Limitations_____	166
API Sandbox and Scalability Limitations_____	166
Database Scalability Limitations_____	166
Development Complexity / Time Limitations_____	166
User Feedback / Use_____	166
Overall Conclusion_____	167
Code Appendix_____	168
Key Function: get_title1()_____	169
Key Function: get_basic_create()_____	169
Key Function: get_commerce_create()_____	170
Key Function: hash_password(password)_____	171
Key Function: window1_run()_____	172
Key Function: get_sign()_____	175
Key Function: window2_run()_____	176
Key Function: get_searchAPT(SearchEntApt)_____	178
Key Function: get_searchFLT(SearchEntFlt)_____	178
Key Function: window3_run()_____	181
References_____	189

Analysis of the Problem

Problem Definition and Resolutions

For my programming project, I am going to be designing and constructing a flight information and management system for the aviation industry that integrates both information-providing, alert systems and some other features for both a commercial perspective (airlines, airports, etc) and a personal perspective (groups and individual passengers), with user interfaces (graphical and possibly menu-driven) through the FreeSimpleGUI library that has an extensive number of widgets and interactive elements, as well as provided documentation and a collection of resources.

The FreeSimpleGUI library is also known for moderate performance regarding API and database queries, which means that real-time database transfers and API calls are not likely to be delayed or have freezes occur. Even if they were to occur for a number of (even) seconds, this would not impact my solution as data would not need to be transferred in direct real-time, for a number of reasons.

The commercial point of view will include data that is specific to an airlines fleet of aircraft, of which they will be able to customise and add to (through each individual aircraft's unique registration, like a car registration). Commercial data will include: flight numbers, registrations, live and possibly historic aircraft data - through an integrated API and provided test data that can be downloaded - and other information you may need as a commercial user. Personal data will include: flight numbers, departure and arrival times of flights (as well as estimated times) and other information you may need as a passenger.

The use of a computer to solve this problem will significantly benefit a user by giving them the ability to effectively manage a variety of flights on a single software, meaning there is a significantly reduced need for paper documentation of flights on the commercial side and less need for users to have to continuously oversee multiple different applications that provide flight alerts and updates on the personal side. Hence, saving time and resources on both sides.

Potential Challenges

The main challenges will likely be:

- My previous A level and GCSE level projects have been primarily text based, I have little knowledge of how to create an effective graphical interface based program.
 - However, selection of more basic and widely documented GUI libraries, with the potential for tutorials being publicly available, will be beneficial.
- No knowledge of integrating external APIs into a program.
 - Similarly, I think with previous experience and knowledge of this company and their sites, I think there will likely be at least some documentation available, if not some that could be requested.

Suiting to a Computational Solution

The use of a computer to solve this problem will significantly benefit a user by giving them the ability to effectively manage a variety of flights on a single software, meaning there is a significantly reduced need for paper documentation of flights and fleets on the commercial side and less need for users to have to continuously oversee multiple different applications that provide flight alerts and updates on the personal side, saving time and other physical resources on both sides.

Additionally, a computational solution to this problem will be highly beneficial due to the potential of vast amounts of data and handling of the data being required. This is where integration of an API which provides this data will come into place, over the simply unrealistic approach of storing all global aviation traffic data locally. Sorting of this data is much more rapid and efficient than human sorting and calculation.

Computational Methods and Approaches

Thinking Ahead

Thinking ahead is the use of a computational solution in order to solve a problem, when provided with relevant information.

For my project, I am going to be coding in the Python language through the use of the IDE 'PyCharm'. I am doing this for many reasons. The main reason behind my choice of coding language is the fact that I have been taught and have practiced this programming language from the beginning of my secondary education and wrote in Python for my GCSE exams and in-lesson tasks; I am knowledgeable regarding the language syntax and requirements.

Variety of beneficial features regarding the PyCharm IDE:

- Debugging and Testing Tools.
 - Code navigation and error handling.
 - Unit testing with test driven development. When creating a PyCharm project, you have the option to create a virtual sandbox environment (meaning you can test sections of code with it being isolated from your main computer system; any errors will not affect system functionality).
- Supportive Features.
 - Code navigation and advanced code assistance (with provided prompts).
 - Direct database integration features (including SQLite, MySQL, PostgreSQL) with queries that can be run directly from the IDE.

My solution will also require the use of at least one database that will be hosted on phpMyAdmin (an open source, commercial grade RDBMS) locally and queried via SQL structures. I am doing this for many reasons. Furthermore, I will be querying the database(s) and tables within, through SQL query structures, as I currently have little but some experience with this through my A-Level course.

RDBMS - Relational Database Management System.

Variety of beneficial features regarding phpMyAdmin hosting:

- An open source RDBMS (Relational Database Management System)
 - Free for personal use.
 - Relational databases.
 - Data integrity and consistency through the use of primary keys (unique record identifiers in a single table) and foreign keys (used to establish a relationship between two tables and records within each).
 - High reliability and scalability.
 - High security.
- A commercial grade RDBMS.
 - It could be used in the future to expand this personal coursework solution into a commercial solution.
 - High reliability, scalability and well documented - well developed, rigorously tested and reliable sources of documentation and support with it being a well renowned commercial solution (used by companies including: Netflix, Amazon and Google).
 - High performance - a commercial RDBMS is heavily optimised for speed and efficiency with intelligent execution of complex SQL queries in the most efficient ways possible.
 - High security - robust security features with it being a vastly used commercial solution for world leading technology companies and governments (including: HMRC and the U.S. Department of Defence).

Possible IDE and Library Disadvantages:

The main disadvantage to the use of PyCharm is that it is known for high memory usage due to the high complexity of its features that are suited to high-level and complex development projects.

FreeSimpleGUI is a recreation of a well renowned open source library, PySimpleGUI, that has now been converted to a closed source library as it no longer provides free hobbyist license keys, and free versions being promptly removed from PyPi archives. FreeSimpleGUI is closely an older version of PySimpleGUI (prior to license keys being required) that has been developed to make it as close to the current version as possible. Therefore, all tutorials, documentation and resources, should still apply.

Abstraction

I am going to be using abstraction in order to simplify data entries and to simplify the graphical user interface (from 3D elements to 2D) that the user will visually see on their respective interface, both personal and commercial. Abstraction will also be used to hide irrelevant information and variables from users that aren't necessary, such as record IDs and also possibly simplifying acronyms that a non-aviation user may not understand, such as STA for Standard Time of Arrival, etc.

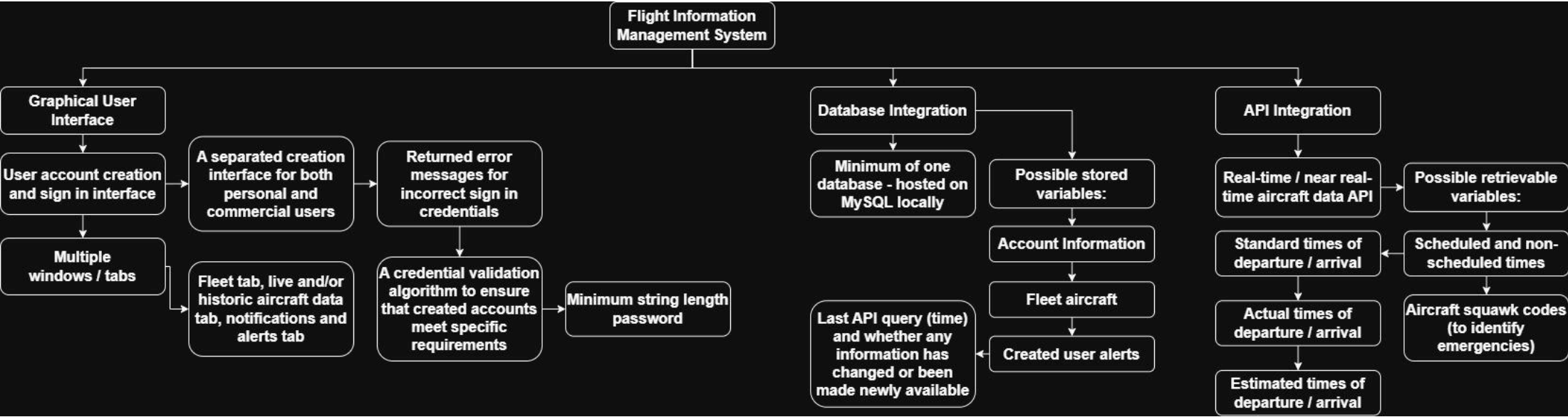
Ultimately this will save time and be more accessible for the user.

Decomposition Diagram

Decomposition is the process of breaking down a large or complex problem into smaller or more straightforward smaller problems, that are therefore much more manageable on a smaller scale and can be completed within different time frames.

For example, decomposition of a programming project would involve breaking it down into individual requirements or development sections, using computational methods to progress through each. Below shows my decomposition full structure diagram (breakdown) of my programming project and individual steps.

Decomposition is going to be used in order to break down the problem of having multiple software variations that are publicly available, especially for personal use flight information systems, since the majority of airlines have their own individual flight status apps; if you fly with multiple airlines frequently, say 3 or 4 airlines, then you will have to use each different app respectively. The process of decomposition will merge flight statuses and management into a single software development using computational methods, not only taking up less storage on a clients device, etc, but also making it easier to manage with having all relevant information in one model. This is all done through the use of computational methods to resolve a problem.



Heuristics

The baseline idea of heuristics is that the speed, regarding the development of the solution, is in preference over the perfections and optimisation of the solution (i.e. focussing less on aesthetics of the user interface and focussing less on performance capabilities of the solution). A heuristic approach or ideology brings the solution to a point where it is practical and functional when a particular solution is needed in a short time frame. Integration of shortcuts on the surface to focus on the main functions under the surface is key.

This will be implemented to prioritise API and data handling functions, GUI functions, etc, over aesthetic and non-essential features that can be added at a later date. Non-essential extras would be added when I am sure that all essential features are working to my standards.

Identifying Stakeholders

Direct (someone directly involved with the program, such as R&D) and indirect (someone not directly involved with the program, but may benefit from it).

Stakeholder	Direct / Indirect	Interactions / Uses	Availability (direct stakeholders)
Airlines	Indirect	Commercial use - Airlines will use this application to track and manage aircraft within their fleet while being able to view live and historic data regarding individual aircraft within their fleet (e.g. flight times, ages of aircraft). Airlines will be able to send (notification) alerts to their passengers regarding flight updates and receive alerts regarding aircraft in their fleet.	N/A - Indirect
Airports	Indirect	Commercial use - Airports will use this application to track and manage arrivals and departures into the airport, while being able to view past departures and arrivals as well as current and historic airport data (e.g. weather information). Airports will be able to send (notification) alerts to their passengers regarding general information updates about the airport..	
Passengers	Indirect	Personal use - Passengers will use this application to view the notification alerts from airlines and airports and can enter their upcoming flight information to receive alerts and to view estimated airport departure and destination arrival times, as well as information upon arrival, like arrival airport information (e.g. weather).	
Lee Spowart	Direct	A primary program tester for my project, who has industry experience in SQL development and cloud-hosted databases.	Weekends, evenings, occasional times in the week.
Jack Mackay	Direct	An additional program tester for my project, who has intermediate to advanced level skills and experience, particularly in procedural programming and the Python language.	A few hours in a working week.

I believe this is a valid range of stakeholders as it includes a range of different customers, who ultimately influence the design of the program. There is also the addition of direct stakeholders, who can support me in my design and implementation. A range of opinions across individuals who have different use requirements and additionally, different skillsets and experiences across different programming architectures.

Secondary Research - Existing Solutions

The purpose of research is to gather ideas about **what current clients and customers would look for in a software solution**. There are many methods to do this, including: interviews, questionnaires and you can even research existing solutions that may be missing wanted aspects which you could incorporate into a better solution. Below displays my researched stakeholders that have been chosen for their popularity, variety of features and the accessibility that will likely remain available for the duration of my project.

Existing Solution 1: FlightRadar24

[FlightRadar24](#) is an online website and app based, live flight tracking and information software with its primary function being a world map, with customisable overlays (such as: weather, air traffic control boundaries and airspace alerts) that displays real-time positions of aircraft as a basic ideology (*see Figure 1.1a*) based in Stockholm, Sweden.

Clicking on an aircraft icon on the world map displays a smaller window with current variables and information about that aircraft (airspeed, callsign etc)(*see Figure 1.1b, marker 1*) and provides further links to separate tabs that include all publicly available information regarding that individual aircraft and previous callsigns under an airline, therefore, you are able to not only see data about that aircraft, like age, country of registration, etc, but also data about previous flights under an airline.

Data can only be retrieved for aircraft that are currently / have been previously tracked under the umbrella of transmitters and receivers fed into FlightRadar24, or registered aircraft registrations.



Figure 1.1a



Figure 1.1b

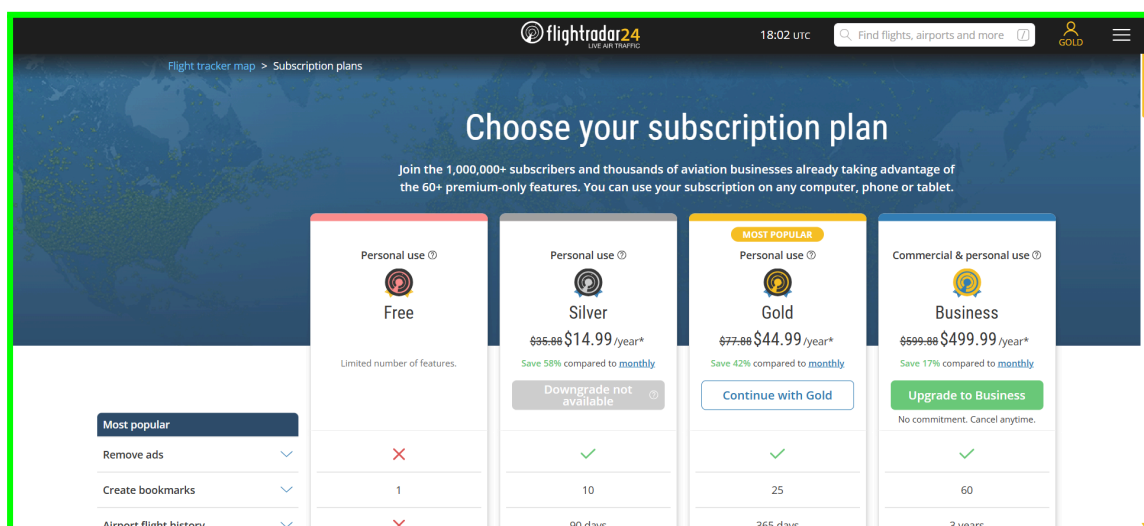


Figure 1.2

Index**Figure 1.1a**

1. Real-time moving positions of aircraft that update 5-10 times per second and are displayed through the yellow icons that vary in design based on the type of aircraft (wide body aircraft, drones, etc).
2. Additional map overlays:
red line - air traffic control boundaries.
Blue line - oceanic tracks.
3. Search bar that allows you to search across a range of fields regarding any: aircraft currently in live-flight, historic flights and registered aircraft registrations. Also any registered airports and airfields.
4. Toolbar including: settings for overall use of the website / app (1st), weather overlays (2nd), filters for aircraft displayed on the world map (3rd), widgets for aircraft (4th), global live-tracking playback (5th).

You can purchase varying subscription plans, with the more expensive plans offering a wider range of features (as shown in Figure 1.2, where the different plans and some of their features are displayed below). The most popular plans and the business plan are the most expensive.

FlightRadar24 also offers a business subscription (that can be used for both personal and commercial use, although expensive) and provides the user with the complete set of features, along with some being more directed to commercial use (including fleet overview and management).

Disadvantages to...:

- FlightRadar24s API returns values from a query in a JSON format, although this can be resolved through tools within Python that can convert the data.
- Non-basic plans can be expensive and basic plans don't offer a good range of features.
- The business plan is very expensive for small aviation businesses or for users who would benefit from a business plan but for personal use (non-commercial).
- Focuses more on the live flight tracking and information from a personal perspective over a commercial perspective in regards to fleet management, etc.
- Doesn't offer an alert system for your scheduled flights (delays, etc) or detailed alerts for a commercial perspective.

Advantages to...:

- FlightRadar24 offers an API alongside a sandbox "environment" where you can test your API integration through free queries but with fixed parameters and returns.
- Easy graphical and menu driven interfaces.
- Live data integration of not only flights but also for world map overlays and other airport and aircraft data, such as weather overlays and the new feature of viewing recent GPS jamming globally.

Ideas I could take from this solution:

I am planning to use the integration of FlightRadar24s API and the feature where you can download CSV files of historical in-flight data in my project to retrieve data about aircraft and routes to display information and statistics to users.

From FlightRadar24, I could use both the paid access (via purchase of input and output tokens) and free sandbox environment in order to retrieve the specific information I need to display to the personal and commercial interface. This could further advance my project as the integration of the API could be a challenge in regards to certain features, including queries, which I would have to work around if I want my software to check flight statuses, for mostly the personal perspective but could also apply to the commercial perspective, as I would need to query the API at consistent times that would be frequent enough for it to be effective (every week, 5 days, etc, for a certain period of time prior to an event being triggered) but not frequent enough for it to be unsuitable cost wise.

Language Consideration:

FlightRadar24 involves PHP server programming and storage of extremely large quantities of aviation data that is updated in real-time from the same tracking sources used by air traffic monitoring systems. Including: around 1.2 million aircraft, 300,000 flights, 8000 airports and 2000 airlines.

On the client side, the use of HTML and JavaScript is used.

I won't be using these languages as I am not hosting my program on a website.

Existing Solution 2: FlightAware

[FlightAware](#) is an online solution focussed on providing real-time updates (to the second) for flight number / registration statuses for any flight tracked under the network of ADS-B (Automatic Dependent Surveillance-Broadcast) receivers, which provide information on an aircraft:

- GPS location (including altitude).
- Ground speed.
- Other data categories transmitted between other ground stations and aircraft.

On the homepage of the website (**Figure 1.3**) you can enter any flight number, callsign or airport and it will display a basic live flight tracking map for that airport, including a real-time updated list of arrivals and departures, or an information interface (**see Figure 1.4a and 1.4b**) for flights, including:

- Flight Numbers and Callsigns
 - Expected departure time (hours and minutes).
 - The equivalent flight number / equivalent callsign.
 - Departing and arrival airport, plus a link to provide relevant information of that airport.
 - A text box alongside the estimated departure and arrival time, indicating if that flight is on time or delayed.

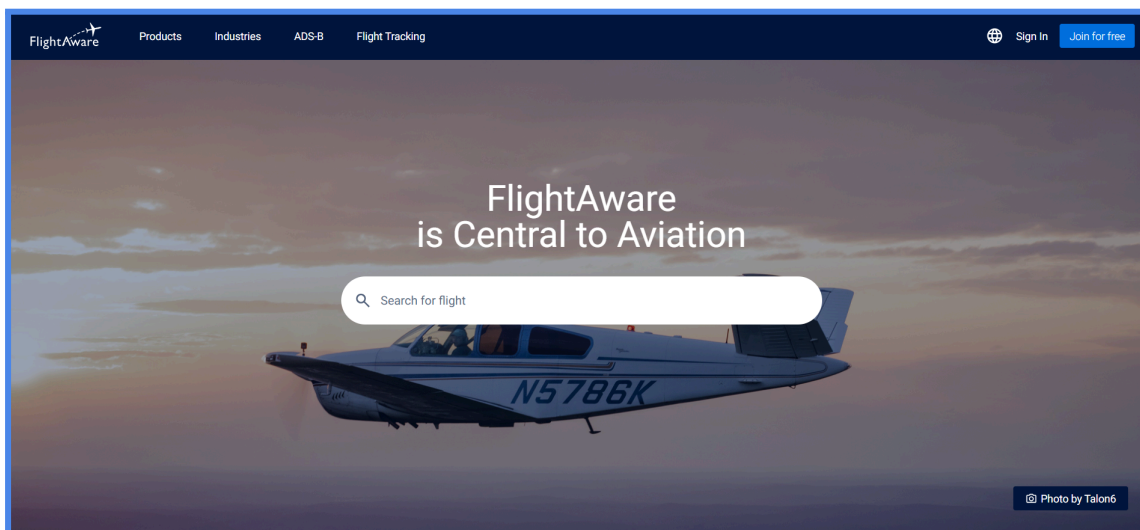


Figure 1.3▲

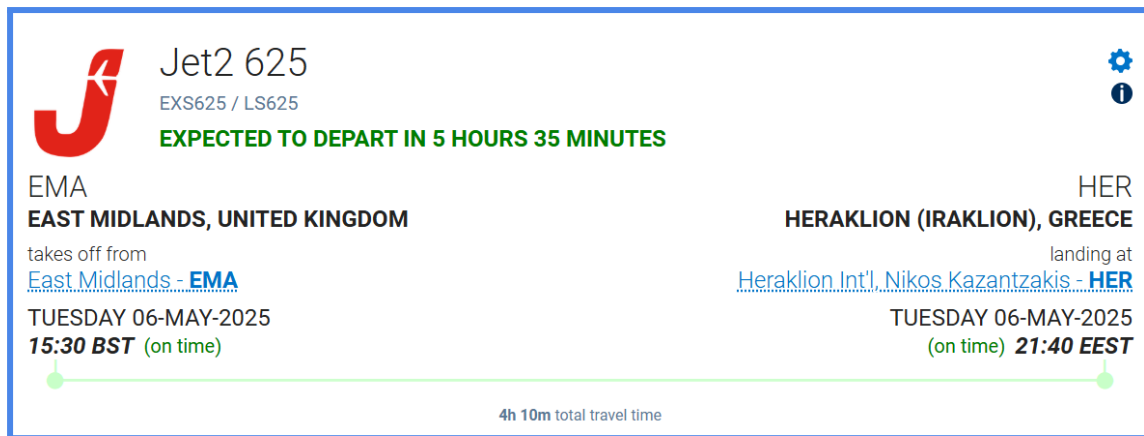


Figure 1.4a

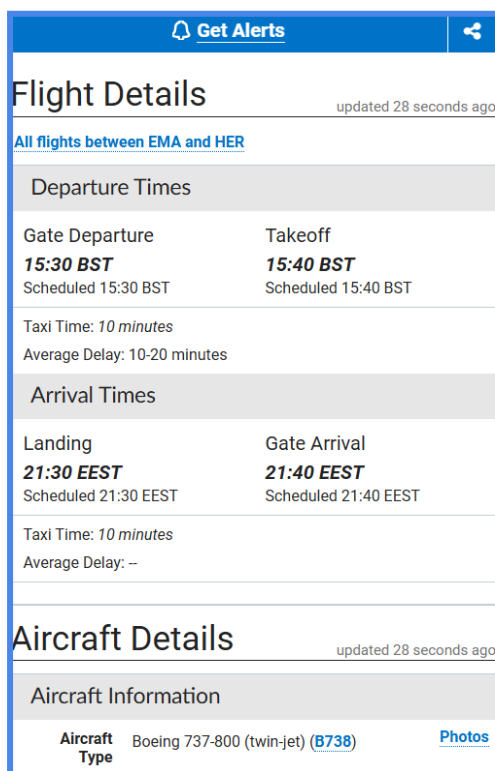


Figure 1.4b

Unlike other solutions, this solution provides details and information that could be considered more relevant to a personal use customer and would meet their requirements better - likely due to the fact that FlightAware has the main purpose of providing reliable flight statuses, rather than flight tracking or another aspect aimed towards a commercial standpoint. For example, to a personal user, **information displayed in Figure 1.4b**, could be more beneficial when displayed in more detail, such as, average delay time and taxi time.

Other solutions user interfaces may not be as accessible to some users as they can be hard to read and don't save the user time (which can often be key use of a computational method). Unlike this solution, which includes information that a user may have to otherwise process and calculate and is also friendly to those who have little to no experience in

aviation. Again, such as: average delay time and taxi time, since other solutions may not provide this directly and would have to be calculated (i.e. delay = the difference in minutes between the current time (of departure **or** arrival) and the initial time (of departure **or** arrival). However this kind of preliminary information could be avoided via other solutions due to its range of accuracy (e.g. an aircraft may travel from the same runway to the same gate on two occasions, but could be given a longer taxi route on the second occasion).

While not the direct focus, FlightAware do also offer a commercial solution ([GlobalBeacon](#)) which allows companies to manage their fleet of aircraft, while autonomously generating reports and logs and also distributing alerts for potential distress situations and active distress situations regardless of air traffic control borders.

Disadvantages to...:

- A large proportion of information that personal users would find more beneficial can be preliminary, meaning it is displayed with initial or early-stage data, which can make it appear inaccurate if it is not close to the true value at a final decision point.
- Personal flight status alerts are not available with a basic subscription and are very expensive for possibly infrequent occurrences (\$44.95 as the second most basic subscription that includes alerts).

Advantages to...:

- This solution provides information that could be considered more beneficial and useful to personal users.
- This solution has an easy to use, graphical user interface, which makes filtering for flights, registrations and airports easier to someone who has little to no experience in aviation.
 - For example: airports, airlines and aircraft can be identified either through their name, ICAO or IATA code. This can be confusing if there is a consistent variation between the required identification.
- The ability for both commercial and personal users to set up alerts for different flights, with commercial being able to access alerts for emergencies and personal being able to receive general status alerts.

Ideas I could take from this solution:

FlightAware has integrated a key idea I have in mind for my solution into their existing solution, which is having an easy to access user interface and also the ability to distribute alerts on a commercial side (regarding emergencies). Although it is expensive.

Language Consideration:

FlightAware is programmed using TCL (Tool Command Language). Additionally, it uses Python, Ruby and C++ for various aspects to this solution, including any API handling. TCL could potentially have been a good language for me to implement, especially due to its known simplicity but at the same time, power, common in rapid application development scenarios. Additionally its support across many different programming paradigms.

However, I do still have preference over Python, due to my current experience with the language and its characteristics, such as its scalability, efficiency, etc.

Existing Solution 3: Veryon

[Veryon](#) is a United States-based company that offers a wide range of logistical and analytical commercial solutions - from general and business aviation solutions, to commercial aviation solutions - with 75,000 users over 142,000 aircraft being managed. The company has three key products that can be integrated into different solutions required for different stakeholders and offer different features alongside each:

Veryon Tracking

An integrated maintenance and flight operations solution dashboard that can be used for either a single aircraft or across an entire fleet. The solution includes the key aspects of: maintenance tracking, inventory management, flight operations and the work centre that merges all of the below features together (**see Figure 1.5a and 1.5b**).

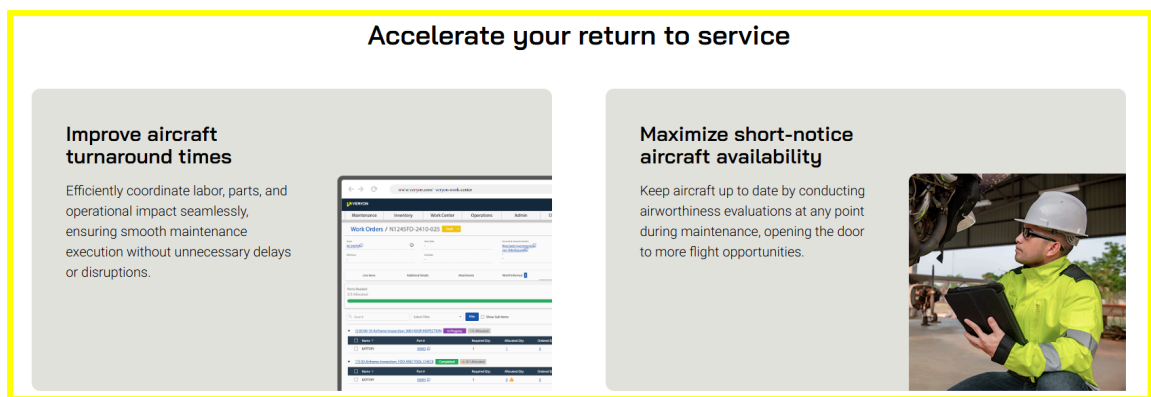


Figure 1.5a ▲

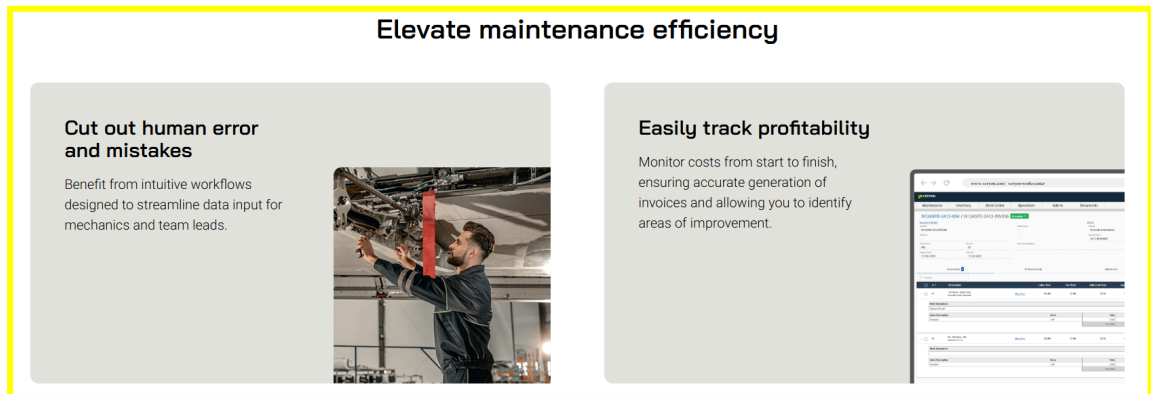


Figure 1.5b ▲

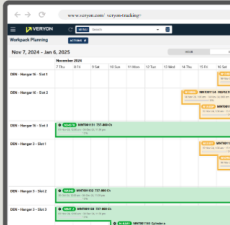
Veryon Tracking+

Veryon Tracking+ integrates different maintenance modules in order for customers to effectively manage and schedule maintenance for each individual aircraft across their fleet. This solution includes the following modules: fleet management, inventory management, flight operations and MRO management (a paperless maintenance system which allows users to schedule maintenance via a GUI calendar; reducing maintenance downtime, error and risk. **For other key details, see Figure 1.6a and 1.6b.**

Get the most out of your resources

Plan and monitor with ease

Interactive planning boards keep you up to date on the status of maintenance events and allow you to quickly reallocate work to alternate locations based on capacity.



Reduce technician downtime

Easily assign tasks to technicians with a live view of their current workload and skillset. Pre-allocate parts and tools to work orders so that everything they need is ready and waiting for them.

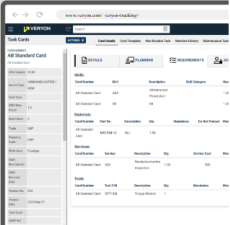
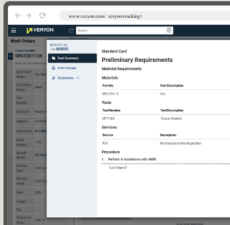


Figure 1.6a

Go paperless

Reduce errors, capture more data

Digital, authorable task cards let you modify feedback fields to allow only certain data types or inputs within set thresholds. You can also add feedback fields directly to the card to capture more insights from technicians.



Don't let record keeping slow you down

Our mobile app enables technicians to execute tasks entirely digitally, significantly speeding up the maintenance process. They can also request additional parts, tools, and services directly from within the app.



Figure 1.6b

Veryon Diagnostics

Veryon Diagnostics is Veryon's final solution which, in essence, is a maintenance focussed analytics dashboard for a custom fleet of aircraft and offers a further three key features: defect analysis - which uses an artificial intelligence platform to provide an overview on maintenance tasks and displays estimated statistics and data (**see Figure 1.7**), a reliability centre - used for efficiently providing estimates and suggestions for component replacement and repair and finally, guided troubleshooting on maintenance.

This system is estimated to reduce troubleshooting time by 75% and labour and parts costs for NFF issues (No Fault Found) by 35%.

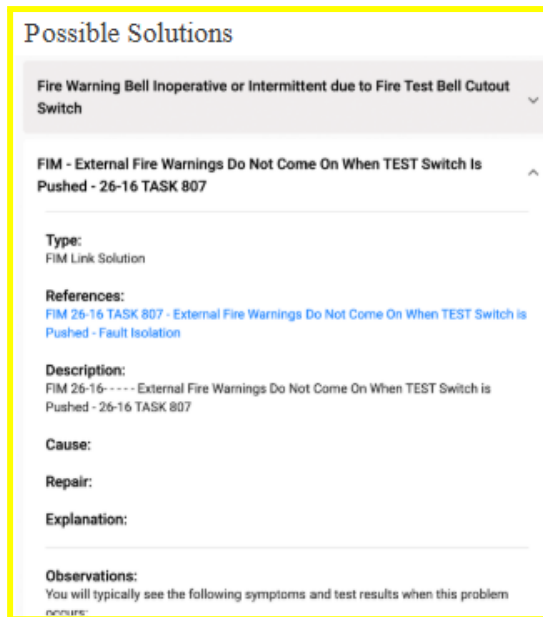


Figure 1.7▲

The company itself has a large number of key aviation partners globally (both manufacturers and airlines), suggesting its effectiveness within analytical and logistical departments and the ease of access to their key solutions. Some of their most well known partners include: Airbus, Lockheed Martin, United Airlines and British Airways.

However, at the moment, the company does not display an estimated cost of each solution but this is most likely due to a wide range of costs, depending on the requirements of specific companies (i.e. fleet size, fleet requirements, customisation, etc).

Disadvantages to...

- Even though Verson offers their wide range of solutions for many different aspects of analytical and logistical management, they don't currently offer a solution that integrates all three of their solutions into one single solution (maintenance, flight operations, inventory management and MRO management).
- Unknown estimated costs of the solution without booking a consultation.

Advantages to...

- Verson offers a wide range of solutions for many different aspects of analytical and logistical management within the industry.
- The ability to manage an entire fleet of aircraft through one solution, without having to individually manage each aircraft.

Ideas I could take from this solution:

The concept that you can group together aircraft into a fleet and manage them as an entire fleet - so the idea of being able to view statistics and data about the entire fleet - rather than having to individually view information about individual aircraft in a fleet that could have up to hundreds of models.

Language Consideration:

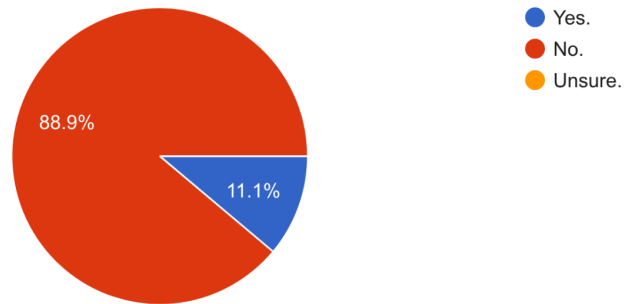
Veryon do not explicitly disclose what language their solutions or elements to their solutions, such as APIs, are coded and developed in.

Primary Research - Consumer Questionnaire

For my solution, I have conducted research via a questionnaire to potential stakeholders (personal use stakeholders in majority) on their opinions for different concept details and designs for my solution. The following displays the results:

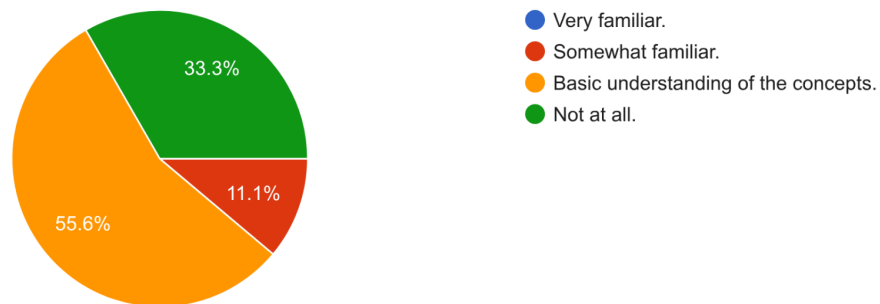
Do you personally know anyone who owns or operates a personal or commercial aircraft?

9 responses



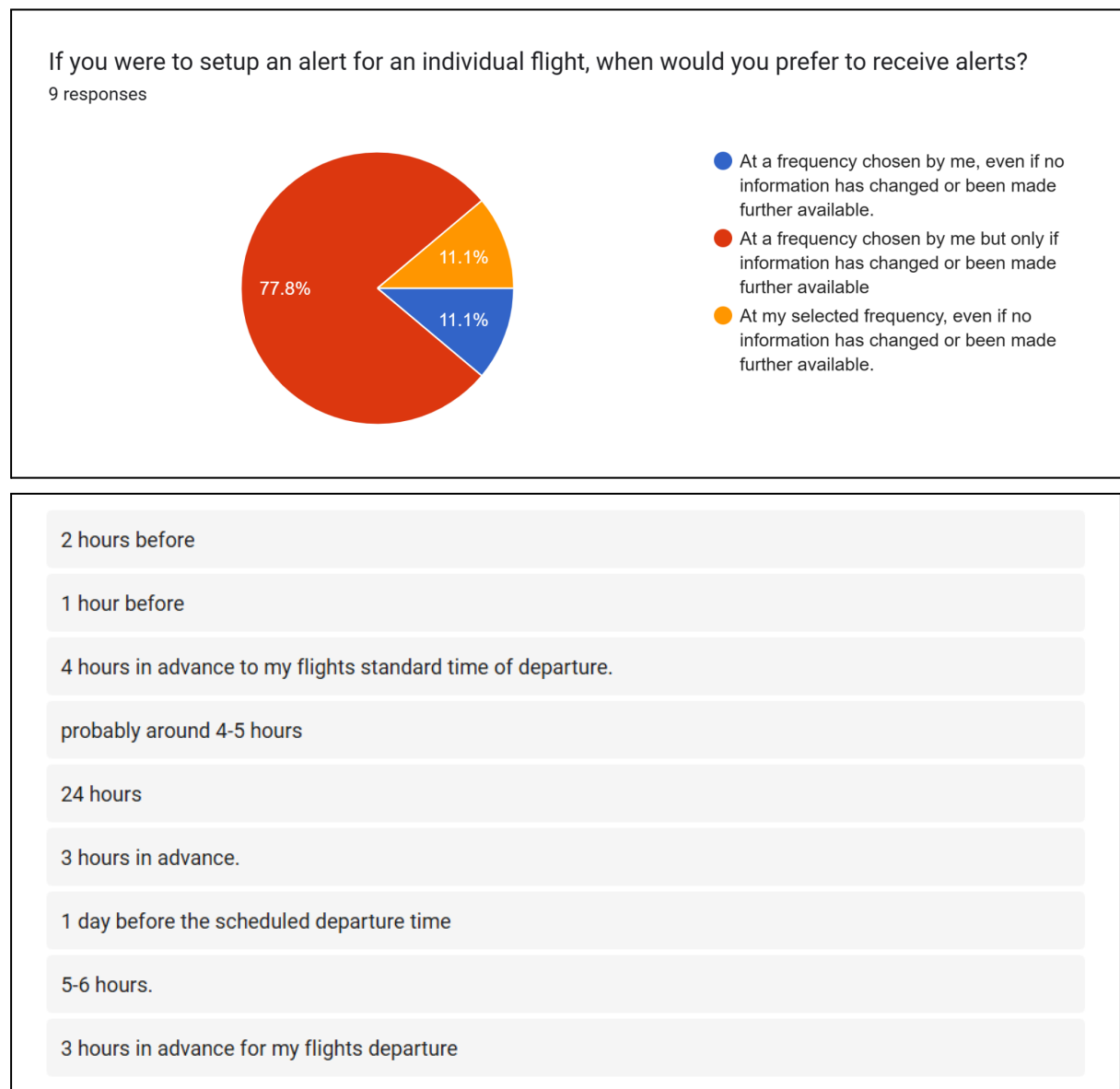
How familiar are you with the tools that help manage aircraft and flight information and/or the concepts behind them?

9 responses



These control questions briefly help me in determining what kind of knowledge around aviation, each individual and collectively has as a group. As we can see, 75% of respondents have a basic or somewhat familiar understanding of tools used in aviation information management, suggesting the responses should be valid.

Initially, I asked about solution preferences in order to gain ideas that I could then integrate into my own solution regarding specific aspects. These were the following results:

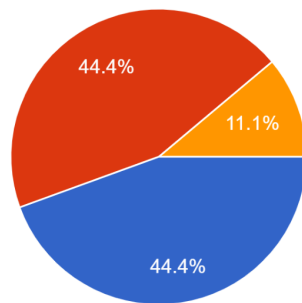


From these figures, the majority of respondents (77.8%) suggested that for the notification system, where you can create alerts for a specific flight, the frequency at which information is made available should be chosen by the user. However to save development time, I might choose this myself and revert back, should there be time (following my heuristic approach).

I calculated the median of these suggested values in order to find the midpoint of my values, while moving away from some outliers. **This left me with a median value of 4 hours**, which I think would be far enough in advance to begin pushing these alerts (considering you are typically advised to leave 2.5-3 hours at the airport, before your flight time).

If you were to setup an alert for an individual flight, would you want to receive alerts via:

9 responses



- 1. A notification page which you would manually check frequently prior to your flight for key information.
- 2. An email alert sent to you frequently prior to your flight (e.g. every 30 minutes) regarding key information.
- push notifications?

Following on from determining how far in advance users would want to receive alerts, I also needed to determine the method of receiving these alerts and the frequency at which this would occur. This showed an even split between wanting an in-app notification page and wanting email-based notifications, so therefore, I will likely implement a notification page system (which could be as simple as something command line driven) as I assume it would be more straightforward than setting up email alerts (which would likely require a fee from me).

Email Alerts

How frequently would you like email alerts sent to you regarding key information prior to your flight (e.g. every 30 minutes)?

5 responses

every hour

Every 6/7 hours

30-45 minutes

Every 1-2 hours

35 minute

Notification Page Alerts

How frequently would you like an in-"app" notification alerts sent to your alerts page regarding key information prior to your flight (e.g. every 30 minutes)?

4 responses

When something changes

Every 20 minutes

only when there's an update of information, otherwise only show message once

Every 30 minutes.

I initially planned on calculating a median of these suggested values, similar to that of the question asking when the user would want to start receiving these alerts (and determined 4 hours is effective), but I figured some of these responses could instantly be discounted since they either exceed that value or aren't suitable for the aviation industry, where flight times, delays, etc, can change down to the minutes before departure. Therefore, I think the best frequency would be one of the lowest. **Every 20 minutes should be suitable** out of these responses.

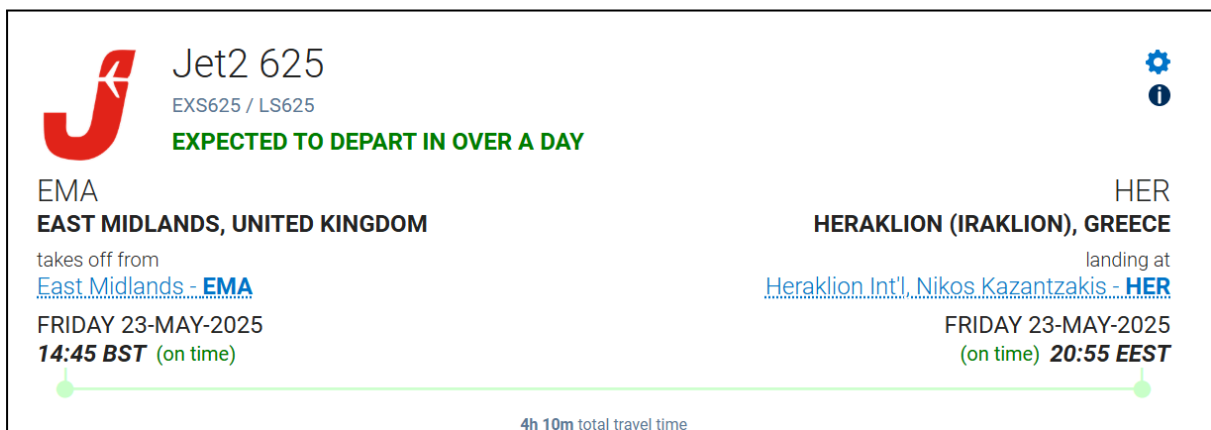
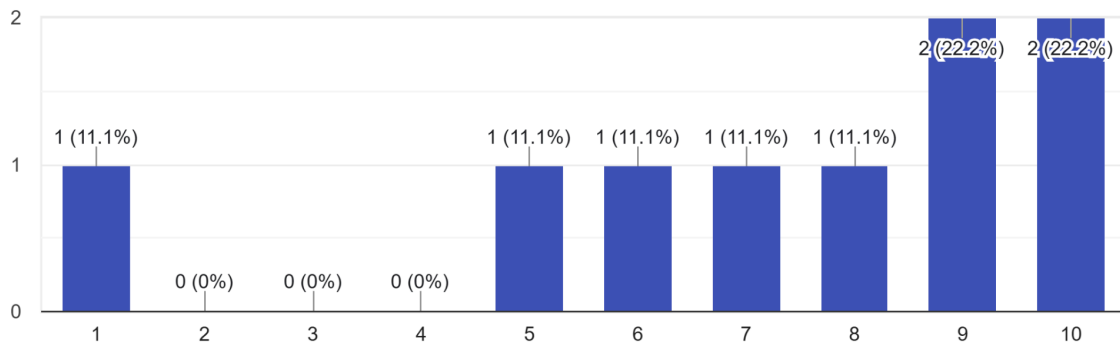
As part of my questionnaire, I provided respondents with some screenshots of the existing solutions I had found as part of my **Researching Existing Solutions**, and asked them to rate their design, providing feedback on positive and more negative aspects. Therefore, I know what I can potentially avoid / improve in my own program:

DATE	FROM	TO	AIRCRAFT	FLIGHT TIME	STD	ATD	STA	STATUS	
30 May 2025	Nottingham (EMA)	Heraklion (HER)	73H	—	14:45	—	20:55	Scheduled	▶ Play
27 May 2025	Nottingham (EMA)	Heraklion (HER)	73H	—	15:30	—	21:40	Scheduled	▶ Play
25 May 2025	Nottingham (EMA)	Heraklion (HER)	73H	—	17:10	—	23:20	Scheduled	▶ Play
23 May 2025	Nottingham (EMA)	Heraklion (HER)	73H	—	14:45	—	20:55	Scheduled	▶ Play
20 May 2025	Nottingham (EMA)	Heraklion (HER)	B738 (G-GDFR)	3:41	15:30	16:00	21:40	Landed 21:41	↓ KML ↓ CSV ▶ Play
18 May 2025	Nottingham (EMA)	Heraklion (HER)	B738 (G-DRTB)	3:29	17:10	18:04	23:20	Landed 23:32	↓ KML ↓ CSV ▶ Play

From this image, many respondents suggested that this simplified layout (the table layout with clearly labelled headings and columns) made the interface and its functions easily accessible. That is the main reason why I have selected this kind of interface, a tabular simplified interface (for ease of access). Although, some respondents agreed that they were unsure on the given terminology (such as ATD - Actual Time of Departure), so I think it would be a good idea to include descriptions of information and state the meaning of abbreviations if I think it may be uncommon.

(FA) Please rate the overall design (appearance, understanding of terminology, ease of access) of information regarding this flight:

9 responses



Now from the top image of the two, the ratings were more spread out but 6/9 feedback responses were positive and similarly to the above solution, many agreed that the information given was “direct and simple” and that “a lot of the terminology has been broken down into more basic wording,” supporting my idea of this in my own solution.

Comparing the two, I can see why some respondents agreed that this image appeared more cluttered as from a first glance, it can seem very packed with information but once you read the data through in a chronological order, the layout makes sense and is quite nicely organised. Separating departure and arrival information.

Make terminology simplified

Depends on how committed you are, you can split the app into regular passenger mode/nerd mode? (Maybe that's what your commercial/personal interface means?) Also I'm not too sure what the difference between alerts and notifications will be.

Good luck with your coursework you will smash this!!!

Finally, questioning any final feedback or ideas I could incorporate into my solution left knowing that ensuring clear terminology throughout should be a key feature. I did like the idea of a "nerd mode" which would include more advanced terminology, I would like to include this should I have time, or beyond the NEA deadline!

Language Choice

Through my previous experience and research into existing solutions, as well as consumer research, I plan to develop my program in Python. I think there is a wide range of support available in case I experience a difficulty in my development. Even though none of my researched solutions involve the use of Python, it is strongly accessible towards many aspects of coding (such as API integration), which I will include within my project.

I am going to take advantage of PyCharms debugging tools and unique features, such as the ability to integrate to-do lists within comments, etc.

See Secondary Research - Existing Solutions, for justifications of language choice based on individual existing solutions.

Solution Requirements

Based on my independent research conducted regarding specific stakeholders (individuals / groups / companies) that would benefit from my solution - I have comprised a table below which lays out the requirements of my solution in order for it to be functional in a suitable and working capacity.

See Analysis of the Problem >> Identifying Stakeholders for details and requirements of each stakeholder researched, etc.

List

Self-Sought Requirements:

- **Search tab.**
 - Drop-down menu to distinguish between searching for different instances (e.g. flights / airports).
 - The ability to search for a flight number or airport IATA (International Air Transportation Association) abbreviation. This supports user accessibility.
 - Equivalent error messages to be displayed if the flight can't be found.
- **Account Creation.**
 - Validation.
 - Password - minimum 8 characters.
 - Password - input and re-input must be identical.
 - Option to create a personal account.
 - Option to create a commercial account.
 - Verification
 - Company Name - must be recognised under the domains table of my database.
 - Database commitment of credentials, should all validation and any verification criteria be met.
- **Functioning event trigger elements.**
 - Some elements should have an assigned event trigger that, for example, correlates to a specific dictionary identifier (to link specific event triggers to specific validation / verification methods).
 - Data input methods (text boxes).
 - Improved ease of access for the user - ability to use the "Enter" key to progress, over a button.
 - Buttons.
 - Tabs.
 - Menu drop-downs.
- **Account sign-in.**
 - Validation.
 - Password entered is hashed, since stored passwords are already hashed (and is a one-way function so cannot be reverse engineered).
 - The email address entered has the corresponding hashed password.

- **Database Elements.**

- Credentials table - to store users email addresses and hashed passwords.
- Domains table - to store the top airline domain names, for later verification.
- Alerts table - to store user created alerts parameters, e.g. callsigns that the alerts are based upon, date/time of creation for use later on for when the alerts tab should begin displaying messages, etc.

Secondary Research-Driven Requirements:

- **API Elements** (50/50 Existing Solution Research-Driven

- Integration of the FlightRadar24 API, to search and retrieve data of both flight and airport instances.
 - FlightPositionsFull and FlightSummarFull functions.
 - Airport detail queries.

Primary Research-Driven Requirements:

- **Notifications tab.**
 - An area to view message notifications, on behalf of the alerts you have created.
 - Provides notifications at my determined frequency, from **Primary Research - Consumer Questionnaire**, every 20 minutes, from 4 hours prior to departure.
 - I think I will provide the notifications in a similar to a phone style notification centre, in small stacked panels, within the application.
- **Alerts tab.**
 - An area to view the alerts you have created.
 - Provides information about the alert.
 - Provides the option to delete/edit alerts, etc.

Table (Any Further Details)

Section	Ref	Sub-Sec	Requirements	Justification
Graphical User Interface	1	1a	User account creation and log-in window	There will need to be an option to sign-in as either a personal user and a commercial user (separately) and also an option to create a personal or commercial account. The personal account will require an email, username and password in order to sign up. The commercial account will require a commercial email (i.e. checking an email domain to ensure they are a commercial user: @jet2.com, @britishairways.com etc), username (being the company title) and a password in order to sign up. <i>Credentials regarding created accounts will be stored in a users database and credentials will be entered into input boxes.</i>
		1b	Log-in verification	Upon entry of a user log-in, the information provided into the input boxes will have to be compared to the credentials stored in the users database. Credentials that are incorrect when compared (i.e. an entered username with an incorrectly entered password for that username) will return an "incorrect credentials" message.
		1c	Multiple windows / tabs	Different personal and commercial aspects within the solution will need their own individual windows and tabs in order to cleanly separate data for efficiency and ease of access, such as an upcoming flight tab and a search for flights tab on a personal user interface. Research shown in Analysis of the Problem >> Primary Research - Consumer Questionnaire >> Questionnaire Comments and Feedback (Figure 1.8 and 1.9) , suggest that a simplified layout that explains and/or simplifies different aviation acronyms would be most accessible to personal users.
Solution Use	2	2a	Search tab	This will allow the user to search for specific flights or airports and this is also where they will be able to create specific alerts.

		2b	Live and/or historic aircraft and flight data	In order for personal users to be able to create alerts for individual upcoming flights; delays, etc, and for commercial users to be able to receive alerts for distress signals, delays, etc, live aircraft data will need to be provided to make this possible. I am doing this through the querying of the FlightRadar24 (or equivalent), which, when prompted by a set duration delay: will provide an updated estimated time of departure for an aircraft to be compared to the standard time of departure (e.g. every hour / 30 minutes prior to an aircraft's departure time.
		2c	Notifications and alerts	Both commercial and personal users will have access to an integrated notifications and alerts tab but will have slightly different aspects for each perspective. The commercial perspective will involve alert setups for fleet distress signals, delays, weather and possibly other issues an airline or airport would need to be made aware of. The personal perspective will involve alert setups for individual upcoming flights - that they can create through providing information. Users will also have the ability to setup email alerts as suggested (<i>Analysis of the Problem >> Primary Research - Consumer Questionnaire >> Other Questionnaire Comments and Suggestions</i>)
Database	3	3a	Access to a locally hosted phpMyAdmin database	In order to store data for my solution, it will require at least one database (that will be hosted on phpMyAdmin locally). There will be a wide variety of data that will need to be stored securely and accessible at any time, such as log-in details, fleet aircraft, etc, with the ability for the database to be easily expandable - which would be a key concept if my solution were to become commercially viable.
API Integration	4	4a	Access to a real-time flight information API	My project will require the access to a real-time or close to real-time API (likely to be the FlightRadar24 API) in order to provide accurate information about specific flights and aircraft for different aspects that (<i>labelled in Solution Requirements >> Ref 2 >> Sub-Sec 2a, 2b and 2c</i>).

Hardware and Software Requirements

Aspect	Requirements	Justification
Operating System	Windows 10 - Version 1809 (or later).	The requirements to each aspect I have displayed are both the minimum and recommended requirements for Python (language) and PyCharm (IDE). Currently, I meet all of these requirements at minimum at least, if not at the recommended. If these requirements change in the future, they can be adapted to. <i>I.e. installing a later Python version in the future.</i> However, I am unsure on what the total secondary storage requirement will be at the conclusion of my project (executable file size, IDE and library sizes, etc), although, I have a current SSD capacity that well exceeds the recommended storage requirements via my laptop storage capacity and storage capacity on a network area storage device (NAS).
CPU	- Modern CPU (minimum) - Multicore CPU (recommended).	
RAM	2GB available (minimum) 8GB available (recommended).	
Secondary Storage	3.5GB free (minimum) - SSD with 5GB free (recommended).	
Python Version	- Python 2 - Version 2.7 - Python 3 - Version 3.6 (or later).	

Solution Limitations

Time Constraints

With the coursework duration being only one year (from April 2025 to approximately April 2026), there is a limited time window to complete not only the documentation for my project but also the development of my project. Since the concepts I will be coding, such as integration of API, databases, etc, are new to me: I am unsure how long this could take to develop and that would be assuming no additional complications occur. Therefore I will be following a development ideology that integrates a more heuristic approach (creating a solution that is practical and functional over focussing on optimisation and appearance).

Graphics

With a more specific focus on the pure functionality of my project (ensuring that all functions work to my highest ability and that the success criteria has been met) there will likely be a reduced focus on the aesthetic design for my project due to this. Therefore, graphical interfaces will likely be basic and just above the minimum threshold for them to be functional. For example: being able to input data, output data, move simply between windows, etc.

API Integration

As of yet, I am unsure how difficult and/or time consuming the integration of the flight data providing API will be. Therefore, it may end up being more efficient for me to change the idea of using an API to provide data and to use my own test data instead (for queries) from downloaded CSV files via Flightradar24.

Hardware and Software Requirements

This solution will be designed for a Windows desktop interface and therefore, I am assuming that it will be inaccessible for non-desktop users (i.e. mobile, console, etc). It will also be untestable for non-Windows desktops (i.e. Linux, Mac, etc) due to the fact that I don't have access to these environments easily.

Success Criteria

The points below will label the success criteria I am aiming to meet for my solution requirements.

See Solution Requirements. Each point will be in an indentation format:

- **1 (GUI)**
 - 1a (User Account Creation and Sign-in Window).
 - (i) Users will need to be able to have the option to create either a personal or commercial account, so that there can be a clear distinction between specific features, such as interface attributes, validation requirements, etc. I will develop this base around my list of stakeholders, that has a split of personal users (e.g. passengers) and commercial users (e.g. airport and airline based stakeholders).

Simply defined with two separate windows.

Met 3.2 “Create Account” Interface

- (ii) Validation - a password cannot be less than 8 characters and the email address cannot be already associated with an existing account, ensuring account security across the platform when personal information may be available, as in line with industry and data protection standards (GDPR).

Measured through feedback to the user, such as warning messages, and not committing further within the program until validation requirements are met.

Met 3.2 “Create Account” Interface

- (iii) A password and username input box and a log-in button to submit the data values, with verification and error messages for credentials that don't meet the verification criteria. This improves accessibility to the user via a GUI interface and comes under the idea of usability features, to ensure validation is met confidently. This additionally stems from secondary research, with similar reputable services implementing this, and in good practice should be applied.

Most, if not all, text entry opportunities should have an equivalent specific GUI text input element.

Met 3.2 “Create Account” Interface

- (iv) The ability to insert the account information (with passwords hashed) into a credentials table and to query the account table in order to sign-in with the correct credentials (additionally ensuring data security from secondary research and general ideas following security practices), to allow the user to end the current interface “session” and re-log into the interface, without a loss of data (e.g. all alerts are kept, etc). This improves the user experience.

Minimum of an account credentials table, if not also other smaller linked tables to store additional information in a relational database structure (for efficiency).

Met 3.3 “Create Account” Execution, Validation and DB Commitment

- (v) When creating a commercial account, there should be some form of verification to ensure that the user is in partnership with some form of airline, to prevent improper use. Stakeholders, particularly commercial, will search for a strong level of security for their data, hence why this idea stemmed from stakeholder research. Additionally, from similar services researched in secondary research, I’m sure they would require you to verify your company status (again following good practices with security, user experience, etc)

This would likely be measured through a commercial “domains” table, maybe consisting of the top 25 or 50 global airlines, to cover most interactions. However, I can’t implement all known global airlines.

Met Commercial Only Aspects

- 1b (Tabs / Menu Options for Different Sections of the Solution). This allows for a cleanly separated interface layout, ensuring ease of access between users and aids in code development, for separate functions (improved maintainability, etc), as suggested and derived under usability.
 - (i) Your Alerts.

Met 3.7 “Manage Alerts” Tab

- (i2) Search Flights.

Met 3.6 “Search for Information” Tab

- (i3) Search Airports.

Met 3.6 “Search for Information” Tab

- (ii) Each tab or menu option should take you to a different interface with different features, depending on the title.

Met 3.5 Main Program Interface

- 1c (Data Inputs).
 - (i) Data inputs are collated into a dictionary titled “values”, which is outputted on the trigger of an event. Input boxes require their own individual index key to reference their output position within the dictionary. E.g. key = ‘-PASS-’ for password inputs.

Most, if not all input and output GUI elements should have a key identifier, via the GUI library “key” attribute. This will allow me to push updates to specific identifiable elements. This also aids in returning and comparing data, as strings are assigned to these identifiers within the main window dictionary structures.

Met 3.3 “Create Account” Execution, Validation and DB Commitment, 3.4 “Sign-In” Interface, 3.5 Main Program Interface, 3.6 “Search for Information” Tab, 3.7 “Manage Alerts” Tab, 3.8 “Notifications” Tab

- **2 (Solution Use)**

- 2a (Search Tab).

- (i) A list display of each aircraft within an airlines / companies fleet, to give commercial users an enhanced ability, over personal users, at managing multiple aircraft and flights.

Measured through the implementation of additional GUI elements, such as tabs, to facilitate this, and distinguishing between how alerts may be presented visually for commercial users. For example, instead of alerts updating at a given frequency, commercial stakeholders (**see *stakeholders list***) may need real time updates.

Unmet

- (ii) A search feature to be able to search for certain aircraft callsigns and airports, to retrieve information about them (linking to the main idea, emphasised in **primary research**, of using an API to return live aircraft data). This then follows into **2b**, so that the user can create an alert from that specific search.

Met [3.5 Main Program Interface](#)

- GUI display of the returned information, split into different sub-lists to put under headings for user accessibility.

Met [3.6 “Search for Information” Tab](#)

- The ability to show additional “nerdy” information regarding flights. This is an extra feature, which was suggested to me during my [Primary Research - Consumer Questionnaire](#), that I thought I could fairly easily implement.

Met [3.6 “Search for Information” Tab](#)

- 2b (Notifications and Alerts).
 - (i) A window that provides users with the ability to create a specific alert and also view, edit and delete active alerts. Improving **user accessibility** and embedding SQL functions within even triggered elements, so that users (both commercial and personal) do not have to manage their own alerts using complex SQL queries.

Met 3.7 “Manage Alerts” Tab

- (ii) A notification centre where users can read new and past notification alerts, almost like a message system, at the desired frequency and time until, as detailed under **Analysis of the Problem > Customer Research**, from questionnaire responses.

Met 3.8 “Notifications” Tab

- (ii.2) Potentially a database table to retrieve alerts between application sessions.

This will require at least one alerts table, to store the alerts and different attributes, such as date/time of creation, the flight number of the alert, etc.

Met API Integration, Query and SQL Functions

- (iii) A GUI element that informs users of flight emergencies, if required, as each element of information that can be shown should have a respective output element, to display to. From **secondary research**, many services allow you to receive emergency alerts for aircraft (e.g. FlightRadar24), however this would likely be a commercial oriented feature (**see stakeholders**), for logistics purposes, etc.

Unmet

- 2c (Commercial Aspects).
 - (i) An additional tab that allows commercial users (**see stakeholders**) to search for aircraft registrations, then give them the option to add them to a fleet of aircraft. This was an idea determined from my stakeholders list, as typically only those users will need to create an aircraft fleet.

Unmet

- (ii) The ability for this fleet to be sharable across users of the same company, in a shared location. Determines from my stakeholders list, with company offices and users being distributed, potentially globally, if not across different devices.

This will require at least one shareable table, to store alerts, different attributes and different flights within a fleet. Then, since commercial users are required to identify the company they work for in the verification process, access to that table will be assigned individually.

Unmet

- (iii) An additional tab that provides aircraft statistics to commercial users so that they can optimise their flights, such as mean route journey times, fuel economy, etc.

Potentially a separate “Aircraft Statistics” tab.

Unmet

- **3 (Database)**

- 3a (Access to a locally hosted phpMyAdmin database).

Met [3.1 Library and SDK Imports and Database Integrations](#)

- (i) The database will likely require multiple tables in order to store different categories of data separately and in an organised format. For example, as a minimum, separate tables will be needed to store user credentials with account information and user alerts that have been set up.
 - Credentials table.
 - Domains table.
 - Alerts table.

Met [API Integration, Query and SQL Functions](#), [API Integration, Query and SQL Functions](#), [API Integration, Query and SQL Functions](#)

- (ii) Encryption within the database via a hashing algorithm, therefore, passwords stay secure within the database but can still be cross-referenced for the sign-in, since hashing is a one way function. Additionally, salting could be implemented to enhance security even further.

This ensures account security across the platform when sensitive information may be available, as in line with industry and data protection standards (GDPR).

Measured through implementing different output lengths of hashing algorithms, to balance and test security and efficiency.

Met [3.3 “Create Account” Execution, Validation and DB Commitment](#), [3.4 “Sign-In” Interface](#)

- **4 (API)**

- 4a (API functions).

- (i) Integration of a live flight data API, in order to query data regarding different flights that users can set alerts and search for.

Measured through the ability for each user to have a specific or unlimited number of queries per time frame, to meet the APIs general and economic limitations.

Met [3.1 Library and SDK Imports and Database Integrations](#), [3.6 "Search for Information" API Integration](#)

- (ii) A fallback branch, should the API query fail due to availability, etc. The API should re-attempt to return data and if all fails, return an error message to the user. This supports and was derived from user accessibility and usability features, with fallback features to prevent program misuse.

This could be a simple IDE output via the IDE terminal.

Unmet

- 4b (API alternatives).

- (i) Should an API be unavailable for use at the time of development or too complex, regarding the project deadline, or any other issues: I have the alternative to download records of flight test data so that the system can still remain functionable. Again, being derived from user accessibility and usability features.

In this case, I think downloading a small number of JSON or CSV files, containing individual flights (10-20 flights) should be suitable.

Unmet (Not Required)

Design of the Solution

Decomposition

Decomposition already referenced in Analysis of the Problem > Computational Methods > Decomposition.

Throughout my project, decomposition will be used in order to not only break down a general problem into smaller and more manageable “sub-problems,” but also to make the solution more manageable, particularly beneficial with it largely following a modular design, **see Architecture**. This means that my approach to solving my defined problem will flow easier, as it allows for the organisation of separate “modules” and how much time I should spend on each, in order to complete my project.

Graphical UI Designs

The following images are my initially created graphical user interface designs for my solution, the index below will list what each image represents:

Figure 2.1a▲

An initial sign-in page (**top left**) where you select whether you wish to sign-in as a personal or commercial user, there will also be a button at the bottom of this page for the option to create a personal / commercial account. A personal account sign-in page (**top right**) and a commercial account sign-in page (bottom left) where you enter your email address and password, which is entered via the “Sign In” button at the bottom of the screen. The sign-in page will need to be able to censor your password as you enter it for privacy reasons.

Figure 2.1b▲

A commercial account creation window (**left**) and a personal account creation window (**right**). In order to create an account, you will require an email address, a password (which you will have to confirm) and your company name if creating a commercial account.

Any relevant validation or confirmation messages will be displayed via a GUI message, with red indicating an issue that needs to be resolved, and green indicating a valid progression onto the next “step.” **See Development of the Solution > 3.2 “Create Account” Interface or 3.2 “Sign-In” Interface.**

Figure 2.2▲

This will be the main design for my solution that overlooks all aspects that I am planning on hopefully implementing, with each different section being accessible via a tab system (*left*) or menu system (*right*). The main content for each section will then be displayed in a large canvas or text box, as seen in the figure, either in the centre of the window below the tabs or padded to the right of the window to the right of the menu.

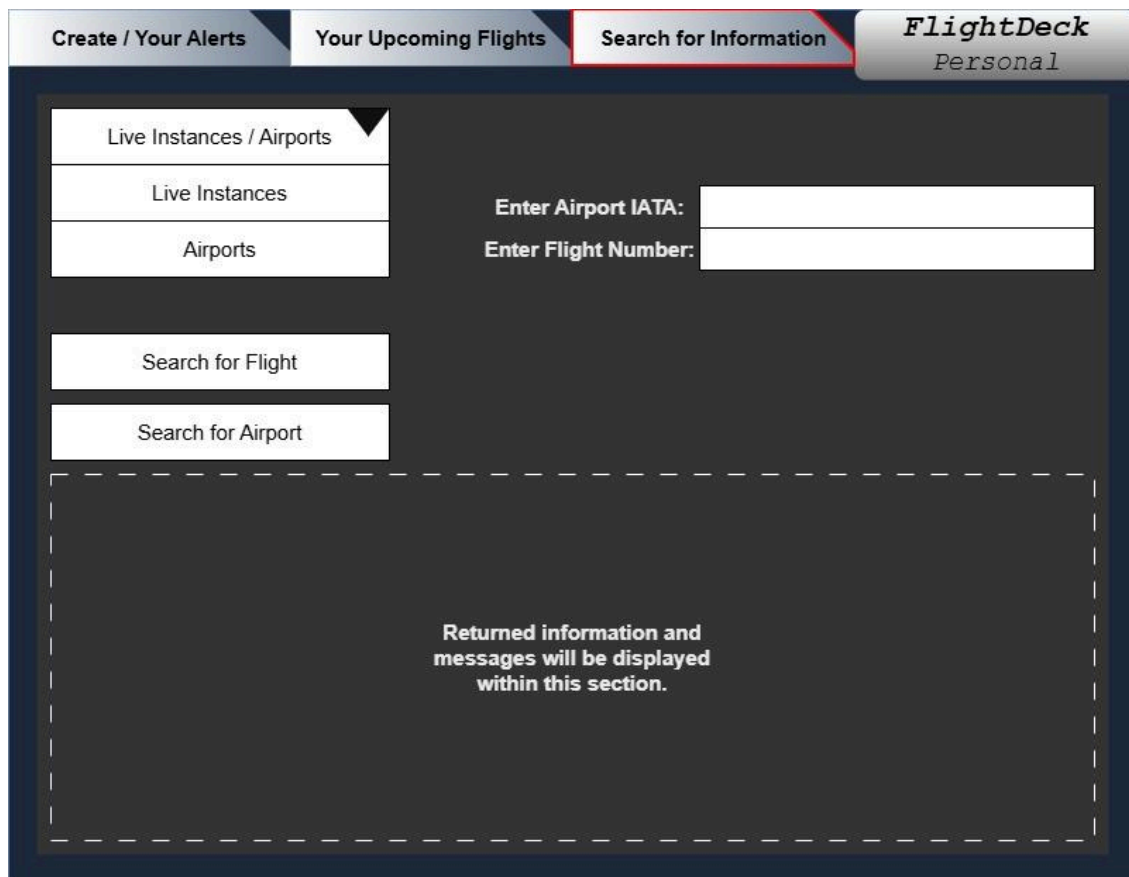


Figure 2.3▲

A later updated version of **Figure 2.2**, which will be the main interface layout for any selected tab ("Create / Your Alerts", "Your Upcoming Flights" and "Search for Information"). From the start of my GUI creation, I chose to stick with the layout similar to **Figure 2.2 (right)** purely for slightly more ease of development, as the FreeSimpleGUI elements are structured in a specific chronological order and some elements don't give the ability for you to provide a position discretion as a parameter.

Architecture

Architecture

My project has been developed with a modular architecture in mind, as it consists of isolated individual modules and functions within those modules. This is clear through the separation of physical modules and functions within the appearance of my GUI, **as shown in Graphical UI Designs > Figure 2.2**, with the separation of different tabs. It is also seen in the **Development of the Solution**, where different functions are within respective sub-routines, such as the different search functions under the “Search for Information” tab.

The use of a modular design means that many aspects could be repeatable, saving development time and improving efficiency.

Additionally, client-server architecture plays an important role in the backend side of my projects development (over visual aspects that revolve around modular architecture), in terms of API integration for data query and handling. Client-server architecture is involved in the following:

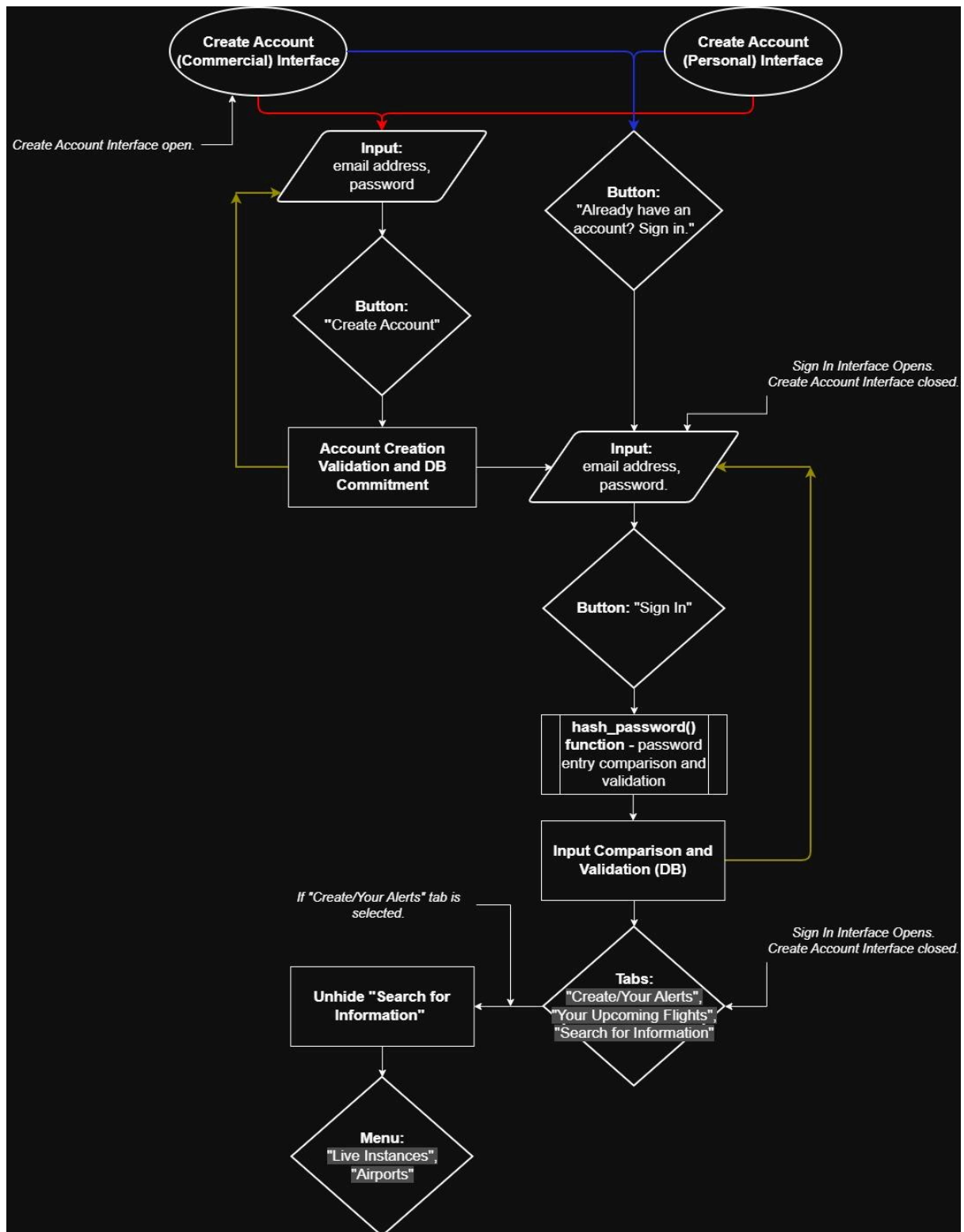
- **API Integration.**
 - Live and historic data queries.
- **Data Handling.**
 - Account details,e.g. Email addresses and database stored passwords.
 - Potentially for the management and use of user-defined and specific alerts.

Programming Paradigms

My project has been developed using the procedural programming paradigm, as I have the most experience in this paradigm and it is suited to my project, mainly with advantages in performance, which is key for this project, particularly with the return of live API data and SQL queries. Additionally, the use of procedural paradigms allows for reusable sections of code, which is common within my project, such as through reusable GUI output elements, etc.

In comparison to paradigms like OOP, they have an increased complexity and testing can be made more difficult with a more limited ability for program reuse. Although, it does favour data security through methods, such as encapsulation, where back-end code, variables, etc, is hidden from the user when unnecessary.

User Navigation



Red Connector:

Initial step of creating an account, without a prior account having been created.

Blue Connector:

Initial step of signing into an existing account.

Yellow Connector:

Return to a previous step due to an error, such as a validation error, missing data, etc.

Create Account Validation Rules:

- The password must be longer than 8 characters.
- The email address must not already be linked to an existing account.
- The password input and re-entry must be identical.

Sign In Validation Rules:

- The password entered will be hashed and must correspond to the hashed password stored against that email address in the database table.
- An existing account must have already been created with the email address entered.

The user is informed of any validation errors through GUI based error messages, that will appear below any invalid text entry or selection, etc, points.

Key Components

Referenced in Analysis of the Problem > Solution Requirements > List / Table (Any Further Details).

Data Flow

Where Data Comes From

User provided data inputs, such as password entries, etc. are provided graphically, unlike a client driven design. They are provided through different event-triggered GUI elements, like text boxes that are entered via button clicks.

Data inputs from these elements are mostly allocated to variables if they will be used individually for that client over a single session, such as variables that track what tab a user currently has open (as these will be reset per session, since you default on the account creation window when the program is executed initially). However, data inputs that require longer term storage for use over multiple sessions, such as user credentials, are stored in a database table (hosted on phpMyAdmin). This data can then be retrieved in a fairly straight forward manner through SQL queries, via mysqlconnector.

On the other hand, user provided data flows purely from the FlightRadar24 API, through Python queries. These include: live flight positions, estimated flight data, logistical flight data, etc.

How Data is Processed

Data is processed through different methods, including validation and verification rules (**referenced in Design of the Solution > Architecture > User Navigation**).

Where is Data Stored

User inputs are stored in an in-memory structure, a dictionary, which states the data input and variable or identifier name. This is useful so that it can be accessed at a later time in a session and is used, for example, in identifying what tab or interface the user is currently interacting with. By setting an identifier to a specific title, which can later be used in validation or branching purposes.

Referenced in Design of the Solution > Architecture > Where Data Comes From.

Structure

Use of Data Structures

Data Structure	Justification of Use	Some Examples of Use
Dictionary	Stores data, mainly event triggers, and can correspond data to variables using specific indices. Dictionaries are standard for the FreeSimpleGUIs event-handling architecture. Additionally, in terms of validation and other processes, it is beneficial to have a data structure that can not only store data, but also variables and triggers that can be assigned to them.	Data inputs from a text box corresponding to an event trigger, like a named button press.

Key Variables

Variable Name	Description / Justification of Use	Examples of Use
client (string)	Stores the FlightRadar24 API key, which is required to be referenced in a query statement when queries are made.	The get_searchFLT() function, which returns information regarding a specific flight number, requires this: <pre>def get_searchFLT(): With Client(api_token="***") flight = client.love.flight_positions.get()full(***) print(flight)</pre>
db (object)	Object used in the mysql.connector library and includes set parameters required to access my database (where and how it is hosted, the user access level it operates under and the database name). It acts as a prefix for mysql.connector functions.	Opening the cursor, which is a tool used to run MySQL queries and retrieve data from specific tables: <pre>Cursor = db.cursor()</pre>
visible (boolean)	A boolean attribute of GUI elements, that determines whether it is visible or invisible, by setting it to True or False, respectively.	It is used for user messages as a usability feature, rather than generating a message every time, the message is generated one and hidden when not required.

key (index key)	An attribute of GUI, event-triggered, elements, that allows specific elements to be identified, altered and used for validation purposes, it acts as a dictionary index key. Most, if not all, text outputs have a unique key that appears in the dictionary as their variable name and index	key = ['8CHAR_ERR_PER-'], which is the key used to identify the specific error message for the minimum of 8 characters validation criteria, when creating an account. The unique key is used with .update functions to make the message visible under certain conditions. Similar for other user messages.
enable_events (boolean)	This comes under the FreeSimpleGUI library as a parameter for specific elements, and allows interaction with an element to convert to an event trigger (with a unique index key in the dictionary).	This is used in elements, like tabs, so that when users switch between specific tabs, an event is triggered and can be used with validation and in hand with the "visible" variable, to show specific messages and elements, etc.
event_close_attempted_event (boolean)	A parameter included within the GUI.Window function (that forms the actual window, what is included within it, dimensions, etc), that allows the user to use the navigation bar to close that window on command.	Any created window will have this parameter (and set to True), so that under whatever circumstances, the window can be closed. Validation is also included within this to prevent accidental closures.
Data input variables for account creation (string)	All these are used to store data inputs and commonly are set to a dictionary index key, so that inputs from GUI elements can come straight from the dictionary to be stored in a specific variable (with use of the same key).	E.g. password_per, password_co, password_reinput_per, password_reinput_co, email_address_per, email_address_co, company_name. password_per = ['-PASS_ENT-']
Validation temporary variables (boolean)	Throughout some of the different validation processes, different conditions have corresponding boolean variables. This is used in cases where multiple conditions of validation might need to be met in order to progress and are useful to keep a track of this.	See below: under the account creation window (personal account creation tab), should the entered password not exceed the eight character minimum, that validation variable is set to False. Later on as part of a selection statement, should all account creation validation variables not be met, the credentials will not be committed to the database.
<pre> if active_tab == 'Personal': if len(password_per) < 8: window1['-8CHAR_ERR_PER-'].update(visible=True) valid_8char_per = False if password_per != password_reinput_per: window1['-PASS_MATCH_ERR_PER-'].update(visible=True) valid_passmatch_per = False </pre>		

timeout (integer)	In initial development, to set a delay for an action, I used the Python time library and "time.sleep**", however I later learnt that by using this, all events (including registering inputs and even drawing the window) become frozen. Therefore the built in function must be used.	timeout = 3000 <i>3000 being measured in milliseconds.</i>
----------------------	--	---

Validation Involving Key Variables

Many of my key variables **listed above** are used in validation. Validation is mainly included within the account creation and sign in window, but some methods are in the main window interface.

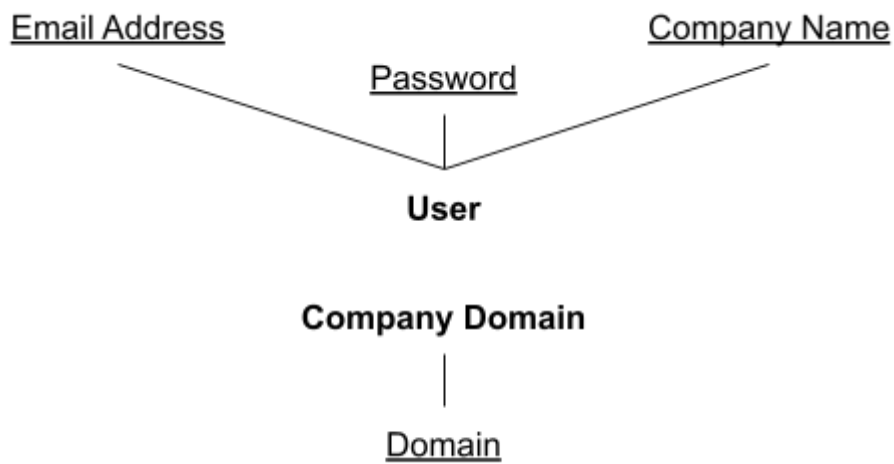
For example, in the account creation and sign in window, to create an account you must: enter a password that is longer or equal to 8 characters, enter identical values for the "password input" and "password re-input". The account creation stage requires the **above** validation booleans to be true (under a selection statement), as a way of ensuring that each individual validation has been met (since some stages may require multiple criteria to be met), before committing account credentials to the database. This prevents the need of requiring an option to edit/delete accounts, which would mean adding extra functionality on top of current functionality. Linking to the heuristic development of my program (**see Analysis of the Problem > Computational Methods > Heuristics**).

For the sign-in stage, validation is performed through a hashing algorithm. In the account creation stage, the entered credentials (specifically the password) are hashed for data security. Therefore, since this is a one-way function, during the sign-in stage, we must hash the entered password and compare that with the stored hash value.

Database Design (Data Table)

credentials	
`Email-Address`	Inputted String
`Password`	Inputted String (hashed)
`Company-Name`	Inputted String

domains	
`CompanyDomain`	String

Database Design (ERD's)

Post-Development (Function and Robustness) Test Plan*Function and robustness, validation, SQL queries, API queries, error handling, etc.***Follow in order.**

Test ID	Applies to which Section	Test Data / Action and Justification	Requirement and Justification	Expected Result
1a	Initial Program Execution	Execution of the program executable (via the IDE).	The account creation window should open.	The initial window that opens should be the account creation window.
1b		Check Terminal.	Text confirmation (via IDE terminal) that the database connection, via the msqlconnector library, has been successful.	"DB - Connection Successful" should appear in the terminal.
2b	Account Creation	Press the input button or "enter" key, with no values entered into any input opportunities, to trigger validation criteria for testing, ensuring false entries cannot be made, which may break further program logic.	An error message should appear, asking you to enter valid inputs, ensuring that the user is given feedback when validation criteria are not met, and reasoning.	"Password must be at least 8 characters long." <i>OWTTE</i> "Passwords must match." <i>OWTTE</i>
2c		Press the input button or "enter" key, with a password less than 8 characters entered (identically into the password input and re-input). This is to trigger validation criteria for testing.	An error message should appear, asking you to enter valid inputs (at least 8 characters long). Inputs should be also cleared. This ensures user feedback.	"Password must be at least 8 characters long." <i>OWTTE</i> All inputs cleared.
2d		Press the input button or "enter" key with a password greater than 8 characters (but unidentical password input and re-input). This is to trigger validation criteria for testing.	An error message should appear, asking you to enter valid inputs (password input and re-input identical). Inputs should be cleared. This ensures user feedback.	"Passwords must match." <i>OWTTE</i> All inputs cleared.
2e		Press the input button or "enter" key with fully valid inputs (password greater than 8 characters and identical password input and re-input). This is to trigger validation criteria for testing.	A message should appear, indicating that validation has been "passed" and your account has been created. The sign-in window should open. This ensures user feedback.	"Account creation successful" <i>OWTTE</i> All inputs cleared. The sign-in window should open.
3a	Account Sign-In	Press the input button or "enter" key, with no values	An error message should appear, asking you to enter	"Account does not exist or incorrect credentials" <i>OWTTE</i>

		entered into any input opportunities. This is to trigger validation criteria for testing.	valid inputs.	
3b		Press the input button or "enter" key, with an incorrect password entered. This is to trigger validation criteria for testing.	An error message should appear, asking you to enter valid inputs. This ensures user feedback.	"Account does not exist or incorrect credentials" <i>OWTTE</i> .
4a	"Search For Information" Tab	Enter nothing into the data entry box and press the "Search" button or "enter" key. This is to trigger validation criteria for testing.	A validation statement should be returned, prompting re-entry. This ensures user feedback.	"Flight number cannot be empty" <i>OWTTE</i> .
4ai		Enter a flight number into the data entry box and press the "Search" button or "enter" key.	An API statement should successfully be returned. This ensures the API works successfully and that GUI returns and logic works correctly.	Statements of "Flight_Positions" and of "Flight_Summary" returned graphically.
4b		Enter a flight number into the data entry box, check the "Show Stats for Nerds" checkbox, and press the "Search" button or "enter" key. This ensures /	An API statement (with additional attributes) should successfully be returned. This ensures extra features work as expected, adding more depth to my program.	Statements of "Flight_Positions" and of "Flight_Summary" returned graphically, with additional attributes.
4c		Enter an airport ICAO/IATA code into the data entry box and press the input button or "enter" key.	An API statement should be successfully returned.	One statement of "AirportsFull" returned graphically.
4d		Enter a flight number into the data entry box, check the "Create An Alert For This Flight" checkbox, and press the "Search" button or "enter" key. The justification of entering a flight number is to store that data within the "alerts" table of my database.	The alert record ("alerts" table) should be created and a message should appear, indicating an alert has been created, which will later be used to retrieve alerts (e.g. during different user sessions, where a user's created alerts must be fetched each time) and update them. This will also be needed to store information, such as departure time, which is used to determine when alerts need to be pushed.	An alert with an "AlertID", "EmailAddress", "FlightNumber", "DepartureTime", "CreationTime" and "LastAlerted" attributes should be created. The text entry box should automatically clear (after searching) and a message should appear, indicating an alert has been created, detailing the flight number.
5a	"Manage Alerts" Tab	Select the "Manage Alerts" tab.	An SQL query should be performed and your created	An SQL query should be performed and your created

			alert should be displayed graphically. This supports user accessibility through string splitting and concatenation, allowing me to add labels to data.	alert should be displayed graphically, detailing the flight number, creation time, etc.
5ai		Enter nothing into the data entry box and press the "Delete" button or "enter" key. <i>Validation test.</i>	A validation statement should be returned, prompting re-entry, preventing the user from entering invalid data which would otherwise break program logic (e.g. concatenating empty strings would leave labels with blank results).	"Flight number cannot be empty" <i>OWTTE.</i>
5aii		Enter a flight number (which an alert has not been created for) into the data entry box and press the "Search" button or "enter" key. <i>Validation test.</i>	A validation statement should be returned, prompting re-entry, preventing invalid entry of data.	"You can only delete alerts you have created" <i>OWTTE.</i>
5b		Enter the alert flight number (returned previously - see above) into the "delete alert" data entry box and press the "Delete Alert" button or "enter" key.	The alert record ("alerts" table) should be deleted and an indication should appear that the alert has been deleted. This allows for customisation of alerts and prevents cluttering of the notification interface.	The text entry box should automatically clear (after pressing the "Delete Alert" button or equivalent) and the alert (displayed previously - see above), disappear.
6a	"Notifications" Tab	Select the "Notifications" tab.	An SQL query should be performed and your created alerts should be displayed graphically (supporting user accessibility).	An SQL query should be performed and your created alerts should be displayed graphically, with different outputs dependent on whether the flight is prior to departure, how prior to departure, in flight, etc.

Usability Test Plan

GUI navigation, layout, feedback messages, etc.

Follow in order.

Test ID	Applies to which Section	Test Data / Action	Requirement	Expected Result
1a	Initial Program Execution	Execution of the program executable (via the IDE).	The account creation window should open.	The initial window that opens should be the account creation window.
1b		Attempt to quit the program using the GUI quit "cross" box (top navigation bar).	The user should be able to quit the program without having to return to the IDE.	Same as requirement, with a confirmation action appearing.
1c		Attempt to minimise the program using the GUI minimise box (top navigation bar).	The user should be able to minimise the program window.	Same as requirement.
2a	Account Creation	Attempt to select and enter values into any (input) text box elements.	For text data entry elements to be accessible and work as intended.	The cursor indicator should appear within any text box elements, allowing data entry.
2b		Attempt to press any button elements.	For button elements to be accessible and work as intended.	All button elements should act as an event trigger, so further development into the program should occur.
3a	Account Sign-In	Attempt to select and enter values into any (input) text box elements.	For text data entry elements to be accessible and work as intended.	The cursor indicator should appear within any text box elements, allowing data entry.
3b		Attempt to press any button elements.	For button elements to be accessible and work as intended.	All button elements should act as an event trigger, so further development into the program should occur.
4a	"Search For Information"	Select the "Search For Information" tab.	The "Search For Information" tab selector should become	Same as requirement.

	Tab		highlighted.	
4b		Select the "Search For Information" tab.	No text input elements should appear until you select a "Search for Flights / Airports" drop-down option.	<i>Same as requirement.</i>
4c		Right click the "Live Flights / Airports" drop-down menu.	Drop-down options appear for different entities to search for, which correspond to different API query functions.	The "Flights" and "Airports" option appears.
4d		Select the "Flights" drop-down menu option.	The flight's data entry box and search confirmation button appears. The airport's data entry box and search confirmation button disappears	The data entry box appears alongside "Enter a Flight Number:" and the "Search for Flight" button appears. The airport's data entry box and search confirmation button disappears.
4e		Select the "Airports" drop-down menu option.	The airport's data entry box and search confirmation button appears. The flight's data entry box and search confirmation button disappears.	The data entry box appears alongside "Enter an airport ICAO/IATA code:" and the "Search for Airport" button appears. The flight's data entry box and search confirmation button disappears.
5a	"Manage Alerts" Tab	Select the "Manage Alerts" tab.	The "Manage Alerts" tab selector should become highlighted.	<i>Same as requirement.</i>
6a	"Notifications" Tab	Select the "Notifications" tab.	The "Notifications" tab selector should become highlighted.	<i>Same as requirement.</i>
7a	General	Sign-in as a commercial user (may require creating an account).	The company logo should be displayed on the main interface, under any tabs that are interacted with.	<i>Same as requirement.</i>

Algorithms

Included screenshots are from my decomposition diagram, showing what aspects are met. Final implementation is located under individual sections of Development of the Solution.

Pseudocode Interpretations

Account Creation and Separate Sign In

Involves the data collection for forming an account record and respective data entry points, allocating dictionary locations (since the data needs to be stored for many purposes and works in unison with the GUI library - FreeSimpleGUI).

```
if values["-OPTION-"] == "Commercial" then
    get_basic_create()
```

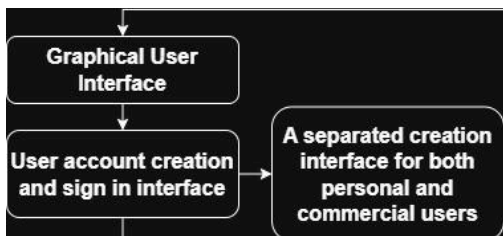
```
if values["-OPTION-"] == "Personal" then
    get_commerce_create()
```

Basic Account Creation:

```
procedure get_basic_create
    print("Create Account")
    values["-PERSONAL_EMAIL-"] = input("Email Address: ")
    values["-PERSONALL_PASSWORD-"] = input("Password: ")
    values["-PASSWORD_REENTRY-"] = input("Re-enter Password: ")
endprocedure
```

Commercial Account Creation:

```
procedure get_commerce_create
    print("Create Account")
    values["-COMPANY-"] = input("Company Name: ")
    values["-EMAIL_CO-"] = input("Email Address: ")
    values["-PASS_ENT_CO-"] = input("Password: ")
    values["-PASS_REENT_CO-"] = input("Re-enter Password: ")
endprocedure
```



Account Creation Validation and Error Messages

Validation criteria for the account creation interface, with each having its own validation boolean variable in order to use for later stage validation if needed. The entered password must be at least 8 characters long, matching the password re-input.

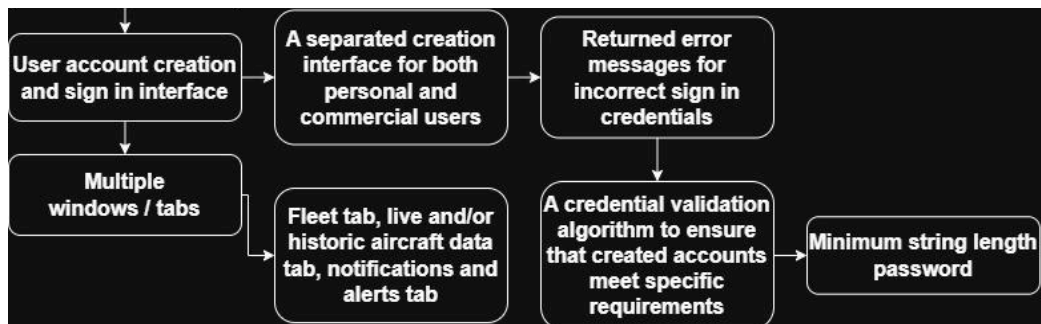
```

valid_8char_per = True
valid_8char_co = True
valid_passmatch_per = True
valid_passmatch_co = True

if active_tab == "Personal" then
    if len(password_per) < 8 then
        window1['-8CHAR_ERR_PER-'].update(visible=True)
        valid_8char_per = False
    if password_per != password_reinput_per then
        window1['-PASS_MATCH_ERR_PER-'].update(visible=True)
        valid_passmatch_per = False

elseif active_tab == "Commercial" then
    if len(password_co) < 8 then
        window1['-8CHAR_ERR_CO-'].update(visible=True)

        valid_8char_co = False
    if password_co != password_reinput_co then
        window1['-PASS_MATCH_ERR_CO-'].update(visible=True)
        valid_passmatch_co = False
  
```



Sign In Validation

This process occurs during the sign-in stage and confirms that an account email address matches the password entered (that credentials are valid). However, since the password is entered into the database as a hashed value, the entered password on sign-in also must be hashed, if identical, the hash values should match.

```
password_check = sign_values["-PASSWORD-"]
```

```
email_address_check = sign_values["-EMAIL-"]
```

```
hashed_password_check = hash_password(password_check)
```

```
Cursor.execute(creds_check, (email_address_check, hashed_password_check))
```

```
endprocedure
```

```
result2 = Cursor.fetchone()
```

```
If not result2 then
```

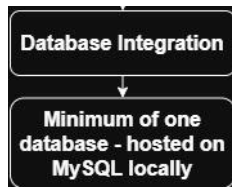
```
    ...
```

```
else
```

```
    ...
```

Database Integration

```
db = function mysql.connector.connect(
    host = "localhost",
    user = "root",
    password = "",
    database = "flightdeckdb"
)
endfunction
```



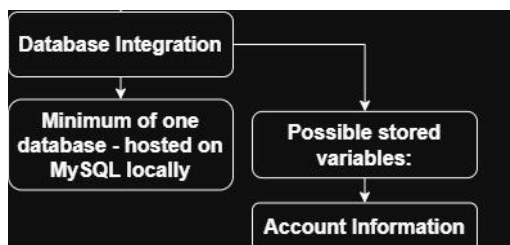
Database Commitment with Validation

This is my proposed plan for committing credentials to my database, after ensuring validation criteria are met. Passwords being committed to the database will be hashed.

```
if (active_tab == "Personal" and valid_8char_per and valid_passmatch_per) or (active_tab
== "Commercial" and valid_8char_co and valid_passmatch_co) then
    hashed_password = hash_password(password_per or password_co)
```

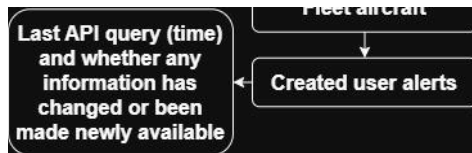
```
exist_check = "SELECT `Email-Address` FROM credentials WHERE `Email-Address` = %s"
Cursor.execute(exist_check, (email_address_per or email_address_co,))
result1 = Cursor.fetchone()
```

```
if not result1 then
    sql = "INSERT INTO Credentials (`Email-Address`, `Password`, `Company-Name`)
VALUES (%s, %s, %s)"
    values = ((email_address_per or email_address_co), hashed_password,
company_name)
    Cursor.execute(sql1, values)
    db.commit()
```



```
if main_values["-ALERTCHK_FLT-"] then
    ExistAlert1 = "SELECT `EmailAddress`, `FlightNumber` FROM alerts WHERE
`EmailAddress` = %s AND `FlightNumber` = %s
```

```
Cursor.execute (ExistAlert1, (Email, flight_number))
ExistAlertResult1 = Cursor.fetchone()
```



API Integration

```
from fr24sdk.client import Client
client = Client(api_token)
```

API Queries***Airport Data Query***

```
procedure get_searchAPT(SearchEntApt)
    airport = client.aiports.get_light(code = SearchEntEpt)
    return airport
endprocedure
```

Flight Data Query

```
procedure get_searchFLT(SearchEntFlt)
    flight_positions = client.live.flight_positions.get_full(flights = [SearchEntFlt])

    flight_summary = client.flight_summary.get_full(
        flights = [SearchEntFlt],
        flight_datetime_from = TimeFrom.isoformat(timespec = 'seconds'),
        flight_datetime_to = TimeTo.isoformat(timespec = 'seconds'))
    return flight_summary, flight_positions
endprocedure
```

Usability

GUI Designs

See Graphical UI Designs or Development of the Solution > 3.2 “Create Account Interface” onwards.

Usability Features

Simplified GUI Elements and Back-end Functions

All GUI elements are, in theory and practice, easily accessible to the user through contrasting colour schemes (dark colours, in this case darkened navy, against light colours, white). In terms of back-end functions, such as validation rules, account credential alterations, etc, pre-programmed messages are provided to the user (via GUI elements) so that they don't feel “left out” of what is happening behind the scenes, which is beneficial at stages where many events may be triggered that could confuse the user (e.g. window switches, refusal of account creation due to validation mismatches).

Pre-programmed messages are colour coded in a logical sense, again to prevent confusion of the user and to ease access further. Green messages show successful movement of the user through the program and successfully completed events (e.g. initial sign-in, **see Development of the Solution > 3.4 “Sign-In” Interface**). Red messages show events that might require the user's attention, such as failed validation upon account creation (**see Development of the Solution > 3.2 “Create Account” Interface**).

Additionally, see Design of the Solution > Architecture > Data Flow, for examples of when these messages may be needed and the response to them in program logic, etc.

Abbreviations, as suggested from my **Analysis of the Problem > Primary Research - Consumer Questionnaire**, are to be simplified and explained.

Data Entry

Data entry is, again, simplified and is clearly stated to the user, what should be entered in different fields. All entries of data are asked in a way that would seem familiar to a user (that provides correspondence to other commonly used programs, applications and interfaces), for example: examples of branching through an option selection is simplified via labelled drop down menus or clearly separated tabs, text entry boxes label what should be entered and are of a pale colour to stand out from the dark background.

Buttons and Navigation

Similarly to the above, in terms of a familiar interface and ease of access, the option of using physical buttons (again, clearly outlined and labeled) works hand in hand with common keyboard prompts, like “Enter / Return” keys. Also, a typical navigation bar remains at the top of my interfaces (both in full screen and window dimensions), including at least an “X” icon to quit the application.

Confirmations and Data Re-entries

Confirmation of event triggered actions and re-entry of data is simply a discreet method of validation and verification to ensure a user actually wants to carry out that action. For example, should the “X” quit application icon, on the navigation bar, be pressed, a pop-up window appears to ask the user to press another button to ensure that is the desired action.

An example for data re-entry, would be during the account creation stage, where you are required to input your email address, password, and are then asked to re-enter your password. This acts as a method of validation as if they aren’t identical, an error will be flagged and you will be asked to re-enter all data.

Usability Features from Research

See Analysis of the Problem > Secondary Research - Existing Solutions > Given Solution...

Extra details can be found above under Usability Features. This is only a summary of comparisons:

- Simplified GUI Elements and Back-end Functions
 - Exist Solution 2: FlightAware
 - Exist Solution 3: Veyon
- Data Entry **and** Confirmations and Data Re-entries
 - Exist Solution 1: FlightRadar24
 - Should you enter an invalid flight number into the search bar on FlightRadar24, a message appears stating “No matches found. Try checking the spelling, type a new search or try a search shortcut.” I have similarly implemented this method of validation for account creation, sign-in, etc.
- Buttons and Navigation
 - All existing solutions use some form of common method of navigation across their interfaces, through listed menus, drop down menus, buttons, etc. I have similarly implemented this for ease of access of the user.

Development of the Solution

Line numbers may not represent the exact to-date line numbers in the program script due to rearrangements or additions to code later on and my decision to keep any documented evidence of code development in a chronological order. However they have still been included and changed where possible.

Note: When showing API query returns in development, they return an identical value, no matter the input. This is due to the fact that I am using an API sandbox (as it is free) and still allows me to test. Ideally, should I swap the API key for one linking an account with credits, unique values will be returned.

3.1 Library and SDK Imports and Database Integrations

```
1      #Imports the library FreeSimpleGUI and assigns the alias of "GUI" for faster coding.
2      #E.g. "GUI." is equivalent to "FreeSimpleGUI."
3      import FreeSimpleGUI as GUI
4      GUI.theme('DarkBlue')
5
6      #Imports the FlightRadar24 API and SDK.
7      from fr24sdk.client import Client
8      #Takes the API token from a pre-defined environmental variable, via Windows PowerShell,
9      #that is used by the SDK.
10     client = Client(api_token="0198c22c-876f-718e-bca1-a360872c8820|")
11
12     #Other imports
13     import mysql.connector
```

Figure 3.1.1(a) ▲

Ln 3: Imports the “FreeSimpleGUI” library for use of its attributes and functions across the program and assigns the module namespace access with the alias of “GUI”. “GUI” can then be written over “FreeSimpleGUI” to access attributes (such as `.theme()`) - reducing development time.

Ln 7-10: Imports the “Client” instance from the FlightRadar24 SDK (Software Development Kit) and then allocates the pre-defined “client” variable with my sandbox API token.

Ln 13: Imports the “mysql.connector” module which allows me to integrate my database into my program via the XAMPP console and a locally hosted phpMyAdmin database.

```
11  #Uses the MySQL connector in order to integrate the flightdeck database to my program.
12  db = mysql.connector.connect(
13      host = "localhost",
14      user = "root",
15      password = "",
16      database = "flightdeckdb"
17  )
18
19  #Quick reference messages for successful or unsuccessful database connection on start.
20  if db.is_connected():
21      print("Connection Successful")
22  else:
23      print("Failed to Connect")
24
25  #Opens the cursor (a tool used to run MySQL queries and retrieve data).
26  Cursor = db.cursor()
```

Figure 3.8.1(b) ▲

Ln 12-16: Defines the built-in “mysql.connector.connect()” function and assigns it to the variable “db” for use in other built-in functions. The function includes set parameters, including: **host** (the location of the database, if it were stored externally this would be an IP address), **user** (the user access level that is defined when the program is executed, with “root” being the administrator tier), **password** and **database name**.

Ln 20-23: Upon executing the program, the message “Connection Successful” will appear if the built-in “db.is_connected()” procedure returns a true boolean state. Else, the “Failed to Connect” message will appear.

Ln 26: Opens the cursor, which is a tool used to run MySQL queries and retrieve data from my “flightdeckdb” database.

3.2 “Create Account” Interface

Figure 3.2.1(a) ▲

“Create Account” interface that demonstrates the validation features implemented and their error messages when not met. For example (below the input boxes), error messages have been returned for a password that isn’t of at least 8 characters and a password and re-entered password that don’t match. There is a confirmation message upon a quit attempt, a create account option and a sign in option if you are to already have an account.

```

15 #get_title1 getter procedure for title text.
16 def get_title1(): 1 usage
17     return [
18         [GUI.Text(text="FlightDeck", expand_x=True, justification='c', font=("Courier New", 14))],
19         [GUI.HorizontalSeparator(color = 'red')]
20     ]

```

Figure 3.2.1(b) ▲

Ln 16: Defines the “get_title” function for the title heading that displays the name of the project, “FlightDeck.”

Ln 18-19: Implements the title (with text, justifications and font parameters) and a separator line to add a cleaner look between the title and data inputs/outputs.

```

35 #get_basic_create getter procedure for basic account creation layout.
36 #Includes all error message variants for the account creation page.
37 def get_basic_create(): 1usage
38     return [
39         [GUI.Text(text="Create Account", expand_x=True, justification='c', font=("Courier New", 12))],
40
41         [GUI.Push(), GUI.Text('Email Address'), GUI.Input(do_not_clear=False, key='-EMAIL_PER-')],
42         [GUI.Push(), GUI.Text('Password'), GUI.Input(do_not_clear=False, password_char='*', key='-PASS_ENT_PER-')],
43         [GUI.Push(), GUI.Text('Re-enter Password'), GUI.Input(do_not_clear=False, password_char='*', key='-PASS_REENT_PER-')],
44
45         [GUI.Text(text="Password must be at least 8 characters long.", expand_x=True, justification='l', font=("Arial", 11), text_color=
46         [GUI.Text(text="Passwords must match.", expand_x=True, justification='l', font=("Arial", 11), text_color=("#de0000"), key='-PA
47         [GUI.Text(text="Account Creation Successful. Standby..", expand_x=True, justification='l', font=("Arial", 11), text_color=("#1
48         [GUI.Text(text="This email address is already associated with an account.", expand_x=True, justification='l', font=("Arial", 1
49
50     ]

```

```

45     expand_x=True, justification='l', font=("Arial", 11), text_color=("#de0000"), key='-8CHAR_ERR_PER-', visible=False)],
46     ation='l', font=("Arial", 11), text_color=("#de0000"), key='-PASS_MATCH_ERR_PER-', visible=False)],
47     _x=True, justification='l', font=("Arial", 11), text_color=("#1ede00"), key='-CREATED_MSG_PER-', visible=False)],
48     n account.", expand_x=True, justification='l', font=("Arial", 11), text_color=("#de0000"), key='-EXIST_ERR_PER-', visible=False)]

```

Figure 3.2.1(c) ▲

Ln 37: Defines the “get_basic_create” function which is later implemented into the “create_tabs” variable for the “create_FLayout” variable (to go into the “window_run” function).

Ln 39-43: Varieties of different instances under the “GUI.” namespace, including: text and input instances (for email address, password and the re-entered password). Certain inputs and text outputs contain individual keys, such as the password input having the key “-PASS_ENT-”, which provides an index pointer to the dictionary where all event-triggered inputs are stored. Key references can also then be used later for statements, etc.

Ln 45-48: These text statements are individual error messages for different conditions, such as a password minimum length of 8 characters and the password and re-entered password being required to match. Each error message has its own individual key so that later on, should the conditions be met for an error message to be returned, the key can be updated and “visible” can be set to true: “visible = true”.

Figure 3.2.1(d) ▲

Following on from figure 4.1, there is the option (accessible through the tab bar below the red horizontal separator) to differentiate between creating either a personal or commercial account.

```

52 #get_commerce_sign getter procedure for basic sign in integrated into commercial sign in layout.
53 def get_commerce_create(): 1usage
54     return [
55         [GUI.Text(text="Create Account", expand_x=True, justification='c', font=("Courier New", 12))],
56
57         [GUI.Push(), GUI.Text('Company Name'), GUI.Input(key='-COMPANY-')],
58         [GUI.Push(), GUI.Text('Email Address'), GUI.Input(do_not_clear=False, key='-EMAIL_CO-')],
59         [GUI.Push(), GUI.Text('Password'), GUI.Input(do_not_clear=False, password_char='*', key='-PASS_ENT_CO-')],
60         [GUI.Push(), GUI.Text('Re-enter Password'), GUI.Input(do_not_clear=False, password_char='*', key='-PASS_REENT_CO-')],
61
62         [GUI.Text(text="Password must be at least 8 characters long.", expand_x=True, justification='l', font=("Arial", 11), text_color="#de0000"),
63         [GUI.Text(text="Passwords must match.", expand_x=True, justification='l', font=("Arial", 11), text_color="#de0000", key='-PASS_MATCH_ERR_CO-'),
64         [GUI.Text(text="Account Creation Successful. Standby..", expand_x=True, justification='l', font=("Arial", 11), text_color="#1ede00", key='-CREATED_MSG_CO-'),
65         [GUI.Text(text="This email address is already associated with an account.", expand_x=True, justification='l', font=("Arial", 11), text_color="#de0000", key='-EXIST_ERR_CO-'),
66     ]

```

Figure 3.2.1(e) ▲

Ln 53: Defines the “get_commerce_create” function which is later implemented into the “create_tabs” variable for the “create_FLayout” variable (to go into the “window_run” function).

Ln 55-60: Additional instances under the “GUI.” namespace, which includes the input for the company name, with the key “-COMPANY-”. Error messages have been copied over but altered for a commercial format, therefore, when performing validation, it is only performed for the commercial inputs not both the commercial and personal inputs.

Ln 62-65: These text statements are individual error messages for different conditions, such as a password minimum length of 8 characters and the password and re-entered password being required to match. Each error message has its own individual key so that later on, should the conditions be met for an error message to be returned, the key can be updated and “visible” can be set to true: “visible = true”.

```

68 #Integrates the basic creation interface and the commercial creation interface into one variable.
69 #Allows me to use the "enable_events" feature to track what account creation tab the user is actively using
70 #Via the "-TAB_GROUP-" key identifier.
71 create_tabs = [
72     [GUI.TabGroup(
73         layout: [[
74             GUI.Tab( title: "Personal", get_basic_create()),
75             GUI.Tab( title: "Commercial", get_commerce_create())
76         ]],
77         key="-TAB_GROUP-",
78         enable_events=True
79     )]
80 ]
81
82 #Create button for the event-triggered dictionary creation.
83 create_buttons = [
84     [GUI.Button( button_text: 'Create', bind_return_key=True)],
85     #The "Already have an account? Sign in." button has been assigned with the event identifier "-GO_SIGN_IN-" to skip validation
86     #and to close "window1" and create "window2", the same as if validation were to be met when creating an account.
87     [GUI.Button( button_text: 'Already have an account? Sign in.', bind_return_key=True, key="-GO_SIGN_IN-")]
88 ]

```

```

90 #Combines all the sub-layouts into one main layout for the window1_run() function.
91 create_FLayout = get_title1() + create_tabs + create_buttons

```

Figure 3.2.1(f) ▲

Ln 71-80: Defines the “create_tabs” variable which groups all the interface functions into one memory location for later use when creating the “create account” window. The “create_tabs” variable consists of the personal and commercial interface which are split into two separate tabs (see **Figure 4.1**) using the built-in “GUI.TabGroup” function. The dictionary index key “-TAB_GROUP-” has also been set and “enable_events” have been set to “True”. Therefore, I can differentiate between what tab the user is currently active on by either “Personal” or “Commercial” being shown in that “-TAB_GROUP-” dictionary index. This is beneficial for validation later on.

I had initially not included this, however I deemed it unavoidable regarding validation statements as continuous logic errors were caused by the inability to decipher between the active tab (personal / commercial). Statements were often just never met and inaccurate validation was a result of.

Ln 83-88: A similar concept (regarding a variable that groups a category of functions / attributes) has been done with any buttons required for the “create account” window. The “Already have an account? Sign in.” Button is given the event identifier “-GO_SIGN_IN-”; if that button is pressed, a later selection statement can skip the create account process and go straight to the sign-in process.

Ln 91: Combines the “get_title1()” function, “create_tabs” variable and “create_buttons” variable into one variable to then implement into the layout parameter when creating the window later on.

3.3 “Create Account” Execution, Validation / Verification and DB Commitment

Personal and Commercial Aspects

```

97 def window1_run(): 1usage
98 #Any functions / procedures regarding physical and device specific changes to the window can only be made after
99 #it has been finalised using the procedure ".Finalize()"
100 window1 = GUI.Window(title: 'FlightDeck', create_FLayout, size = (700, 400), enable_close_attempted_event = True).Finalize()
101
102 #Infinite loop that keeps the window event actively running.
103 while True:
104     #“values” is the dictionary created that contains inputs.
105     event, values = window1.read()
106     print(event, values)
107     active_tab = values['-TAB_GROUP-'] if "-TAB_GROUP-" in values else None
108
109     #If an event is triggered by the user switching tabs, this condition ensures false error messages aren't returned without
110     #an invalid input.
111     if event == '-TAB_GROUP-':
112         window1['-8CHAR_ERR_PER-'].update(visible=False)
113         window1['-PASS_MATCH_ERR_PER-'].update(visible=False)
114         window1['-8CHAR_ERR_CO-'].update(visible=False)
115         window1['-PASS_MATCH_ERR_CO-'].update(visible=False)
116         continue
117
118     #If the user attempts to close the window, a popup will appear (center screen) to confirm their decision.
119     if (event == GUI.WINDOW_CLOSE_ATTEMPTED_EVENT or event == 'Exit') and GUI.popup_yes_no('EXIT - Confirm you want to exit.') ==
120         Cursor.close()
121         db.close()
122         break

```

Figure 3.3.1(a) ▲

Ln 97-100: Defines the “window1_run” function, along with the parameters of: the window title (“FlightDeck”), the GUI layout to be included in the window (the previously created “create_FLayout” variable that includes the interface title, personal account creation tab and commercial account creation tab), the size and the “enable_close_attempted_event” True boolean (which will trigger an event, which writes all inputs, etc, to the dictionary “values” when a window quit attempt occurs).

Ln 103-122: The while loop that keeps the window event running and also provides the event trigger and dictionary, “values”, which are read from window1 using the “window1.read()” function.

Ln 111-116: A selection statement, where if the event includes the “-TAB_GROUP-” key, which will be defined on execution of the account creation window (**as shown in Figure 3.3.1(a)**), all the validation error messages will have their “visible” attribute set to “False” initially and the program will continue.

Ln 120: Closes the cursor (a tool used to run MySQL queries and retrieve data) before the window is closed.

Ln 121: Closes the connection to the database before the window is closed.

Ln 122: Escapes the while loop should the window quit attempt be successful, therefore, closing the window as the continuous while loop that previously kept the window active, has been exited.

```
95 #Assigns an index of the output dictionary "values" to different equivalent variables using keys.  
96 #One variable may have multiple keys, if the first key index is blank, it uses the second.  
97 password_per = values["-PASS_ENT-"]  
98 password_co = values["-PASS_ENT-1"]  
99 password_reinput_per = values["-PASS_REENT-"]  
100 password_reinput_co = values["-PASS_REENT-2"]  
101 email_address_per = values["-EMAIL-"]  
102 email_address_co = values["-EMAIL-0"]  
103 company_name = values["-COMPANY-"]
```

Figure 3.3.1(b) ▲

Ln 97-103: Each index for the “values” dictionary is assigned to a variable. This then allows for different variables to be validated together, against each other, etc, easily. For example, in order to check the personal account password (located at the “-PASS_ENT” index) and re-entered password (located at the “-PASS_REENT-” index) are identical when creating the account, we can compare the “password_per” and “password_reinput_per” variables in a selection statement.

```

142     #Sets all validation variables / markers to "True"
143     valid_8char_per = True
144     valid_8char_co = True
145     valid_passmatch_per = True
146     valid_passmatch_co = True
147
148     #Validates that the original password is longer than 8 characters.
149     if active_tab == 'Personal':
150         if len(password_per) < 8:
151             window1['-8CHAR_ERR_PER-'].update(visible=True)
152             valid_8char_per = False
153         if password_per != password_reinput_per:
154             window1['-PASS_MATCH_ERR_PER-'].update(visible=True)
155             valid_passmatch_per = False
156
157     elif active_tab == 'Commercial':
158         if len(password_co) < 8:
159             window1['-8CHAR_ERR_CO-'].update(visible=True)
160             valid_8char_co = False
161         if password_co != password_reinput_co:
162             window1['-PASS_MATCH_ERR_CO-'].update(visible=True)
163             valid_passmatch_co = False

```

Figure 3.3.1(c) ▲

Ln 143-146: Sets all the validation markers to “True”, therefore, in the following validation statements, all I need to do is set them to “False” if validation isn’t met. This hopefully speeds up the validation process as I don’t have to correct each one to True/False, only correct from “True” to “False”.

Ln 149-155: Since I have enabled events upon changing between the personal and commercial create account tab and assigned this event with the key “-TAB_GROUP-”, I can now use this to aid my validation, previously, this was not used and caused many logic errors, such as certain validation markers not being set to “False” even when the conditions were unfavourable. The “active_tab” variable is set to either “Personal” or “Commercial” depending on which interface the user is actively using.

If the “active_tab” variable is set to “Personal” then the next level of validation conditions is reached:

- If the length of the personal password input is less than 8 characters (the targeted error message is set to be visible - via “visible=True” - and the “valid_8char_per” validation variable is set to “False”).
- If the personal password input is not equal to the personal password re-input (the targeted error message is set to be visible and “valid_passmatch_per” is set to “False”).

If the “active_tab” variable is set to “Commercial” then the next level of validation conditions is reached:

- If the length of the commercial password input is less than 8 characters (the targeted error message is set to be visible - via "visible=True" - and the "valid_8char_co" validation variable is set to "False").
- If the personal password input is not equal to the personal password re-input (the targeted error message is set to be visible and "valid_passmatch_co" is set to "False").

Email-Address	Password	Company-Name
	9707dc58c076a752c775c9de8459278112bddd71f98a7bb163...	

Figure 3.3.1(d) ▲

This demonstrates the use of the hashing algorithm that applies a hash function to the entered password (assuming all validation criteria are met) before it is committed to the "credentials" table within the "flightdeckdb" database.

In this example, the password "TestTest1" has been hashed into a 256 bit string length.

```

165 if (active_tab == "Personal" and valid_8char_per and valid_passmatch_per) or (active_tab == "Commercial" and valid_8char_co an
166     #If password validation is met (minimum 8 characters and both entries matching) the password is hashed via the
167     #hash_password(password) function defined above.
168     hashed_password = hash_password(password_per or password_co)
169
170     #Inserts the email address, password and company name from the above inputs into the column names
171     #stored in "values". The cursor then executes that operation with "sql" and "values" and is committed.
172     exist_check = "SELECT `Email-Address` FROM credentials WHERE `Email-Address` = %s"
173     Cursor.execute(exist_check, params: (email_address_per or email_address_co,))
174     result = Cursor.fetchone()
175
176     #Checks if an account has already been created with the same email, if so, the program will close.
177     if not result:
178         sql = "INSERT INTO Credentials (`Email-Address`, `Password`, `Company-Name`) VALUES (%s, %s, %s)"
179         values = ((email_address_per or email_address_co), hashed_password, company_name)
180         Cursor.execute(sql, values)
181         db.commit()
182         window1['-CREATED_MSG_PER-'].update(visible=True)
183         window1['-CREATED_MSG_CO-'].update(visible=True)
184         event, create_values = window1.read(timeout=3000)
185
186         window2_run()
187
188     else:
189         window1['-EXIST_ERR_PER-'].update(visible=True)
190         window1['-EXIST_ERR_CO-'].update(visible=True)
191         event, create_values = window1.read(timeout=3000)
192         break
193
194     window1.close()

```

Figure 3.3.1(e) ▲

Ln 157-160: If the account creation validation conditions, shown in **Figure 3.6**, are met and all unique validation booleans have been set to True (for either the personal / commercial interface): the “hash_password()” hash function is called, with the parameter “password_per” or “password_co” (whichever has a value stored). The hashed password is then stored in the “hashed_password” variable.

Ln 164: The “exist_check” variable is defined and consists of an SQL query (as shown) that will select all email addresses from the “Email-Address” column in the “credentials” table, located within the “flightdeckdb” database, that match a set placeholder (“%s”) (given as a parameter when executed).

Ln 165-166: The built-in “Cursor.execute()” function is used to execute the SQL query on the “credentials” table and uses the parameter “email_address_per” or “email_address_co” (whichever has a value stored) as the set placeholder that is compared to stored values in the “Email-Address” column. The result is then fetched and stored in the “result” variable.

Ln 169-170: If there is not a result (AKA: no email address match has been returned so the email address is not already stored and linked to an account), the “sql” variable is defined and consists of an SQL query (as shown) that will insert three set placeholders (“%s”), one in each column: “Email-Address”, “Password” and “Company-Name”.

Ln 171: The “values” variable is defined and consists of the three set placeholders that are inputted earlier in the program: “email_address_per” or “email_address_co”, “hashed_password” and “company_name”.

Ln 172-186: The built-in “Cursor.execute()” function is used to execute the SQL query on the “credentials” table and uses the parameters “sql” and “values”. The three placeholders are then committed to the database via the built-in “db.commit()” function. The account created message is then displayed and three seconds later, the loop continues: the “window2_run()” function is executed (which creates the sign-in window) and the “window1.close()” function is executed (which closes the account creation window).

Ln 180-186: If there is a result (AKA: an email address match has been returned so the email address is already stored and linked to an account), an error message is returned, the loop is broken and the window is closed.

Commercial Only Aspects

CompanyDomain

aa.com

delta.com

united.com

southwest.com

csair.com

Figure 3.3.1(f) ▲

In an attempt to verify the identity of someone attempting to create a commercial account, and email address being asked as a data entry, I have created a new table in my "flightdeckdb" database, called "domains." This table consists of a dataset of the domains used (in email address aliases, website domains, social media platforms, etc) by the top 25 global airlines. Therefore, as demonstrated below, an attempt can be made to verify a commercial account creation using this (alongside SQL queries).

```

164     #If the user is currently on the "Commercial" tab (create account window).
165     elif active_tab == 'Commercial':
166
167         #Selects the top record (only consisting of the CompanyDomain attribute) from the
168         #"domains" table under the "flightdeckdb" database, where the "CompanyDomain" is equal to
169         #the user-entered domain.
170         domain_check = "SELECT `CompanyDomain` FROM domains WHERE `CompanyDomain` = %s"
171         Cursor.execute(domain_check, params: (company_domain,))
172         domain_result = Cursor.fetchone()
173
174         if len(password_co) < 8:
175             window1['-8CHAR_ERR_CO-'].update(visible=True)
176             valid_8char_co = False
177         if password_co != password_reinput_co:
178             window1['-PASS_MATCH_ERR_CO-'].update(visible=True)
179             valid_passmatch_co = False
180         #If a result is returned from the domain_check SQL query, the company domain entered
181         #by the user must exist. Verification passed.
182         if domain_result:
183             event, create_values = window1.read(timeout=3000)
184             print("COMPANY DOMAIN CHECK SUCCESS")
185             window1.close()
186             break

```

Figure 3.3.1(g) ▲

An initial prototype of this verification process, where the company domain (inputted by the user in the account creation window) is searched for in the "domains" table under the "flightdeckdb" database.

Ln 182-186: An initial verification process, for testing purposes, where if a user attempts to create an account with a valid company domain (matching one of the top 25 global airlines), the create account window will close with the loop broken and a message will be printed to the console.

Table 3.3.1			
Test ID	Input	Expected Output	Status, Actual Output and Notes
1	A valid company domain from the “domains” table, such as “aa.com,” when attempting to create an account (“Create” button push).	The verification process should be completed and the create account window closed, with a message being printed to the console showing success.	PASS:
<pre>Connection Successful -TAB_GROUP- {'-EMAIL_PER-': '', '-PASS_ENT_PER-': '', '-PASS_REENT_PER-': '', '-COMPANY-': ''}, Create {'-EMAIL_PER-': '', '-PASS_ENT_PER-': '', '-PASS_REENT_PER-': '', '-COMPANY-': 'aa.com'} COMPANY DOMAIN CHECK SUCCESS</pre>			

3.4 “Sign-In” Interface

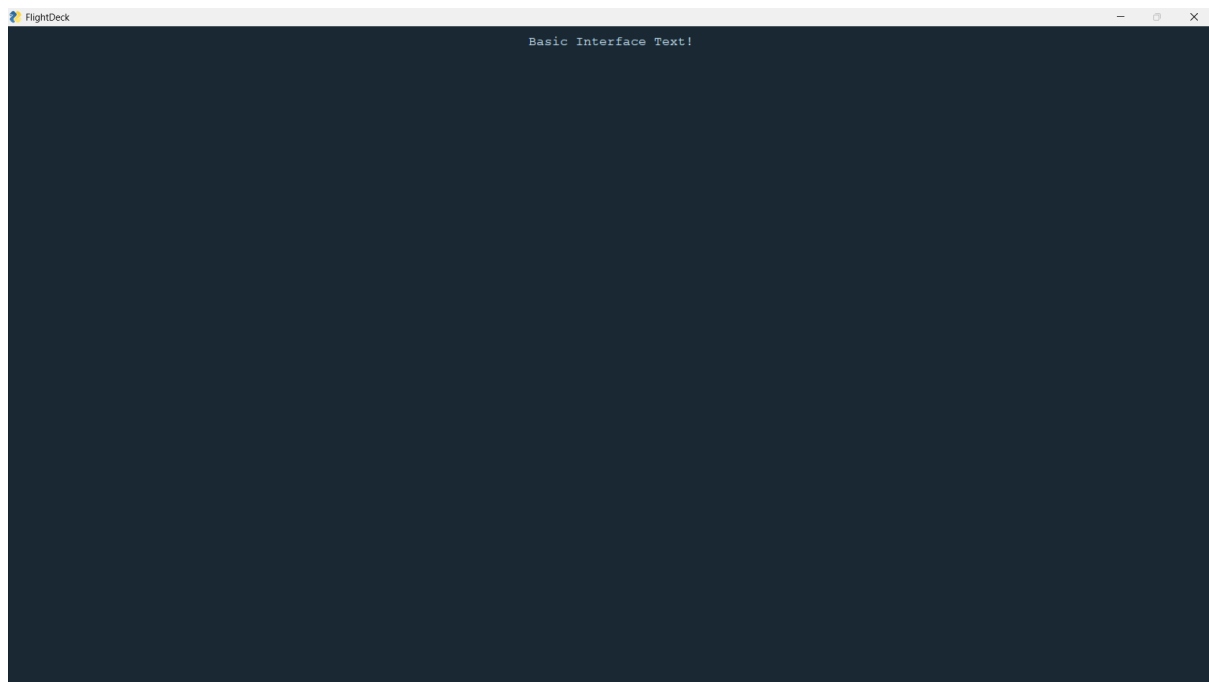


Figure 3.4.1(a) ▲

The first prototype of the basic sign-in window (window2). This window is created when one of the two following conditions are met: either the user has clicked the “Already have an account? Sign in.” button, which is shown under the “Create Account” button (**see Figure 4.1**) **or**, when the user has met all validation criteria for password matching and length and that the same email address isn’t already linked to an account (if creating a new account).

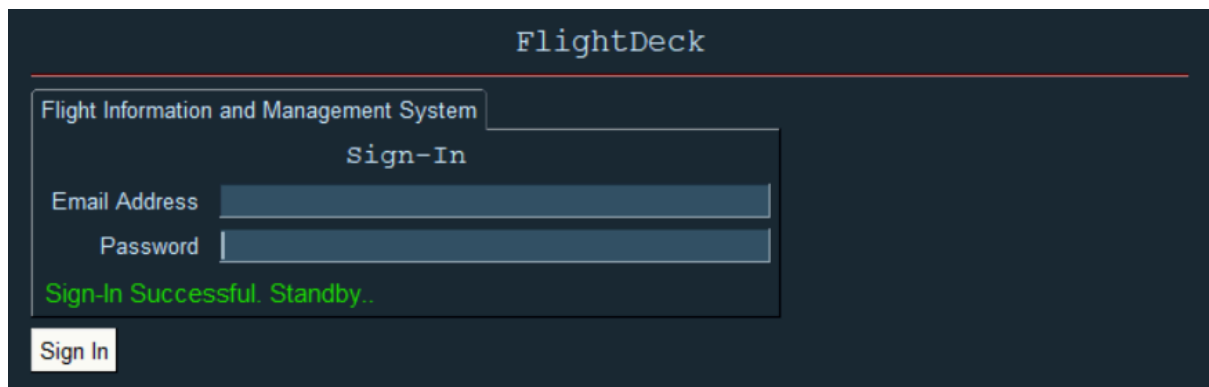


Figure 3.4.1(b) ▲

A finalised initial prototype of the sign-in window - which is accessible either by creating an account via the “create account” window **or** by selecting the “Already have an account? Sign in” option, again on the same window (see Figure 4.1, 4.1a). It comes equipped with both a “Sign-In Successful. Standby..” message (as shown) and also an “Invalid Credentials” message, should a matching record not exist in the ‘credentials’ database.

```

196 #get_sign getter procedure for sign in layout.
197 #Includes all error message variants for the sign-in page.
198 def get_sign(): 1 usage
199     return [
200         [GUI.Text(text="Sign-In", expand_x=True, justification='c', font=("Courier New", 12))],
201
202         [GUI.Push(), GUI.Text('Email Address'), GUI.Input(do_not_clear=False, key='-EMAIL-')],
203         [GUI.Push(), GUI.Text('Password'), GUI.Input(do_not_clear=False, password_char='*', key='-PASS-')],
204
205         [GUI.Text(text="Invalid Credentials.", expand_x=True, justification='l', font=("Arial", 11), text_color="#de0000", key='-CREDS_ERR-'), visible=False]],
206         [GUI.Text(text="Sign-In Successful. Standby..", expand_x=True, justification='l', font=("Arial", 11), text_color="#1ede00", key='-SIGN_MSG-', visible=False)]
207     ]

```

```

205         [GUI.Text(text="Invalid Credentials.", expand_x=True, justification='l', font=("Arial", 11), text_color="#de0000", key='-CREDS_ERR-', visible=False)],
206         [GUI.Text(text="Sign-In Successful. Standby..", expand_x=True, justification='l', font=("Arial", 11), text_color="#1ede00", key='-SIGN_MSG-', visible=False)]

```

Figure 3.4.1(c) ▲

Ln 198: Defines the “get_sign()” function, which includes all the different text and input attributes that are found in the sign-in window (accessed either via creating an account with valid credentials or using the “Already have an account? Sign in.” button).

Ln 202-203: The input text box attributes that are labelled “Email Address” and “Password” respectively, for the user to enter into. Both also have unique keys which then allocate the index in the “sign_values” dictionary, for that input, with the same key for any later use.

Ln 205: An error message for the entering of invalid credentials upon sign in, which has the unique key of “-CREDS_ERR-”.

Ln 206: A message for a successful sign in upon entering valid credentials, which has the unique key of “-SIGN_MSG-”.

```

215 #Create buttons with specific unique keys for the event-triggered dictionary inputs.
216 sign_buttons = [
217     [GUI.Button( button_text: 'Sign In', bind_return_key=True)]
218 ]
219
220 #A singular tab has been used to group the get_sign() elements as I couldn't figure out how to
221 #make them all group to the left-hand side of the window... this saved time...
222 sign_tab = [
223     [GUI.TabGroup(
224         [[GUI.Tab( title: "Flight Information and Management System", get_sign())]]
225     )]
226 ]
227
228 sign_FLayout = get_title1() + sign_tab + sign_buttons

```

Figure 3.4.1(d) ▲

```

230 def window2_run(): 2 usages
231     window2 = GUI.Window(title: 'FlightDeck', sign_FLayout, size = (700, 400), enable_close_attempted_event = True).Finalize()
232
233     while True:
234         event, sign_values = window2.read()
235         print(event, sign_values)
236
237         if (event == GUI.WINDOW_CLOSE_ATTEMPTED_EVENT or event == 'Exit') and GUI.popup_yes_no('EXIT - Confirm you want to exit.') ==
238             Cursor.close()
239             db.close()
240             break

```

Figure 3.4.1(e) ▲

```

242     #Assigns an index of the output dictionary "sign_values" to different equivalent variables using
243     #the equivalent keys.
244     password_check = sign_values["-PASS-"]
245     email_address_check = sign_values["-EMAIL-"]
246     #This executes a hash algorithm on the password entered on the sign-in page to compare later on
247     #and since the original password has been hashed, it cannot be reverted to its original state due to
248     #hashing being a one-way function.
249     hashed_password_check = hash_password(password_check)

```

Figure 3.4.1(f) ▲

Ln 244-245: Declares the dictionary indexes for the password and email address input to their own respective variables via their individual keys for later use in SQL validation queries as parameters.

Ln 249: Since hash functions are a one-way process (therefore a user's original password cannot be seen on the database, only the hashed password), the password inputted on the sign-in page must be hashed before it can be cross-checked with the credentials on the `credentials` table within the `flightdeck` database.

```

251 #The "creds_check" statement selects all records from the table "credentials" where the email address
252 #and password are equal, respectively, to the "email_address_check" and "hashed_password_check".
253 creds_check = "SELECT * FROM `credentials` WHERE `Email-Address` = %s AND `Password` = %s"
254 Cursor.execute(creds_check, params: (email_address_check, hashed_password_check))
255 result2 = Cursor.fetchone()
256
257 #If no result is found, the credentials are deemed incorrect as they must be correct initially at the
258 #create account stage. The credentials invalid error message is displayed by settings its "visible"
259 #attribute to "True".
260 if not result2:
261     window2['-CREDS_ERR-'].update(visible=True)
262 #If a result is found, the credentials are deemed correct. The sign-in successful message ("-SIGN_MSG-") is
263 #displayed by setting its "visible" attribute to "True". Window3 is executed, window2 is closed.
264 else:
265     window2['-SIGN_MSG-'].update(visible=True)
266     #3000ms window2 timeout to allow the user to read the message before closing.
267     event, sign_values = window2.read(timeout=3000)
268
269     window2.close()
270     window3_run()

```

Figure 3.4.1(g) ▲

Ln 253-254: “creds_check” stores the SQL query string that will select all records from the ‘credentials’ table where the “Email-Address” and “Password” placeholders (identified with “%s”) are equal to the user-inputted email address and password on the sign in page (validation).

The “creds_check” query is then executed via the cursor with the parameters “email_address_check” and “hashed_password_check” to fill the previous placeholders. The (what should be single) result is then returned to the “result2” variable.

Ln 260-270: Validation for the users attempted sign in:

- If a result isn’t returned, the credentials entered by the user must be incorrect and the “-CREDS_ERR-” message identifier key is set to “visible=True”.
- Else, the successful sign in confirmation message key (“-SIGN_MSG-”) is set to “visible=True”. Window2 (the sign in window) then times out for up to 3 seconds before it is closed. Window3 (the main window) is then executed.

3.5 Main Program Interface

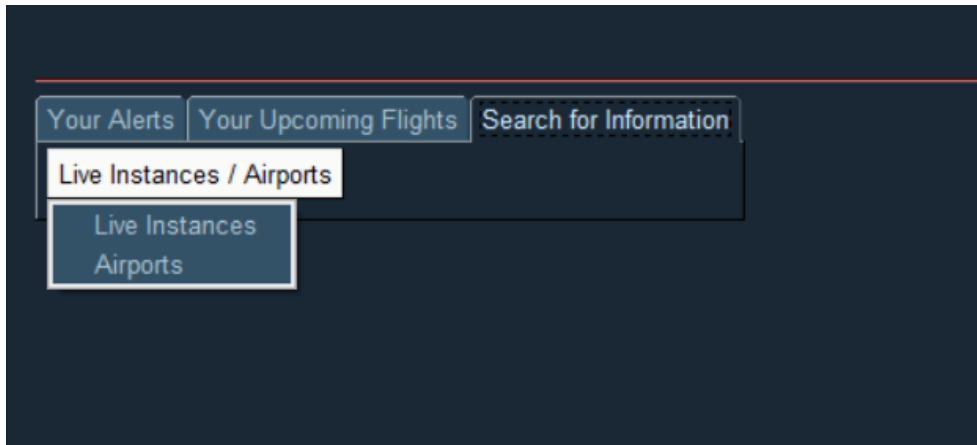


Figure 3.5.1(a) ▲

The first prototype of the main window which includes the window title (hidden in this screenshot) with a horizontal separator between the main interface elements. The main interface currently consists of three tabs: “Create/Your Alerts” - where the user will be able to view their alerts for specific flights. “Your Upcoming Flights” - where the user will be able to see their upcoming flights. “Search for Information” - where the user will be able to search for data through an API and will later be separated into searching for data regarding “Live Instances” and “Airports”.

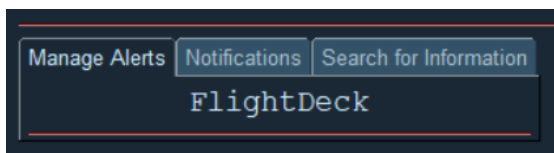


Figure 3.5.1(b) ▲

Final prototype of tabs.

```

272 search_menus = [
273     [GUI.ButtonMenu( button_text: 'Live Instances / Airports', menu_def: ['Menu', ['Live Instances::-INST-', 'Airports::-APT-']])]
274 ]
275
276 main_tabs = [
277     [GUI.TabGroup(
278         layout: [[
279             GUI.Tab( title: "Your Alerts", get_title1()),
280             GUI.Tab( title: "Your Upcoming Flights", get_title1()),
281             GUI.Tab( title: "Search for Information", search_menus)
282         ]],
283         key="-TAB_GROUP-",
284         enable_events=True
285     )]
286 ]
287
288 main_FLayout = get_title1() + main_tabs

```

Figure 3.5.2(a) ▲

Ln 272-274: Creates the “search_menus” variable which contains the “GUI.ButtonMenu” function and consists of the “Live Instances / Airports” selection menu in **Figure 3.5.1(a)**, where users should be able to select between searching for live flights or for airports.

Ln 276-286: The “main_tabs” variable consists of the “GUI.TabGroup” function and then encompasses “Your Alerts”, “Your Upcoming Flights” and “Search for Information”, with each consisting of specific elements. E.g. The “Search for Information” tab consists of the “search_menus” variable (as shown above).

Ln 288: I created a “main_FLayout” to encompass the “get_title1()” function and the “main_tabs” TabGroup functions and instances.

```

290 def window3_run():
291     window3 = GUI.Window( title: 'FlightDeck', main_FLayout, size = (700, 400), enable_close_attempted_event = True).Finalize()
292     window3.maximize()
293
294     while True:
295         event, main_values = window3.read()
296         print(event, main_values)
297
298         if (event == GUI.WINDOW_CLOSE_ATTEMPTED_EVENT or event == 'Exit') and GUI.popup_yes_no('EXIT - Confirm you want to exit.') ==
299             Cursor.close()
300             db.close()
301             break
302
303 window1_run()

```

Figure 3.5.2(b) ▲

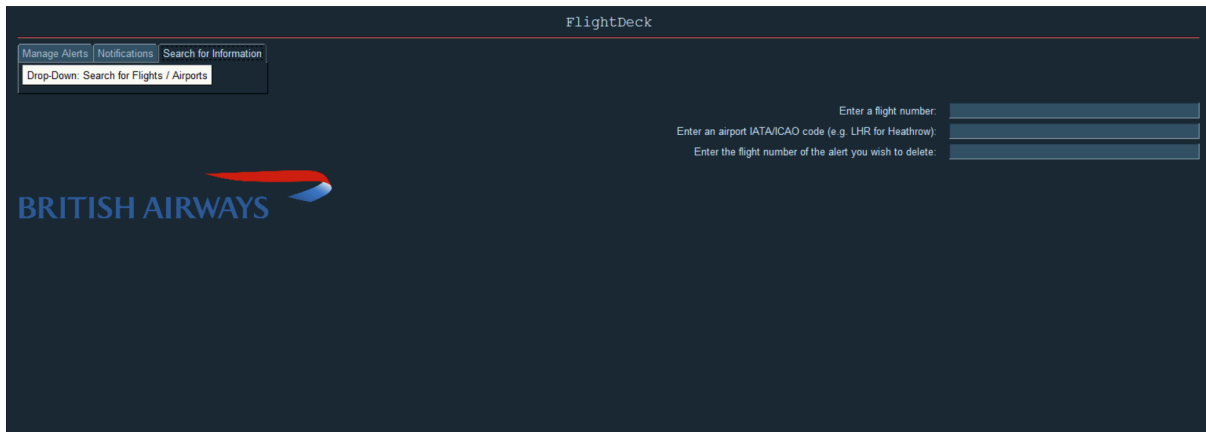


Figure 3.5.3(a) ▲

An initial and final prototype of a method of GUI customisation for commercial users, which when logged in, displays the company logo as shown.

From **Figure 3.5.3(b)**.

```

414 CompanyCheck = "SELECT `CompanyName` FROM credentials WHERE `EmailAddress` = %s AND `CompanyName` IS NOT NULL"
415 Cursor.execute(CompanyCheck, params: (GlobalEmail,))
416 # Fetches one (the first) value.
417 CompanyCheckResult = Cursor.fetchone()
418
419 if CompanyCheckResult:
420     Logo = "SELECT `Logo` FROM domains WHERE `CompanyDomain` LIKE %s"
421     Cursor.execute(Logo, params: (CompanyCheckResult[0],))
422     #Fetches one (the first) value.
423     LogoResult = Cursor.fetchone()
424     Base64 = LogoResult[0]
425
426 window3["-LOGO-"].update(data=Base64, visible=True, subsample=3)
  
```

Figure 3.5.3(b) ▲

Ln 414: Stores the SQL query in "CompanyCheck" which selects the "CompanyName" attribute, from records in the "credentials" table (within the "flightdeckdb" database") with the "EmailAddress" given by the placeholder in **Ln 415** (in this case, the "GlobalEmail" variable which has data assigned upon sign-in). Assuming the attribute "CompanyName" is not blank.

Ln 415: Executes the SQL query (stored in "CompanyCheck") using the parameters "GlobalEmail" as the placeholder for the "EmailAddress" attribute.

Ln 417: Assigns the first and only result (if one meets the conditions) in the variable "CompanyCheckResult".

Ln 419-423: If a result exists (AKA, data is stored in "CompanyCheckResult"), then a second SQL query is executed which selects the "Logo" attribute from the "domains" table, where the any matching part of the "CompanyCheckResult[0]" index value (the company name) is assigned to the "CompanyDomain" value.

Ln 424: Assigns the “LogoResult[0]” index value (which contains the image Base64 value), to the “Base64” attribute (a parameter for the .update() function in **Ln 426**). Providing the correct logo.

3.6 “Search for Information” Tab

General GUI Development

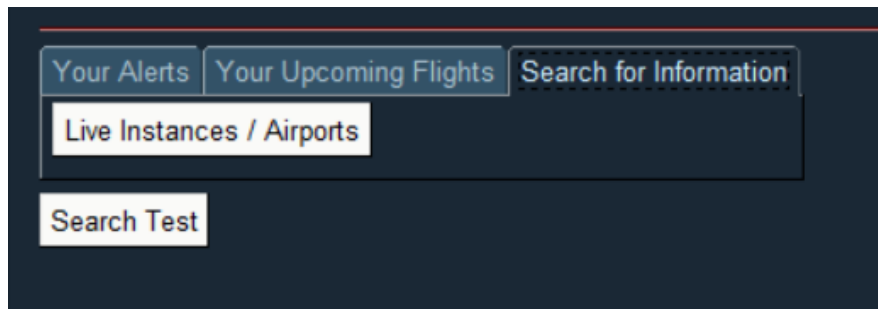


Figure 3.6.1 ▲

An initial prototype for the “Search for Information” tab, the “Search Test” button returns the default return (**see figure 3.6a**), the button has been added outside of the tab group for simpler coding development or until a better solution is found.

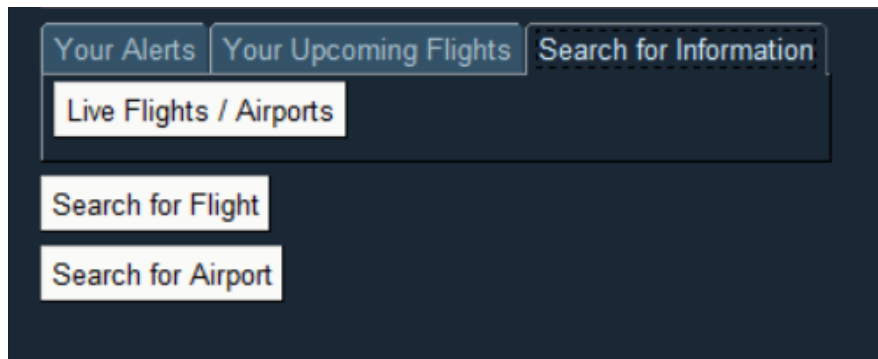


Figure 3.6.1(a) ▲

A 2nd prototype for the “Search for Information” tab, with two separate search buttons that call the “get_searchAPT()” and “get_searchFLT()” functions respectively. Each button only appears when on the specific menu tab using the “Live Flights / Airports” menu drop-down.

A 3rd prototype for the “Search for Information” tab has been made where instead of the “Search for Flight” and “Search for Airport” buttons permanently showing after selecting a Live Flight / Airport drop down menu option, they disappear if not selected. They also disappear if the user changes tabs.

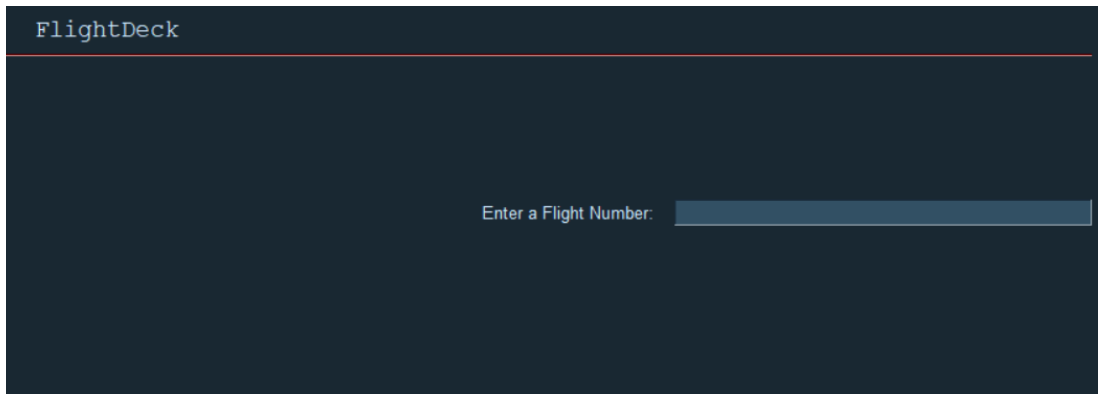
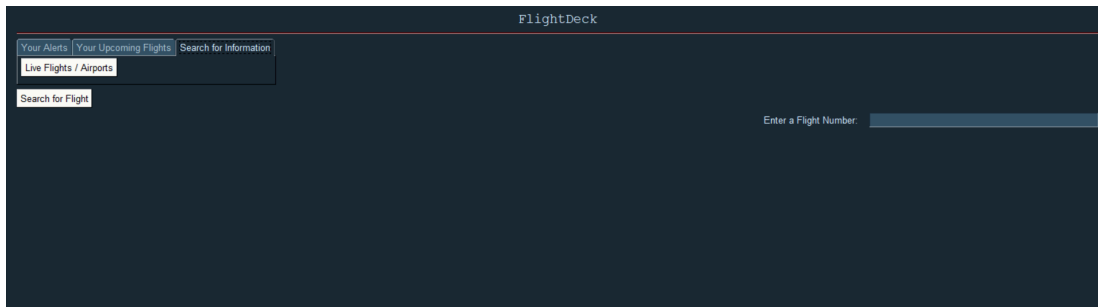


Figure 3.6.2 ▲

Under the “Search for Information” tab, I have now isolated the buttons depending on the “Live Flights / Airports” menu option. If the user wishes to search for flights, a search bar appears, as well as the “Search for Flight” button.



Figure 3.6.2(a) ▲

An initial prototype for a graphical display of API query returns (previously displayed via the terminal, in testing stages), specifically returns from the “get_searchFLT()” function, returning “flight_summary” and “flight_positions”.

The next step will be to split the return into individual attributes, so that they can be assigned to variables and displayed separately with additional information. Following on from **Design of the Solution > Usability > Usability Features**.

```
402 if event == '-SEARCHBT_APT-':  
403     #Triggers the "get_searchAPT" API query function with the user input assigned to  
404     #"SearchEntApt", assigned to a dictionary identifier, being passed as a parameter.  
405     airport = get_searchAPT(SearchEntApt)  
406     #Updates the "-AIRPORT-" GUI element with the data stored in "airport".  
407     window3["-AIRPORT-"].update(str("Airport Name: " + airport.name + "\nAirport IATA: " + airport.iata), visible=True)
```

Figure 3.6.2(b) ▲

A final prototype of **Figure 3.6.2(a)**, where I have concatenated additional information for each attribute, for example, a short description, to the object attribute.

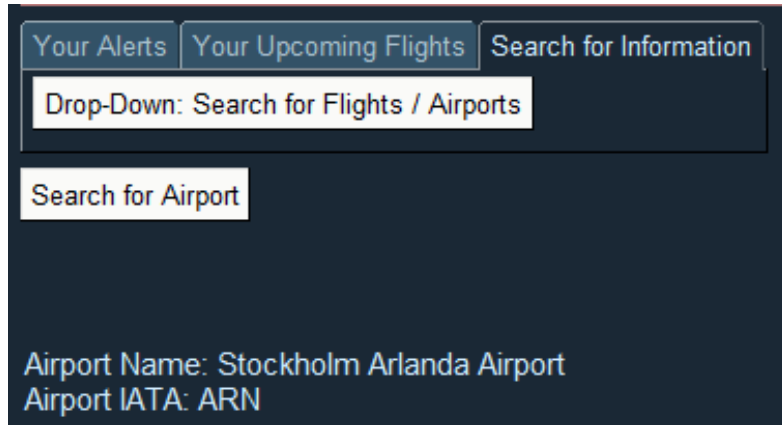


Figure 3.6.2(c) ▲

This is the result of **Figure 3.6.2(b)** in testing, when an input of LHR (the IATA code for London Heathrow) is made.

```

336 SearchElements = [
337     #Buttons
338     [GUI.Button( button_text: 'Search for Flight', key='-SEARCHBT_FLT-', bind_return_key=True, visible=False)],
339     [GUI.Button( button_text: 'Search for Airport', key='-SEARCHBT_APT-', bind_return_key=True, visible=False)],
340
341     #Tickboxes
342     [GUI.Checkbox( text: 'Show Stats For Nerds', default=False, key="-NERD_FLT-", visible=False)],

```

Figure 3.6.2(d) ▲

Ln 342: Programmes the checkbox element for **Figure 3.6.2(e)**, below. It is assigned with the identifier “-NERD_FLT-”

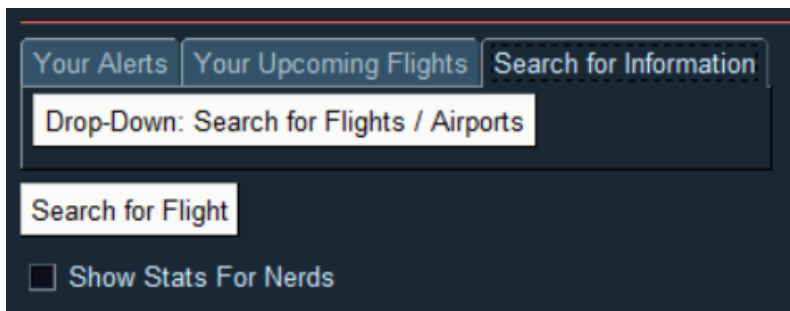


Figure 3.6.2(e) ▲

From my **Primary Research - Consumer Questionnaire**, I was given the idea to add an option to show additional information, portrayed as “stats for nerds.” Similar ideas have been implemented on sites, such as YouTube, that I think are quite interesting.

My initial idea was to simply add an additional button, titled “Show Stats For Nerds,” that the user could press after having searched for a flight, however this was flawed as I was attempting to show additional information to an element that the information wasn’t returned to in the first place. Therefore, I am going to use a checkbox element which must be selected beforehand (first prototype shown above).

Table 3.6.1(a)			
Test ID	Input	Expected Output	Status, Actual Output and Notes
1	Check the “Show Stats For Nerds” checkbox and search for the flight number “BA186.”	The standard query return, as well as the additional “stats for nerds” should appear.	FAIL: Additional information not shown. See below and Test Table T12.

```
478     if not ExistAlertResult:
479         AlertSql = "INSERT INTO alerts (`EmailAddress`, `FlightNumber`, `DepartureTime`,
480         AlertValues = (GlobalEmail, flight_summary.data[0].flight, flight_summary.data[0]
481         Cursor.execute(AlertSql, AlertValues)
482         #Commits the values to the database under one transaction.
483         db.commit()
484         window3["-ALERTCREATED-"].update(visible=True)
485     else:
486         window3["-ALERTEXISTS-"].update(visible=True)
```

Figure 3.6.2(e) ▲

Logic and initial testing behind **Figure 3.6.2(f)**, which determines whether the “-ALERTCREATED-” or “-ALERTEXISTS-” message should appear when a user attempts to create an alert for a flight (via checking the respective checkbox before searching).

The screenshot shows a dark-themed application window. At the top, there is a search bar labeled "Search for Flight". Below it are two checkboxes: "Show Stats For Nerds" (unchecked) and "Create An Alert For This Flight" (checked). The main content area displays flight information: "-- Flight Info --", "Airline: SAS // Flight: SK1415", "Aircraft: A20N // Callsign: SAS1415", and "Departure Airport: ARN // Departure Time: 2023-01-27T05:15:22". Below this, it shows "ETA: 2023-12-15T14:47:28Z" and "Likely Late". At the bottom, a green message states "Alert Creation Successful".

Figure 3.6.2(f) ▲

Shows that if the logic in **Figure 3.6.2(e)** is met, then the appropriate message will appear. Linking to **Design of the Solution > Usability**.

API Integration, Query and SQL Functions

```
Connection Successful
icao='ESSA' iata='ARN' name='Stockholm Arlanda Airport'
```

Figure 3.6.3 ▲

An initial test on the FlightRadar24 API, using the “airports light” endpoint, which returns the ICAO and IATA code for a specified airport, as well as its name. This doesn’t yet include any search input parameters as it is purely a test that API queries can be made and return an expected value.

```
-TAB_GROUP- {'-MENU_OPT-': None, '-TAB_GROUP-': 'Search for Information'}
-MENU_OPT- {'-MENU_OPT-': 'Flights::-FLT-', '-TAB_GROUP-': 'Search for Information'}
-SEARCH_FLT- {'-MENU_OPT-': 'Flights::-FLT-', '-TAB_GROUP-': 'Search for Information'}
data=[FlightPositionsFull(fr24_id='333ca4a2', lat=35.34722, lon=-7.90277, track=219, alt=37000, gspeed=440, vspeed=64
```

Figure 3.6.4 ▲

An initial test on the FlightRadar24 API “Flights Full” endpoint, which returns a large set of variables for identified aircraft within specific bounds of airspace. I am trying to identify changes to this endpoint or a separate endpoint that will allow me to query a specific flight number or callsign, since this is what I need my program to do.

Table 3.6.1(b)

Test ID	Input	Expected Output	Status, Actual Output and Notes
1	“Search Test” button push.	The default return (see figure 3.6.3) should be returned to the console.	PASS:

```
-TAB_GROUP- {3: None, '-TAB_GROUP-': 'Search for Information'}
-SEARCH_BTN- {3: None, '-TAB_GROUP-': 'Search for Information'}
icao='ESSA' iata='ARN' name='Stockholm Arlanda Airport'
```

Table 3.6.2

1	“Search for Flight” and “Search for Airport” buttons push for both respective categories under “search_buttons”, being both “Flights” and “Airports”.	Pressing “Search for Flight” should return information for a flight, as in Figure 3.6.4 Pressing “Search for Airport”, should return the same result as in Figure 3.6.3	FAIL - Buttons did not appear when selecting the specific menu options. Potentially an indentation issue.
2	“”	“” and buttons should appear under	FAIL - “”. Potentially a key identifier

		each category.	issue. Will add a key for the “search_buttons” group of attributes.
3	““”	““”	PART PASS - The buttons appear but the “get_searchFLT” function returns an API linked error.
4	““”	““”	PASS - Used the example API query until a potential solution or workaround is found.

Table 3.6.3

1	“Search for Flight” button push.	The data for the “Flight Summary Full” endpoint should be returned. I can then use that data to call another endpoint. This allows me to convert a flight number into a hexadecimal identifier.	FAIL - Misplace of comma.
2	““”	““”	FAIL - Will use the entire provided example over partial.
3	““”	““”	FAIL - Did not import the “datetime” library.
4	““”	““”	FAIL - Need a later date to meet the subscription plan.
5	““”	““”	PASS:

```
data=[FlightSummaryFull(fr24_id='380ce8ef', flight='SK1415', callsign='SAS1415',
```

```

297 def get_searchAPT(SearchEntApt): 1 usage
298 #Uses "client" as the instance of the FlightRadar24 API key.
299 with Client(api_token="0198c22c-876f-718e-bca1-a360872c8820|Y6pXXBQN250dYwViBPrqGsPWVr7q8AoPLYkCzQBA832cb895") as client:
300
301     #Declares the variable "airport" and uses the API key instance to return a desired collection of
302     #attributes under the "get_light" pre-determined function, with the airport ICAO code as a parameter.
303     airport = client.airports.get_light(code = SearchEntApt)
304
305     return airport

```

Figure 3.6.5 ▲

Ln 297: Declared the “get_search” function, which I created to put the “client.airports.get_light” query statement within and returns attributes and their instances to the “airport” variable.

“client.airports.get_light” consists of “client” being the instance of the API token (that ensures an application should have access to the API, credibility), “airports” being the desired collection of attributes queried for and “get_light” being the function that returns the FlightRadar24 pre-determined set of attributes (light being a smaller, more condensed set of attributes).

Ln 305: Returns the values (returned from the API query), stored in “airport” so that they can be printed to the console for the user (during the test stage).

```

307 #Defines the "get_searchFLT()" function which searches for flight instances.
308 def get_searchFLT(SearchEntFlt): 1 usage
309 with Client(api_token="0198c22c-876f-718e-bca1-a360872c8820|Y6pXXBQN250dYwViBPrqGsPWVr7q8AoPLYkCzQBA832cb895") as client:
310
311     #Performs an API query using the "flight_positions.get_full()" function. This query uses
312     #the user input, identifier "SearchEntFlt", as the value for the "flights" parameter.
313     flight_positions = client.live.flight_positions.get_full(flights = [SearchEntFlt])
314
315     #Performs an API query using the "flight_summary.get_full()" function. This query uses the
316     #same user input as the value for the "flights" parameter.
317     flight_summary = client.flight_summary.get_full(
318         flights = [SearchEntFlt],
319         #The "TimeFrom" and "TimeTo" functions, part of the "datetime" library
320         #are used to automate the date/time range parameters.
321         flight_datetime_from = TimeFrom.isoformat(timespec='seconds'),
322         flight_datetime_to = TimeTo.isoformat(timespec='seconds'))
323     return flight_summary, flight_positions

```

Figure 3.6.6 ▲

Ln 283: calls the “get_full()” predefined function, of which has the parameters of “bounds”, which is the global positioning boundary to search within.

“client” = The instance of the API token.

“live.flight_positions” = The flight positions endpoint.

The difference between a “get_full()” and “get_light()” function is simply the amount of data returned to the end user from the API query, “full” being most, if not all, available data and “light” being a condensed dataset of key information.

```
282 def get_searchFLT(SearchEntFlt):  
    with Client(api_token="0198c22c-876f-718e-bca1-a360872c8820|  
        flight_summary = client.flight_summary.get_full(  
            ##USE EK184##  
            flights = [SearchEntFlt],  
            flight_datetime_from = datetime(2025, 10, 12),  
            flight_datetime_to = datetime(2025, 10, 22)  
        )  
290 print(flight_summary)
```

Figure 3.6.6(a) ▲

Further version of **Figure 3.6.6** where users will be able to input flight numbers into the API call rather than having to use a default flight number that is provided with the API sandbox.

Ln 282: Defines the “get_searchFLT()” function with the parameter “SearchEntFlt” which is the inputted string of the flight number that the user wishes to search for.

Ln 283-288: Using the equivalent API token, I can call the flight_summary.get_full() API procedure (consisting of the flight_summary endpoint as a full extensive set of data) and store the result in the flight_summary variable. The API procedure has the parameters “flights” (the flight number, which is retrieved from the SearchEntFlt variable in the “main_values” dictionary), “flight_datetime_from” and “flight_datetime_to”, which is the range which the API will return data from regarding flights.

Table 3.6.4				
Test	Alteration	Input	Expected Output	Status / Output / Notes
1	Working on a way to display the API query graphically, rather than via the terminal (used when testing).	Airport ICAO of "EGNX".	The API should return flight information and display it as a GUI element.	<u>FAIL</u> Output doesn't appear.
2	Due to it being adapted from a previous line of code, I think I need to remove the key identifier on line 324 See below:	""	""	<u>FAIL</u> ""
<pre> 322 return flight_summary 323 324 SearchAptOutput = [[GUI.Text(text=flight_summary, expand_x=True, justification='l', font=("Arial", 11), visible=True)]] </pre>				
3	The "flight_summary" variable doesn't exist outside of the "get_searchFLT" function and therefore, nothing is stored within it. For fix, see below:	""	""	<u>PASS</u> ""
<pre> 386 if event == '-SEARCHBT_FLT-': 387 flight_summary = get_searchFLT(SearchEntFlt) 388 window3["-SEARCHOUT-"].update(flight_summary) </pre>				

Table 3.6.5				
Test	Alteration	Input	Expected Output	Status / Output / Notes
1	Created a function to duplicate output elements, for API queries, to eliminate redundancy See below:	API query input.	The API should return information graphically. This should be repeatable for any query.	FAIL Forgot to add the “KEY” parameter in later code, where element groups are collated into one variable (for the “window3_run” function parameter).
<pre> 356 #TODO This can be put into a function, pass the parameter of what to show (list index) and the key. 357 def get_SearchOutput(KEY): 2 usages 358 return [[GUI.Text(expand_x=True, justification='l', font=("Arial", 11), key=KEY, visible=False)]] </pre>				
2	Added the “KEY” parameter into the “get_SearchOutput” function, within the “main_Flayout” variable group.	“”	“”	FAIL “KEY” doesn’t exist as a parameter outside of the function.
3	Will define the variable above the function.	“”	“”	FAIL Will remove this feature for now, as part of a heuristic development.

```

-- Flight Info --
Airline: SAS // Flight: SK1415
Aircraft: A20N // Callsign: SAS1415
Departure Airport: ARN // Departure Time: 2023-01-27T05:15:22

ETA: 2023-12-15T14:47:28Z
Likely Late

-- Nerdy Stats --
Tracking and Data Source: ADSB
Squawk Code: 0734
Aircraft Registration: EI-SIN

```

Figure 3.6.6(b) ▲

A finalised prototype of the flight data return (from the two API queries) of **Figure 3.6.2(c)**, where specific attributes have been returned and labelled. The Addition of “Nerdy Stats” has also been included for documentation completeness.

```

428 # Ln 441-447: allows me to calculate a rough idea to whether a flight is early / late.
429 ExpectedDuration = (flight_summary.data[0].circle_distance / 852) + 0.25
430 # By default, "Z" is included within the date/time string, dictating UTC time.
431 # "Z" cannot be included for mathematical calculations, so is replaced with a blank space.
432 ActualDuration = datetime.fromisoformat(
433     flight_positions.data[0].eta.replace("Z", "")) - datetime.fromisoformat(
434     flight_summary.data[0].datetime_takeoff)
435
436 # the ".total_seconds()" / 3600 function and operation converts
437 # seconds into hours, for standardised units for calculation.
438 if (ActualDuration.total_seconds() / 3600) < ExpectedDuration:
439     status = "Likely Early"
440 if (ActualDuration.total_seconds() / 3600) > ExpectedDuration:
441     status = "Likely Late"

```

Figure 3.6.6(c) ▲

Since the FlightRadar24 API doesn't include a "scheduled time of arrival" attribute, I had nothing to generically compare the "estimated time of arrival" attribute to, in order to determine whether a flight was on time or delayed.

Therefore, using **heuristics** I formed a calculation which allows me to reasonably estimate whether a flight is delayed / on time, through the following:

Ln 429: Divides the flight circle distance (straight line distance from point A to point B, in kilometres) by the approximate cruising speed of an aircraft (in kilometers/hour), to give the estimated time taken of the flight. Plus a reserve.

Ln 432-434: Subtracts the time of departure from the estimated time of arrival, to give the estimated actual duration of the flight.

Ln 438-441: Uses the results of above calculations to determine whether a flight is (very roughly) late or on time.

```

446 window3["-FLIGHTSUMMARY-"].update(str("-- Flight Info --" + "\nAirline: " + flight_summary.data[0]
447 window3["-FLIGHTPOSITIONS-"].update(str("\nETA: " + flight_positions.data[0].eta + "\n" + status),
448
449 #If the "Show Stats For Nerds" checkbox is checked, then the following additional
450 #attributes will be displayed, after a flight is searched for.
451 if main_values["-NERD_FLT-"]:
452     window3["-NERD-"].update(str("\n-- Nerdy Stats --" + "\nTracking and Data Source: " + flight_p
453 if not main_values["-NERD_FLT-"]:
454     window3["-NERD-"].update(visible=False)

```

Figure 3.6.6(d) ▲

Ln 451-454: If the checkbox has been ticked (to "show stats for nerds") then the "-NERD-" text element will be updated to show additional information, of my choosing, belonging to that query. If the checkbox has not been ticked, the text element is made not visible.

Validation Triggers

GUI Elements

```

313     if active_tab == 'Search for Information':
314         window3['-SEARCH_BTTN-'].update(visible=True)
315         if event == '-SEARCH_BTTN-':
316             get_search()

```

Figure 3.6.7 ▲

Code that executes the initial testing of API returns with event triggers (see Table 3.6.1 and Figure 3.6a).

Ln 313-314: A selection statement where if the “active_tab” variable, set by the “main_tabs” group attributes of elements (see Figure 3.5.2(a)), is “Search for Information”, AKA, the user is on the “Search for Information” tab, then the search button (indicated by the “-SEARCH_BTTN-” key) has its visibility attribute set to True.

Ln 315-316: When a button is pressed, it triggers an event with the name equal to the elements key identifier; if the event is equal to the search button element key identifier (“-SEARCH_BTTN-”), AKA, the search button has been pressed: the “get_search()” function is called. Which returns the default API call as in testing (see Table 3.6.1 and Figure 3.6.3 / 3.6.4).

```

326     if active_tab == "Search for Information":
327         if main_values["-MENU_OPT-"] == "Airports::-APT-":
328             window3['-SEARCH_APT-'].update(visible=True)
329             if event == '-SEARCH_APT-':
330                 get_searchAPT()
331
332     if active_tab == "Search for Information":
333         if main_values["-MENU_OPT-"] == "Flights::-FLT-":
334             window3['-SEARCH_FLT-'].update(visible=True)
335             if event == '-SEARCH_FLT-':
336                 get_searchFLT()

```

Figure 3.6.7(a) ▲

Ln 326-330: Same as Figure 3.6.7 descriptions but different dictionary index keys (e.g. “-MENU_OPT-”

Ln 332-334: If the “active_tab” is set to “Search for Information”, meaning that the user is currently on the “Search for Information” tab **and** the selected menu option, stored in the dictionary index key “-MENU_OPT-”, is “Flights” (“Flights::-FLT-”), then the “search for flights” has its “visible” attribute set to “True”.

Ln 335-336: If the user presses the “search for flights” button then an event will be triggered and identified with the index key “-SEARCH_FLT-”, where the “get_searchFLT()” function will then be called, to return an API query.

```
326     if active_tab == "Search for Information":
327         if main_values["-MENU_OPT-"] == "Airports::-APT-":
328             window3['-SEARCH_APT-'].update(visible=True)
329             if event == '-SEARCH_APT-':
330                 get_searchAPT()
331         if main_values["-MENU_OPT-"] == "Flights::-FLT-":
332             window3['-SEARCH_FLT-'].update(visible=True)
333             if event == '-SEARCH_FLT-':
334                 get_searchFLT()
```

Figure 3.6.7(b) ▲

Optimised version of **Figure 3.6.7(a)**.

```

#Displays all GUI elements required to form an API query, to search for an
airport.
    if main_values["-MENU_OPT-"] == "Airports::-APT-":
        window3["-SEARCHBT_APT-"].update(visible=True)
        window3["-SEARCHENT_APT-"].update(visible=True)
        #Hides unnecessary elements.
        window3["-SEARCHBT_FLT-"].update(visible=False)
        window3["-SEARCHENT_FLT-"].update(visible=False)

        if event == '-SEARCHBT_APT-':
            #Triggers the "get_searchAPT" API query function with the user
input assigned to
            #SearchEnt
Apt", assigned to a dictionary identifier, being passed as a parameter.
            airport = get_searchAPT(SearchEntApt)
            #Updates the "-AIRPORT-" GUI element with the data stored in
"airport".
            window3["-AIRPORT-"].update(airport, visible=True)

#Displays all GUI elements required to form an API query, to search for a
flight.
    if main_values["-MENU_OPT-"] == "Flights::-FLT-":
        window3['-SEARCHBT_FLT-'].update(visible=True)
        window3['-SEARCHENT_FLT-'].update(visible=True)
        window3["-SEARCHBT_APT-"].update(visible=False)
        window3["-SEARCHENT_APT-"].update(visible=False)

        if event == '-SEARCHBT_FLT-':
            #Triggers the "get_searchFLT" API query function with the user
input
            #assigned to "SearchEntFlt", being passed as a parameter.
            flight_summary, flight_positions = get_searchFLT(SearchEntFlt)
            #Updates the respective GUI elements with the data stored in...
            window3["-FLIGHTSUMMARY-"].update(flight_summary, visible=True)
            window3["-FLIGHTPOSITIONS-"].update(flight_positions,
visible=True)

#Redundancy to ensure that respective GUI elements are hidden,
#should the user leave the "Search for Information" tab.
else:
    window3['-SEARCHBT_APT-'].update(visible=False)
    window3['-SEARCHBT_FLT-'].update(visible=False)
    window3['-SEARCHENT_FLT-'].update(visible=False)
    window3['-SEARCHENT_APT-'].update(visible=False)

```

Figure 3.6.7(c) ▲

Finalised version of **Figure 3.6.7(b)**.

For calling API query functions, I have assigned dictionary identifiers to variables beforehand. This makes referencing them easier, since throughout my program, the dictionary name changes quite frequently, depending on the window, tab, menu, etc, being accessed.

3.7 “Manage Alerts” Tab

General GUI Development

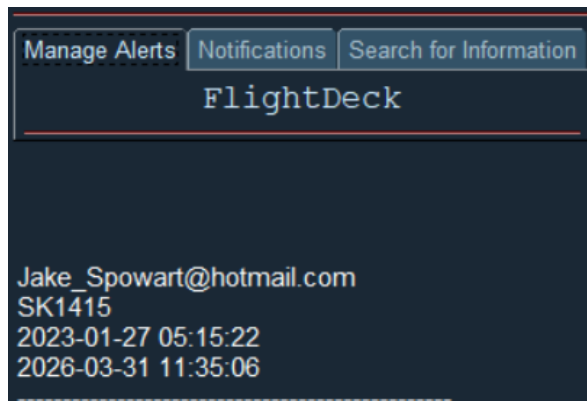


Figure 3.7.1(a) ▲

An initial prototype of the “Manage Alerts” tab, within the main window. In **Figure 3.7.1**, you can see that I have created an alert for a flight as, when selecting this tab, an SQL query (see **Figure 3.7.2(b)**) is executed and details regarding the alert are displayed (see **Figure 3.7.2(c)**).

In a future prototype, I will include the ability to delete a specific alert.

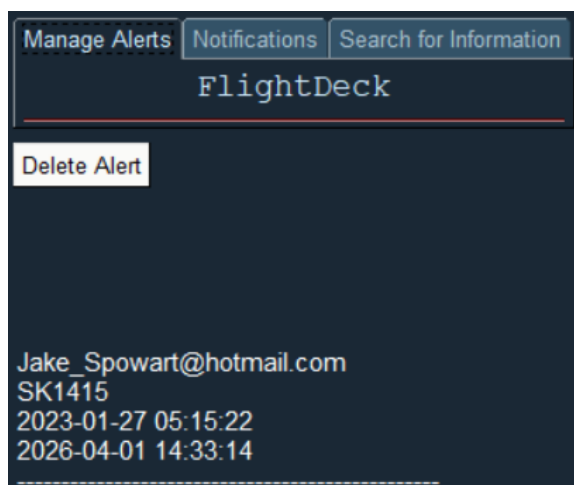


Figure 3.7.1(b) ▲

A final prototype of **Figure 3.7.1(a)** with the ability to delete any previously created alerts (by entering the flight number into a text input element out of shot).

Table 3.7.1 - Deleting a previously created alert (using final prototype).

Test	Input	Expected Output	Status / Output / Notes
1	Enter the flight number of an alert (requirement: must be previously created and appear, as in Figure 3.7.1(b)) and double press the “Delete Alert” button.	The flight alert should be deleted from the “alerts” table within the “flightdeckdb” database. Therefore, that alert should be removed from the “Manage Alerts” tab.	PASS <i>See below:</i>

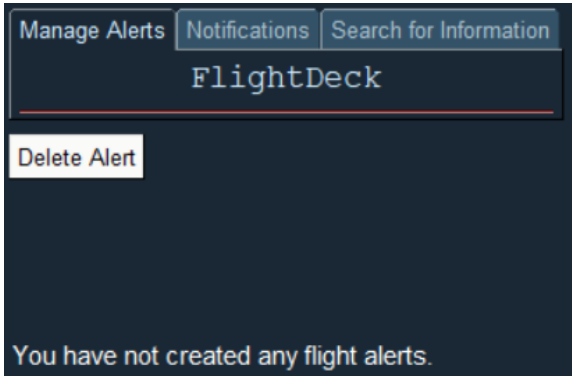


Figure 3.7.1(c) ▲

API Integration, Query and SQL Functions

```

461 #If the "Create An Alert For This Flight" checkbox has been selected...
462 if main_values["-ALERTCHK_FLT-"]:
463     ExistAlert = "SELECT `EmailAddress`, `FlightNumber` FROM alerts WHERE `EmailAddress` = %s AND `FlightNumber` = %s"
464     #Executes the above SQL statement "ExistAlert" and passes the values of "email_address_per" /
465     #"email_address_co" and the flight number of the queried flight into the respective parameters above.
466     Cursor.execute(ExistAlert, (email_address_per or email_address_co), flight_summary.data[0].flight)
467     #Fetches one (the first) value.
468     ExistAlertResult = Cursor.fetchone()
469
470 if not ExistAlertResult:
471     AlertSql = "INSERT INTO alerts (`EmailAddress`, `FlightNumber`, `DepartureTime`, `NotificationTime`) VALUES (%s, %s, %s, %s)"
472     AlertValues = ((email_address_per or email_address_co), flight_summary.data[0].flight, flight_summary.data[0].datetime, flight_summary.data[0].datetime)
473     Cursor.execute(AlertSql, AlertValues)
474     #Commits the values to the database under one transaction.
475     db.commit()

```

Figure 3.7.2 ▲

An initial nested selection statement prototype (initially unsuccessful, see Test Tables > T13) that commits the values needed to sustain a user-created flight alert, under specific conditions:

- If the “Create An Alert For This Flight” checkbox is checked.
 - If an alert hasn’t already been created for the searched individual flight under that individual user.
 - The user email address, flight number, flight departure time and current timestamp are committed to the “alerts” table within the “flightdeckdb” database, under the appropriate attributes.
 - If an alert has already been created for the searched individual flight under that individual user.
 - A future prototype will display some form of error message, as well as including potential validation re-attempt methods.
- If the “Create An Alert For This Flight” checkbox is unchecked.
 - An alert will not be created.

AlertID	EmailAddress	FlightNumber	Departure Time	Notification Time
5	Jake_Spowart@hotmail.com	SK1415	2023-01-27 05:15:22.000000	2026-03-26 21:11:20.000000

Figure 3.7.2(a) ▲

The first test result of committing a record, from the logic and SQL statements in **Figure 3.6.2**.

Table 3.7.1			
Test ID	Input	Expected Output	Status, Actual Output and Notes
1	After having created a flight alert, switch to the "Manage Alerts" tab.	The "-ALERTCREATED-" output element should be updated to show the result stored in the "alerts" table, within my database.	FAIL: Additional information not shown. See below and Test Table T12.

```

497     if active_tab == "Manage Alerts":
498         ExistAlert2 = "SELECT * FROM alerts WHERE `EmailAddress` = %s"
499         # Executes the above SQL statement "ExistAlert" and passes the values of "email_address_per" /
500         # "email_address_co" and the flight number of the queried flight into the respective parameters above.
501         Cursor.execute(ExistAlert2, params: (GlobalEmail,))
502         # Fetches one (the first) value.
503         ExistAlertResult2 = Cursor.fetchone()

```

Figure 3.7.2(b) ▲

Should the user switch to the "Manage Alerts" tab, an SQL query has to be performed in order to return any alerts that they have created. Since they are stored on a database, rather than within program memory.

Ln 498: Selects all records from the "alerts" table within the "flightdeckdb" database, if the "EmailAddress" attribute is equal to that stored in "GlobalEmail" in **Ln 501**. If the user has created a flight alert, the record will be returned and stored in the "ExistAlertResult2" variable.

The SQL statement is stored within the variable "ExistAlert2".

```
505     if ExistAlertResult2:
506         Separator = ""
507         #len() function determines the number of chars' within
508         #ExistAlertResult2 to create a line break.
509         l = len(ExistAlertResult2[1]) * 2
510         for i in range(l):
511             Separator = Separator + "-"
512
513         Output = ""
514         #Returns the 2nd, 3rd and 4th attribute within
515         #the returned "ExistAlertResult2" list.
516         for i in range(1, 4):
517             #Concatenates the previous value of "Output" with the value
518             #at index "i" from "ExistAlertResult2", plus a line break.
519             Output = Output + str(ExistAlertResult2[i]) + "\n"
520         Output = Output + Separator
521         #Passes the "Output" as a parameter into the ".update()" function.
522         window3["-ALERTCREATED-"].update(Output, text_color="#FFFFFF", visible=True)
```

Figure 3.7.2(c) ▲

Ln 505: Should a data structure be returned to “ExistAlertResult2”, this means that the user has already created an alert using the logged in email address. Therefore, it can be returned.

Ln 513-520: Since there is a discreet number of attributes (four of them) within the “ExistAlertResult2” list (excluding the primary key “AlertID”), I can use a for loop to return them. Concatenating the previous value at the index, to the next, and a line break.

Ln 522: Updates the GUI text element with the identifier “-ALERTCREATED-” to display the result of the iterated values, under the “Manage Alerts” tab.

If a result is not returned (as an alert has not been created), the “-ALERTCREATED-” element is updated to state this.

3.8 “Notifications” Tab

General GUI Development

```
575     if active_tab == "Notifications":  
576         Notification = "Test"  
577         window3["-NOTIFICATION-"].update(Notification, visible=True)  
578  
579     else:  
580         window3["-NOTIFICATION-"].update(visible=False)
```

Figure 3.8.1(a) ▲

Ln 575-577: If the user switches to the “Notifications” tab within the main window, an update is pushed, to set the “-NOTIFICATION-” output elements “visible” attribute to “True” (making it visible to the user).

As part of testing, the variable “Notification” is declared above and some basic text is assigned. “Notification” is then passed into the “.update()” function for the “-NOTIFICATION-” output element.

For Figure 3.8.1(b).

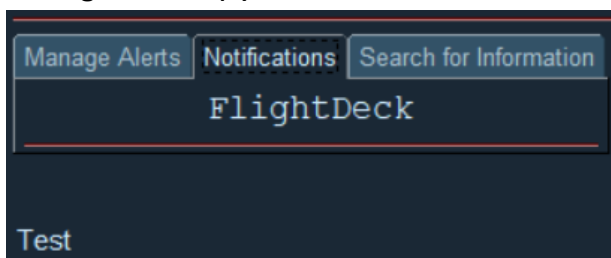


Figure 3.8.1(b) ▲

An initial prototype of the “Notifications” tab, to ensure that the GUI output element functions and displays correctly.

From Figure 3.8.1(a).

API Integration, Query and SQL Functions

```

574     if active_tab == "Notifications":
575         #Sets the "visible" attribute of the "-NOTIFICATION-"
576         #output element to "True"
577         window3["-NOTIFICATION-"].update(visible=True)
578
579         ExistAlert3 = "SELECT * FROM alerts WHERE `EmailAddress` = %s"
580         # Executes the above SQL statement "ExistAlert" and passes the values of "email_address_per" /
581         # "email_address_co" and the flight number of the queried flight into the respective parameters above.
582         Cursor.execute(ExistAlert3, params: (GlobalEmail,))
583         # Fetches one (the first) value.
584         ExistAlertResult3 = Cursor.fetchall()
585
586         #If alert creation time < flight departure time
587         if ExistAlertResult3[4] < ExistAlertResult3[3]:
588             flight_summary, flight_positions = get_searchFLT(SearchEntFlt)
589
590             #Flight departure time - current time = time until departure (via "timestamp"
591             #attribute, under the FlightRadar24 API "flight_positions" endpoint.
592             TDeparture = ExistAlertResult3[3] - flight_positions.data[10]
593             print(TDeparture)

```

Figure 3.8.2(a) ▲

Ln 579-582: Stores the SQL query, which selects all existing records (flight alerts) within the “alerts” table, where the email address is equal to the value stored in the “GlobalEmail” variable (of the logged in user), within the “ExistAlert3” variable.

The “Cursor.execute()” function is then executed, with the parameters “ExistAlert3” (the SQL query) and “GlobalEmail” to be inserted into the “%s” placeholder as the “EmailAddress” attribute. The results are returned to the “ExistAlertResult3” variable.

Ln 586-593:

- If the date-time value at index position 4 of “ExistAlertResult3” is less than the date-time value at index position 3 of “ExistAlertResult3”.
 - The “get_searchFLT()” function is called, with the parameter “SearchEntFlt” (the flight number), in order to access a current date-timestamp. The results are stored in accordance with the two API endpoints within this function.
 - “TDeparture” (the time until departure) is calculated by subtracting “flight_positions.data[10]” (the current time) from “ExistAlertResult3[3]” (departure time).

Table 3.8.1			
Test ID	Input	Expected Output	Status, Actual Output and Notes
1	After having created a flight alert, switch to the “Notifications” tab.	The “ExistAlert3” SQL query should be executed and logical operations carried out to later print “TDeparture” or “Test Complete”.	FAIL: <i>See below and Test Table T12:</i>
<pre> if ExistAlertResult3[4] < ExistAlertResult3[3]: ~~~~~^ IndexError: list index out of range </pre>			

```

579 ExistAlert3 = "SELECT * FROM alerts WHERE `EmailAddress` = %s"
580 # Executes the above SQL statement "ExistAlert" and passes the values of "email_address_per" /
581 # "email_address_co" and the flight number of the queried flight into the respective parameters above.
582 Cursor.execute(ExistAlert3, params: (GlobalEmail,))
583 # Fetches one (the first) value.
584 ExistAlertResult3 = Cursor.fetchall()
585
586 #If alert creation time < flight departure time
587 if ExistAlertResult3[0][4] < ExistAlertResult3[0][3]:
588     flight_summary, flight_positions = get_searchFLT(SearchEntFlt)
589
590     #Flight departure time - current time = time until departure (via "timestamp"
591     #attribute, under the FlightRadar24 API "flight_positions" endpoint.
592     TDeparture = ExistAlertResult3[0][3] - flight_positions.data[10]
593     print(TDeparture)
594 else:
595     print("Test Complete")

```

Figure 3.8.2(b) ▲

A further prototype of **Figure 3.8.2(a)** (initially unsuccessful, see **Table 3.8.1**), where the data structures within “ExistAlertResult3” are indexed correctly.

- “Cursor.fetchone()” function.
 - Returns a singular tuple for the singular (or only first) record returned.
 - Tuple → Index → Value.
- “Cursor.fetchall()” function.
 - Returns a list of potentially multiple tuples for multiple records returned.
 - List → Index → Tuple → Index → Value.

```

574 if active_tab == "Notifications":
575     ExistAlert3 = "SELECT * FROM alerts WHERE `EmailAddress` = %s"
576     # Executes the above SQL statement "ExistAlert" and passes the values of "email_address_per" /
577     # "email_address_co" and the flight number of the queried flight into the respective parameters above.
578     Cursor.execute(ExistAlert3, params: (GlobalEmail,))
579     # Fetches one (the first) value.
580     ExistAlertResult3 = Cursor.fetchall()
581
582     SearchEntFlt = ExistAlertResult3[0][2]
583     flight_summary, flight_positions = get_searchFLT(SearchEntFlt)
584
585     #If alert creation time < flight departure time
586     if ExistAlertResult3[0][4] < ExistAlertResult3[0][3]:
587
588         #Flight departure time - current time = time until departure (via "timestamp"
589         #attribute, under the FlightRadar24 API "flight_positions" endpoint.
590         TDeparture = "Time Until Departure: ", (ExistAlertResult3[0][3] - flight_positions.data[10])
591         window3["-NOTIFICATION-"].update(TDeparture, visible=True)
592
593     else:
594         InFlightInfo = "-- Journey Data --" + "\nDeparture Time: " + flight_summary.data[0].datetime_takeoff +
595         window3["-NOTIFICATION-"].update(InFlightInfo, visible=True)

```

Figure 3.8.2(c) ▲

A further prototype of **Figure 3.8.2(b)**.

“SeachEntFlt” was deemed out of scope, therefore, in Ln 582, the flight number returned from the SQL query “ExistAlert3” (the index value given) was assigned to “SearchEntFlt”.

```

585 #Ensures the two date-times are of the same format, for mathematical operations.
586 TDeparture = datetime.fromisoformat(
587     flight_positions.data[0].timestamp.replace("Z", "")) - datetime.fromisoformat(
588     str(ExistAlertResult3[0][3]))

```

Figure 3.8.2(d) ▲

A replacement for **Ln 590 in Figure 3.8.2(c)** as “flight_positions.data[0].timestamp” and “ExistAlertResult3[0][3]” are both date-time values but are of a different format. Using the datetime library, I can make them the same format, so that mathematical operations can be carried out.

For example, without this, “flight_positions.data[0].timestamp” would contain the letter “Z” suffix, indicating it is of the UTC timezone. The .replace() function, as seen, removes the timezone indicator.

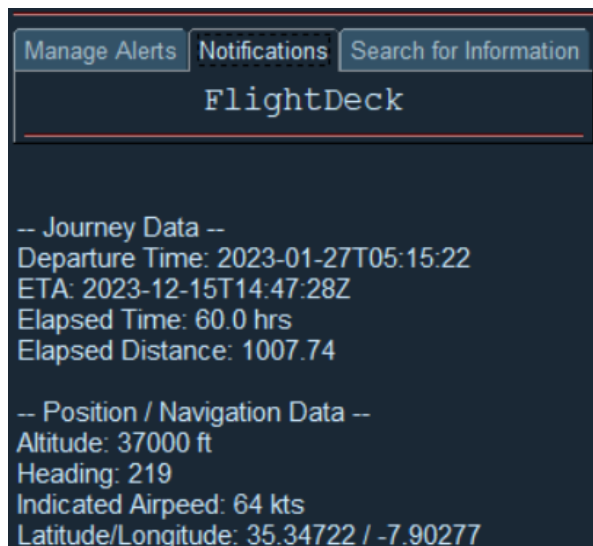


Figure 3.8.2(e) ▲

A final prototype of **Figure 3.8.1(b)**, where specific attributes are returned if the user is currently viewing the “Notifications” tab and the flight has already departed.

From **Ln 594** in **Figure 3.8.2(c)**.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	AlertID	int(10)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	EmailAddress	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
☐ 3	FlightNumber	varchar(10)	utf8mb4_general_ci		No	None			Change Drop More
☐ 4	DepartureTime	datetime(6)			No	None			Change Drop More
☐ 5	CreationTime	datetime(6)			No	None			Change Drop More
☐ 6	LastAlerted	datetime(6)			Yes	NULL			Change Drop More

Figure 3.8.2(f) ▲

An update to the “alerts” table, within the “flightdeckdb” database, to include the “LastAlerted” attribute, which will be used in later logic to determine whether an alert (in the “Notifications” tab) has been pushed within the last 20 minutes. If not, another needs to be.

Code Appendix (Selection Statements)

As will not fit into a singular screenshot without disrupting patterns of indentations.

```

if active_tab == "Notifications":
    ExistAlert3 = "SELECT * FROM alerts WHERE `EmailAddress` = %s"
    Cursor.execute(ExistAlert3, (GlobalEmail,))
    # Fetches one (the first) value.
    ExistAlertResult3 = Cursor.fetchall()

    SearchEntFlt = ExistAlertResult3[0][2]
    flight_summary, flight_positions = get_searchFLT(SearchEntFlt)

    if flight_summary.data[0].flight_ended:
        window3["-NOTIFICATION-"].update("This flight has ended.",
visible=True)
        return

    #Ensures the two date-times are of the same format, for mathematical
operations.
    TDeparture =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][3]))

    #If current time is prior to the flight departure time (BEFORE
DEPARTURE)...
    if datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z",
"")) < ExistAlertResult3[0][3]:

        #If the time until departure is less than 4 hours...
        if TDeparture < timedelta(hours=4):

            #If no alert has been previously "pushed" for this flight...
            if ExistAlertResult3[0][5] is None:
                #Flight departure time - current time = time until departure
(via "timestamp"
                #attribute, under the FlightRadar24 API "flight_positions"
endpoint).
                window3["-NOTIFICATION-"].update("Time Until Departure: " +
TDeparture, visible=True)
                TAlert = None

                #Updates the "LastAlerted" attribute with the current
timestamp.
                UpdateAlertSql = "UPDATE alerts SET `LastAlerted` = %s WHERE
`EmailAddress` = %s"
                UpdateAlertValues =
(datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")),
GlobalEmail)
                Cursor.execute(UpdateAlertSql, UpdateAlertValues)
                db.commit()

```

```

        #Else (if an alert has been previously "pushed" for this
flight)...
        else:
            #TAlert (time since last alert) is calculated.
            TAlert =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][5]))

            #If the time since the last alert (TAlert) is >= 20 minutes...
            if TAlert >= timedelta(minutes=20):

                #An up-to-date API query is made.
                flight_summary, flight_positions =
get_searchFLT(SearchEntFlt)
                #Calculates an up-to-date "TDeparture" and ensures the two
date-times
                #are of the same format, for mathematical operations.
                TDeparture =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][3]))

                #Outputs "TDeparture"
                window3["-NOTIFICATION-"].update("Time Until Departure: "
+ TDeparture, visible=True)

                #Updates the "LastAlerted" attribute with the current
timestamp.
                UpdateAlertSql = "UPDATE alerts SET `LastAlerted` = %s
WHERE `EmailAddress` = %s"
                UpdateAlertValues =
(datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")),
GlobalEmail)
                Cursor.execute(UpdateAlertSql, UpdateAlertValues)
                db.commit()

            #If the time until departure is greater than 4 hours...
            if TDeparture > timedelta(hours=4):
                #Output statement.
                window3["-NOTIFICATION-"].update("Alerts only appear 4 hours in
advance to departure" + "(" + TDeparture + ")", visible=True)

        #Else (if current time is beyond the flight departure time (AFTER
DEPARTURE))...
        else:
            if ExistAlertResult3[0][5] is None:
                TAlert = None
            else:
                TAlert =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][5]))

                if TAlert is not None and TAlert >= timedelta(minutes=20):

```

```

        #An up-to-date API query is made.
        flight_summary, flight_positions = get_searchFLT(SearchEntFlt)
        #Calculates an up-to-date "TDeparture" and ensures the two
date-times
        #are of the same format, for mathematical operations.
        TDeparture =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][3]))

        #Converts elapsed flight time into hours.
        ElapsedTime = (flight_summary.data[0].flight_time / 60 / 60)
        InFlightInfo = "-- Journey Data --" + "\nDeparture Time: " +
flight_summary.data[0].datetime_takeoff + "\nETA: " +
flight_positions.data[0].eta + "\nElapsed Time: " + str(ElapsedTime) + " hrs"
+ "\nElapsed Distance: " + str(flight_summary.data[0].actual_distance) +
"\n\n-- Position / Navigation Data --" + "\nAltitude: " +
str(flight_positions.data[0].alt) + " ft" + "\nHeading: " +
str(flight_positions.data[0].track) + "\nIndicated Airspeed: " +
str(flight_positions.data[0].vspeed) + " kts" + "\nLatitude/Longitude: " +
str(flight_positions.data[0].lat) + " / " + str(flight_positions.data[0].lon)
+ "\n\nTo Date As Of " + flight_positions.data[0].timestamp
        window3["-NOTIFICATION-"].update(InFlightInfo, visible=True)

        #Updates the "LastAlerted" attribute with the current timestamp.
        UpdateAlertSql = "UPDATE alerts SET `LastAlerted` = %s WHERE
`EmailAddress` = %s"
        UpdateAlertValues =
(datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")),
GlobalEmail)
        Cursor.execute(UpdateAlertSql, UpdateAlertValues)
        db.commit()

else:
    window3["-NOTIFICATION-"].update(visible=False)

```

Key Branches, Conditions and Logic Statements:

`"if active_tab == "Notifications":"` - All other nested branches and logic statements, while the user is within the "Notifications" tab, lie within this "active_tab" (via the "-TABGROUP-" identifier) comparison. Determines that the user is within the "Notification" tab. If met:

- "ExistAlert3" SQL query is executed (selecting all records from the "alerts" table, where "EmailAddress" is equal to the value stored in "GlobalEmail").
- "SearchEntFlt" is updated with the index value of "ExistAlertResult3[0][2]" (within the first record returned).
- Calls "get_searchFLT()", with the parameter "SearchEntFlt".

`"else:→window3["-NOTIFICATION-"].update(visible=False)"` - Else, any GUI elements used within the "Notification" tab are hidden (their "visible" attribute is set to False).

`"if flight_summary.data[0].flight_ended:"` - If the "flight_ended" boolean is True:

- "TDeparture" (time until departure) is calculated by subtracting the departure time from the current time.

Departure Time - stored in database.

Current Timestamp - returned via FlightRadar24 "flight_positions endpoint".

`"if datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) < ExistAlertResult3[0][3]:"` - If the current time is prior to the departure time:

`"if TDeparture < timedelta(hours=4):"` - If the time until departure is less than 4 hours:

`"if ExistAlertResult3[0][5] is None:"` - If the "LastAlerted" attribute is null:

- Time until departure is displayed to the user.
- The "UpdateAlertSql" SQL statement is made to set "LastAlerted" to the current timestamp.

`"else:"` - "TAlert" (time since last alert) is calculated.

`"if TAlert >= timedelta(minutes=20):"` - If "TAlert" (time since last alert) is less than or equal to 20 minutes:

- An up-to-date API query is made.
- "TDeparture" is re-calculated and updated.
- Updates "LastAlerted" attribute within the "alerts" table.

`“if TDeparture > timedelta(hours=4):”` - If “TDeparture” is greater than 4 hours:

- Alerts can only appear within 4 hours of departure.

`“if TAlert is not None and TAlert >= timedelta(minutes=20):”` - If “TAlert” (time since last alert) is not null (meaning an alert has been previously made) and “TAlert” is greater than or equal to 20 minutes (last alerted 20 minutes+ ago):

- An up-to-date AI query is made.
- “TDeparture” is re-calculated and updated.
- Journey and position/navigation data is extracted from the API query returned lists (“flight_positions” and “flight_summary”), and displayed to the user.

This branch is executed as, under additional logic, it can be assumed that the flight has already departed (but has not ended / landed).

Test Tables

T1 - Configured validation error messages only showing on “Personal Create Account” tab not “Commercial Create Account” tab, should be both.

Figure T.1a▲

Issue - Even though a password of a string length that is less than 8 characters and the password and re-entered password are not identical, the error messages do not appear as they should in figure 4.1.

Figure T.1b▲

Fixed - Issue in Figure T.1a.

Test ID	Suspected Issue / Issue	Input	Expected Output	Status, Actual Output and Notes
1	Validation error messages only appear on the personal tab, even when attempting to create an account on the commercial tab. Separate error messages have been added to the “get_commerce_create()” function and the new unique error message keys have been added to the validation if statements collection.	Email: “Test_Email” Password: “1” Re-enter Password: “1234”	Error messages to show on the commercial account creation tab.	PASS - The error message now displays on both and when an event is triggered, the input boxes are cleared.

T2 - Validation error messages showing on the “Commercial Create Account” tab even when validation criteria met.

The screenshot shows the 'FlightDeck' application interface. At the top, there are two tabs: 'Personal' and 'Commercial'. The 'Commercial' tab is selected. Below the tabs is a form titled 'Create Account'. The form contains four input fields: 'Email Address', 'Password', 'Re-enter Password', and 'Company Name'. The 'Company Name' field contains the text 'Jake and Co.'. Below the input fields, there is a red error message that reads 'Passwords must match.'. At the bottom of the form, there is a 'Create' button and a link that says 'Already have an account? Sign in.'.

Figure T.2a▲

Issue - Even when meeting the validation requirements on the “Commercial Create Account” tab, the error message “Passwords must match.” still appears when clicking the “Create” button to trigger an event.

The screenshot shows the 'FlightDeck' application interface. At the top, there are two tabs: 'Personal' and 'Commercial'. The 'Commercial' tab is selected. Below the tabs is a form titled 'Create Account'. The form contains four input fields: 'Email Address', 'Password', 'Re-enter Password', and 'Company Name'. The 'Company Name' field contains the text 'Test Company'. Below the input fields, there is a green success message that reads 'Account Creation Successful. Please wait.'. At the bottom of the form, there is a 'Create' button and a link that says 'Already have an account? Sign in.'.

Figure T.2b▲

Fixed - Issue in Figure T.2a.

Test ID	Suspected Issue / Issue / Fix	Input	Expected Output	Status, Actual Output and Notes
1	I suspect that it is failing the validation check even though inputs on the commercial tab meet the validation criteria as I assigned both the personal and commercial input keys to the same variable (e.g. the personal password enter key and commercial password enter key are assigned to the same password variable). Therefore, it expects both tabs to match, not just one. I will create different variables for each account type.	Email: "Test_Email" Password: "test1234" Re-enter Password: "test1234" Company name: "Test Company"	Error messages not to appear when validation criteria is met. Data to be committed to the database.	FAIL - The data was not committed to the database as the new commercial email address key was spelled incorrectly. Spelling fixed.
2	Previous fix re-attempt	""	""	FAIL - Same issue as ID 1
3	Separate if statements have been implemented for each individual key, rather than integrating the same validation criteria keys (for personal and commercial) in the same statement.`	""	""	PART PASS - The data is committed to the database and the account creation message shows, however, the "Password must be at least 8 characters long" error message still shows. This may be a similar issue with personal and commercial keys under the same variable. The account created message also only shows on the personal tab.
4	The -8CHAR_ERR_PER- key (key that when set to visible, shows the at least 8 characters error message on the personal tab) was used for the commercial account. Keys have been switched to correspond to their personal and commercial tab error message displays.	""	"Passoword must be at least 8 characters" error message expected to disappear.	PART PASS - Same issue as ID 3 but only shows on the personal tab.
5	Similar to ID 3, I have separated the valid boolean statements into both commercial and personal validation passes.	""	""	PART PASS - Same issue as ID 4.
6	Is still trying to check the personal	""	""	PASS

	password input boxes even when entering into the commercial password input. Hence, when entering into the commercial password input, the personal password will always be less than 8 characters. Altered the if statement where for the error message to appear, the password for that tab has to be less than 8 but the password for the other tab also has to be empty.			
--	---	--	--	--

T3 - Validation error message ("Passwords must match") showing on the "Personal Create Account" tab when validation criteria are not met, but the data is still committed to the database.

The screenshot shows the 'FlightDeck' application interface. At the top, there are two tabs: 'Personal' and 'Commercial'. The 'Personal' tab is selected. Below the tabs is a 'Create Account' form. The form has three input fields: 'Email Address' with the value 'Test', 'Password' with masked characters '*****', and 'Re-enter Password' with masked characters '*****'. Below the form is a 'Create' button and a link that says 'Already have an account? Sign in.'.

Figure T.3a▲

The screenshot shows the 'FlightDeck' application interface. At the top, there are two tabs: 'Personal' and 'Commercial'. The 'Commercial' tab is selected. Below the tabs is a 'Create Account' form. The form has three input fields: 'Email Address', 'Password', and 'Re-enter Password'. Below the form, there is a red error message that says 'Passwords must match.' and a green message that says 'Account Creation Successful. Standby..'. Below the form is a 'Create' button and a link that says 'Already have an account? Sign in.'.

Figure T.3b▲

Issue - The validation criteria that the "Password" and "Re-enter Password" input boxes must match is not met during the test as "Password" = "TestTest" and "Re-enter Password" = "TestTest12". The correct error messages display as they should, however there is an issue with some of the "validated" booleans so the data is committed to the database mistakenly.

FlightDeck

Personal Commercial

Create Account

Email Address

Password

Re-enter Password

Passwords must match.

Create

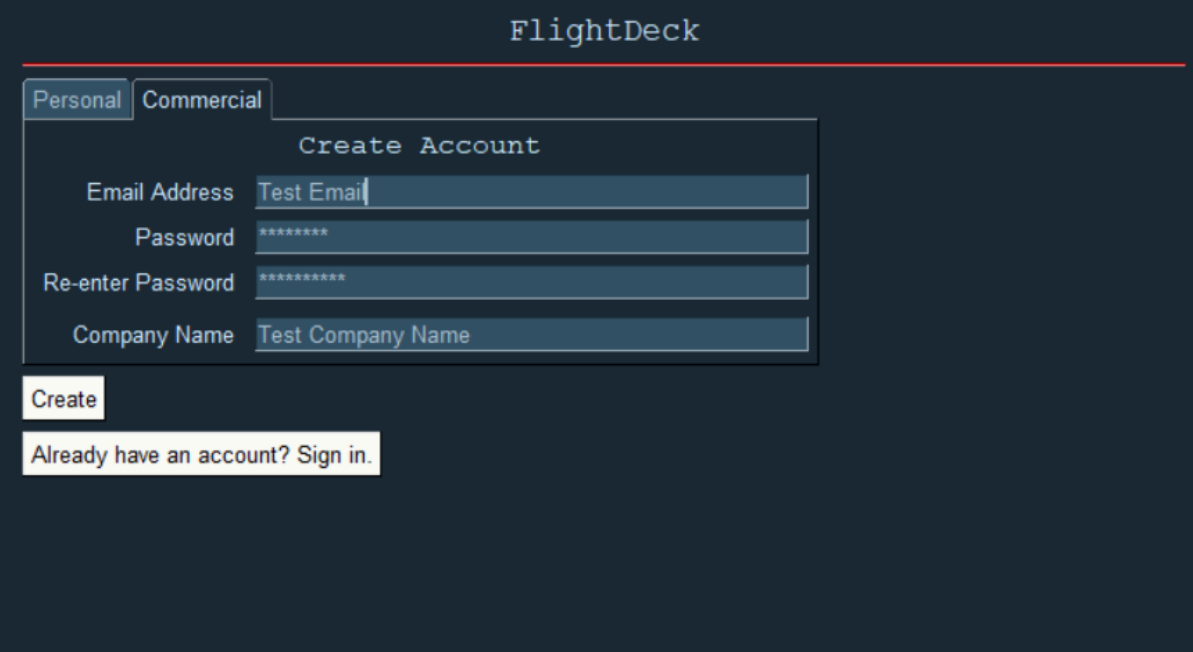
Already have an account? Sign in.

Figure T.3c▲

Fixed - Issue in Figure T.3a and T.3b.

Test ID	Suspected Issue / Issue / Fix	Input	Expected Output	Status, Actual Output and Notes
1	On the validation selection statement (where if the personal password is less than 8 characters and the commercial password is blank, "valid_8char_per" is set to False, else is True). Therefore, if the commercial password input is less than 8 characters, and at the same time, the personal password is blank then "valid_8char_co" is set to True as they are not both met (since the personal password input won't be blank). I think nested statements will be needed for the selection validation with breaks if the condition is met.	Email: "Test" Password: "TestTest" Re-enter Password: "TestTest12"	The "passwords must match" error message should appear. The "account creation successful" message should not appear; data should not be committed to the database as a new record.	PASS

T4 - Validation error message ("Passwords must match") showing on the "Commercial Create Account" tab when validation criteria are not met, but the data is still committed to the database.



FlightDeck

Personal Commercial

Create Account

Email Address Test Email

Password *****

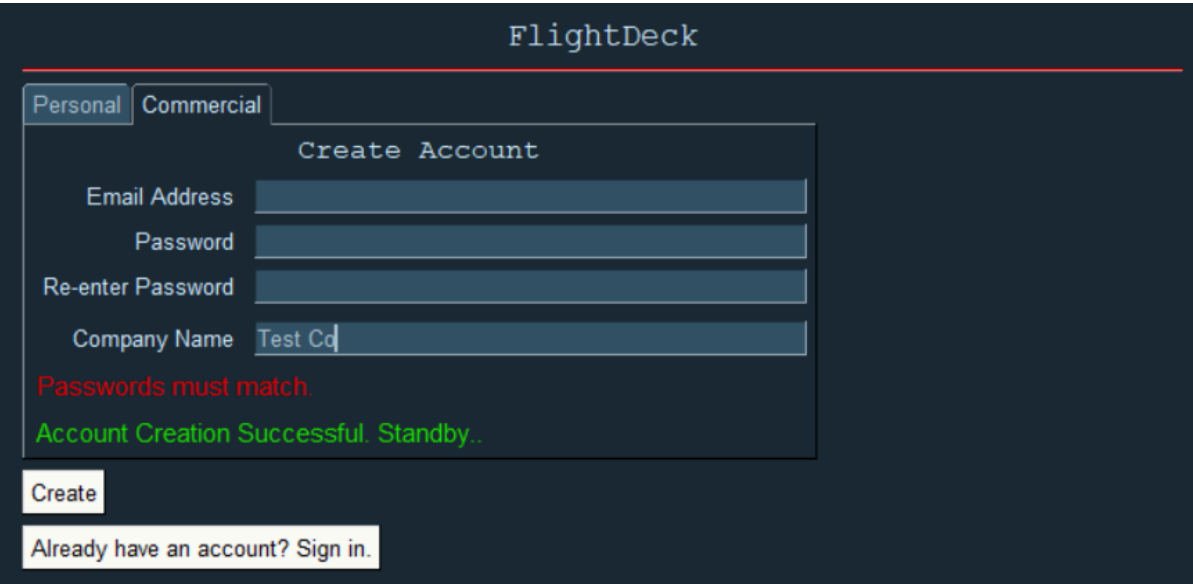
Re-enter Password *****

Company Name Test Company Name

Create

Already have an account? Sign in.

Figure T.4a▲



FlightDeck

Personal Commercial

Create Account

Email Address

Password

Re-enter Password

Company Name Test Co

Passwords must match.

Account Creation Successful. Standby..

Create

Already have an account? Sign in.

Figure T.4b▲

Issue - The validation criteria that the "Password" and "Re-enter Password" input boxes must match is not met during the test as "Password" = "TESTTEST" and "Re-enter Password" = "TESTTEST12". The correct error messages display as they should, however there is an issue with some of the "validated" booleans so the data is committed to the database mistakenly.

Test ID	Suspected Issue / Issue / Fix	Input	Expected Output	Status, Actual Output and Notes
1	The initial declarations of the validation boolean variables, have all been declared as "True" and the validation statements no longer set them as "True", they only correct them to "False" if the conditions aren't met.	Email: "Test4" Password: "TESTTEST" Re-enter Password: "TESTTEST12" Company Name: "Test Co"	The "passwords must match" error message should appear. The "account creation successful" message should not appear; data should not be committed to the database as a new record.	FAIL - Same issue.
2	Only set the initial declarations of the validation boolean variables: "valid_passmatch_per" and "valid_passmatch_co" to "True", "valid_8char_per" and "valid_8char_co" are "False".	Email: "Test6" Password: "TESTTEST" Re-enter Password: "TESTTEST12" Company Name: "Test Co"	The "passwords must match" error message should appear. The "account creation successful" message should not appear; data should not be committed to the database as a new record.	PART PASS - The error messages appear and nothing is committed, however, if the passwords do match, it isn't committed.
3	The validation cannot decipher between whether the user is actively using the personal or commercial tab so inside the event while loop, an "active_tab" variable gets declared with a key to the "values" dictionary to mark which tab the user is using: "values['-TAB_GROUP-']".	Email: "Test7" Password: "TESTTEST" Re-enter Password: "TESTTEST12" Company Name: "Test Co"	The "passwords must match" error message should appear. The "account creation successful" message should not appear; data should not be committed to the database as a new record.	PASS

T5 - Upon entering valid credentials into the sign-in window (window 2), the main window (window3) doesn't open and a very long list of errors are returned.

Test ID	Suspected Issue / Issue / Fix	Input	Expected Output	Status, Actual Output and Notes
1	I previously added a spacing in a statement that declares the API token with a set alias, in order to more easily redact the API token. I have removed the spacing as it previously worked when tested at an earlier date.	N/A	The main window (window3) should open.	FAIL - The same issue occurred.
2	I am attempting to call a button within a "main_tabs" tab; I believe you can only call functions within a tab that contain individual elements, not just individual elements. I have added the button element into the "get_search" function so that it doesn't need to be called individually.	N/A	""	PASS

T6 - A button does not appear under the main window under the “Your Alerts” tab.

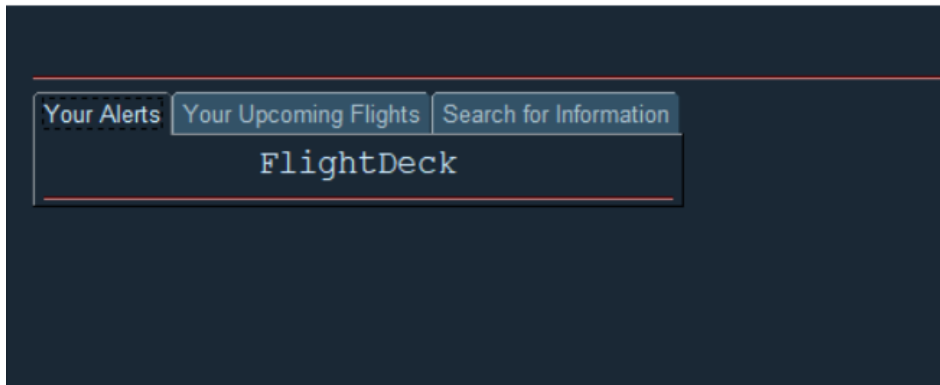


Figure T.4b▲

Issue

Test ID	Suspected Issue / Issue / Fix	Input	Expected Output	Status, Actual Output and Notes
1	Returned the “Search Test” button within the “get_search()” function. See below:	N/A	Button should appear.	FAIL
<pre>return[[GUI.Push(), GUI.Text('Email Address'), GUI.Input(do_not_clear=False, key='-EMAIL-')], [GUI.Button(button_text: 'Search Test', bind_return_key=True)]]</pre>				
2	Done a workaround where the button will be in the main window but outside of the tab and when you select the “Search for Information” tab, where the button should have appeared, the “visible” attribute of the button is set to True.	N/a	Button should appear on the “Search for Information” tab only.	PASS

T7 - The “Search for Flight” and “Search for Airport” do not disappear as expected when the tab is switched.

Test ID	Suspected Issue / Issue / Fix	Input	Expected Output	Status, Actual Output and Notes
1	Altered the selection statement to be opposite conditions with opposite outputs. May need a while loop if fails. See below:	“Flight” drop down menu option selected under “Search for Information” tab. Then switch tabs.	After switching tabs, the “Search for Flights” button should disappear.	FAIL - Button does not appear.
<pre> 331 if active_tab == "Search for Information": 332 if main_values["-MENU_OPT-"] != "Airports::-APT-": 333 window3["-SEARCH_APT-"].update(visible=False) 334 if event == '-SEARCH_APT-': </pre>				
2	Will implement a while loop over an if statement. See below:	“”	“”	FAIL - “”
<pre> 341 if active_tab == "Search for Information": 342 while main_values["-MENU_OPT-"] == "Airports::-APT-": 343 window3["-SEARCH_APT-"].update(visible=True) 344 if event == '-SEARCH_APT-': 345 get_searchAPT() </pre>				
3	Will implement a nested if statement. See below:	“”	“”	PART PASS - The button now disappears if another menu option is selected, but not if you switch tabs.
<pre> 331 if active_tab == "Search for Information": 332 if main_values["-MENU_OPT-"] == "Airports::-APT-": 333 window3["-SEARCH_APT-"].update(visible=True) 334 if main_values["-MENU_OPT-"] != "Airports::-APT-": 335 window3["-SEARCH_APT-"].update(visible=False) 336 if event == '-SEARCH_APT-': 337 get_searchAPT() </pre>				
4	Added an else condition to the, active_tab == “Search for Information”, condition so if the “active_tab” is not set to “Search for Information”, I.e. the user is on another tab, the respective buttons should disappear. See below:	“”	“”	PASS

```
331     if active_tab == "Search for Information":
332         if main_values["-MENU_OPT-"] == "Airports::-APT-":
333             window3["-SEARCH_APT-"].update(visible=True)
334             if event == '-SEARCH_APT-':
335                 get_searchAPT()
336         else:
337             window3["-SEARCH_APT-"].update(visible=False)
338
339         if main_values["-MENU_OPT-"] == "Flights::-FLT-":
340             window3['-SEARCH_FLT-'].update(visible=True)
341             if event == '-SEARCH_FLT-':
342                 get_searchFLT()
343         else:
344             window3['-SEARCH_FLT-'].update(visible=False)
345
346     else:
347         window3["-SEARCH_APT-"].update(visible=False)
348         window3['-SEARCH_FLT-'].update(visible=False)
```

T8 - “DateFrom” and “DateTo” inputs for the “get_searchFLT()” API query, is not recognised by the “datetime” library.

Test ID	Suspected Issue / Issue / Fix	Input	Expected Output	Status, Actual Output and Notes
1	I will look into whether I can use some kind of function within the “datetime” library, so that I can pull yesterday's and a future date. Which will then meet the correct format. See below:	Flight number of “EK184”.	The API should return a correct result.	FAIL - May need to add “date” and “timedelta” to the import scope.
286 287 288	<pre>flights = [SearchEntFlt], flight_datetime_from = datetime(date.today() - timedelta(days=1)), flight_datetime_to = datetime(date.today() + timedelta(days=1))</pre>			
2	Added “date” and “timedelta” to the scope of the imports under “datetime.” See below:	“”	“”	FAIL - Will stick to having to change the day manually (for now).
15	<pre>from datetime import datetime, date, timedelta</pre>			
3	See below:	“”	“”	PASS - I have managed to implement an effective time format. The time scale needed to be in seconds to meet the API format.
<pre>TimeFrom = datetime.now() TimeTo = TimeFrom + timedelta(days=14) flight_datetime_from = TimeFrom.isoformat(timespec='seconds'), flight_datetime_to = TimeTo.isoformat(timespec='seconds')</pre>				

T9 - API "get_searchAPT()" query does not recognise the entered airport code, which should be valid.

Test	Suspected Issue / Issue / Fix	Input	Expected Output	Status / Output / Notes
1	The way that the initial error includes square brackets around the airport code, suggests I may be trying to input a list into the query, rather than a single string. Will remove square brackets on the query call. See below, the initial error:	Airport ICAO of "EGNX".	The API should return (light) airport information, without error.	PASS Issue occurred when I copied code from my previously coded flight query function, to more easily change. However, flights can be queried as a list of information, airports cannot.

400 Bad Request: The provided airport code '['EGNX']' is not a valid IATA or ICAO code.

T10 - Difficulty outputting (graphically) API query returns, without having to create a separate element for each attribute.

Test	Suspected Issue / Issue / Fix	Input	Expected Output	Status / Output / Notes
1	After realising the attributes aren't indexed numerically, I have used concatenation of strings and the attributes, to give additional information on what is to be outputted. I have used the linebreak feature (\n) to separate attributes. See below:	Airport ICAO of "EGNX".	There should be a graphical output of each variable on a separate line with the specific additional information.	FAIL "TypeError: decoding str is not supported"

```

404 if event == '-SEARCHBT_APT-':
405     #Triggers the "get_searchAPT" API query function with the user input assigned to
406     #"SearchEntApt", assigned to a dictionary identifier, being passed as a parameter.
407     airport = get_searchAPT(SearchEntApt)
408     #Updates the "-AIRPORT-" GUI element with the data stored in "airport".
409     window3["-AIRPORT-"].update(str("Airport Name: " + airport.name, "\nAirport IATA" + airport.iata), visible=True)

```

2	I have removed the linebreak feature (\n), as that may be the cause of the issue. I will output attributes on one line. See below:	Airport ICAO of "EGNX".	""	FAIL The window closes itself.
---	---	-------------------------	----	--

```
402 if event == '-SEARCHBT_APT-':
403     #Triggers the "get_searchAPT" API query function with the user input assigned to
404     #"SearchEntApt", assigned to a dictionary identifier, being passed as a parameter.
405     airport = get_searchAPT(SearchEntApt)
406     #Updates the "-AIRPORT-" GUI element with the data stored in "airport".
407     window3["-AIRPORT-"].update(str("Airport Name: " + airport.name, "Airport IATA" + airport.iata), visible=True)
```

3	I have removed the comma from the statement. See below:	Airport ICAO of "EGNX".	""	<u>PASS</u>
---	--	-------------------------	----	-------------

```
402 if event == '-SEARCHBT_APT-':
403     #Triggers the "get_searchAPT" API query function with the user input assigned to
404     #"SearchEntApt", assigned to a dictionary identifier, being passed as a parameter.
405     airport = get_searchAPT(SearchEntApt)
406     #Updates the "-AIRPORT-" GUI element with the data stored in "airport".
407     window3["-AIRPORT-"].update(str("Airport Name: " + airport.name + "\nAirport IATA: " + airport.iata), visible=True)
```

T11 - Difficulty outputting (graphically) API flight query returns and finding the correct attribute name.

Test	Suspected Issue / Fix	Input	Expected Output	Output / Notes and Status
1	When attempting to return a specific object attribute, see below, an error is returned indicating I am using the incorrect	Flight number of "BA196."	There should be a graphical output of the "orig_iata" attribute.	<u>FAIL</u> "AttributeError: 'FlightSummaryFullResponse' object has no attribute 'orig_iata.'"
424	<pre>window3["-FLIGHTSUMMARY-"].update(str(flight_summary.orig_iata), visible=True)</pre>			
425	<pre>window3["-FLIGHTPOSITIONS-"].update(str(flight_positions), visible=True)</pre>			
2	The "flight_summary" and "flight_position" are returned as an object, within a list. Therefore, a different approach is required. See below:	Flight number of "BA196."	There should be a graphical output of the "orig_iata" attribute.	<u>PASS</u>
423	<pre>window3["-FLIGHTSUMMARY-"].update(str(flight_summary.data[0].orig_iata), visible=True)</pre>			
424	<pre>window3["-FLIGHTPOSITIONS-"].update(str(flight_positions), visible=True)</pre>			
3	The "eta" attribute doesn't appear, with the following error: "AttributeError: 'FlightSummaryFull' object has no attribute 'eta'"	Flight number of "BA196."	There should be a graphical output of the "orig_iata" attribute.	<u>FAIL</u>
4	The "eta" attribute belongs to the FlightPositions query, not the FlightSummary query.	""	""	<u>PASS</u>

T12 - Additional stats (attributes) don't appear when the "Show Stats For Nerds" button is pressed, after searching for a flight.

Test	Suspected Issue / Fix	Input	Expected Output	Output / Notes and Status
1	Will move the "-NERDBT_FLT-" event trigger selection statement, to make it nested within the "-SEARCHBT_FLT-" event trigger selection statement. See below:	Flight number of "BA196," then press the "Show Stats For Nerds" button.	The "Nerdy Stats" list of attributes should appear.	FAIL List of additional attributes doesn't appear.
<pre> 421 if event == '-SEARCHBT_FLT-': 422 #Triggers the "get_searchFLT" API query function with the user input 423 #assigned to "SearchEntFlt", being passed as a parameter. 424 flight_summary, flight_positions = get_searchFLT(SearchEntFlt) 425 #Updates the respective GUI elements with the data stored in... 426 #TODO Do the same as with airport function, one output for each key attribute. 427 #TODO See FR24 favourite for attributes. 428 window3["-FLIGHTSUMMARY-"].update(str("Departure Airport: " + flight_summary.data[0].orig_iata + "\nDeparture Time: " + flight_ 429 window3["-FLIGHTPOSITIONS-"].update(str("\nEstimated Time of Arrival: " + flight_positions.data[0].eta), visible=True) 430 431 if event == '-NERDBT_FLT-': 432 #BROKEN Nothing appears when button pressed. 433 window3["-NERD-"].update(str("\n--Nerdy Stats--" + "\nTracking and Data Source: " + flight_positions.data[1].source), visib </pre>				
2	I was attempting to show additional information to an element that the information wasn't returned to in the first place. Therefore, I am going to use a checkbox element which must be selected beforehand.	Check the "Show Stats For Nerds" checkbox and search for the flight number "BA186."	The standard query return, as well as the additional "stats for nerds" should appear.	FAIL ""
<pre> 432 if event == '-NERD_FLT-': 433 window3["-FLIGHTPOSITIONS-"].update(str("\n--Nerdy Stats--" + "\nTracking and Data Source: " + flight_positions.data[0].source), visible=True) </pre>				
2	Grammatical error, plus cuts off a substring, so will re-instate the "-NERD-" text element. See below:	""	""	FAIL ""

```
433 if '-NERD_FLT-':
434     window3["-NERD-"].update(str("\n--Nerdy Stats--" + "\nTracking and Data Source: " +
```

2	See below:	""	""	<u>PASS</u>
---	------------	----	----	-------------

```
433 if ["-NERD_FLT-"]:
434     window3["-NERD-"].update(str("\n--Nerdy Stats--" + "\nTracking and Data Source: " +
```

T13 - "email_address_per" and "email_address_co" variables (used in the main window, to validate flight alert creation) are inaccessible, as they are outside the scope of which they were set / defined.

Test	Suspected Issue / Fix	Input	Expected Output	Output / Notes and Status
1	Will set the two variables as global variables, in previous functions (where needed). Hopefully making them accessible. See below:	Flight number of "BA196," then check the "Create An Alert For This Flight" checkbox.	The alert attributes should be committed to the "alerts" table within the "flightdeckdb" database.	<u>FAIL</u>
<pre> 254 def window2_run(): 2 usages 255 #Declares the value of sign_values["-EMAIL-"] as a 256 #global variable for later use (in "window3_run"). 257 global GlobalEmail 465 #If the "Create An Alert For This Flight" checkbox has been selected... 466 if main_values["-ALERTCHK_FLT-"]: 467 ExistAlert = "SELECT `EmailAddress`, `FlightNumber` FROM alerts WHERE `EmailAddress` = %s AND `FlightNumber` = %s" 468 #Executes the above SQL statement "ExistAlert" and passes the values of "email_address_per" / 469 #"email_address_co" and the flight number of the queried flight into the respective parameters above. 470 Cursor.execute(ExistAlert, GlobalEmail, flight_summary.data[0].flight) 471 #Fetches one (the first) value. 472 ExistAlertResult = Cursor.fetchone() 473 474 if not ExistAlertResult: 475 AlertSql = "INSERT INTO alerts (`EmailAddress`, `FlightNumber`, `DepartureTime`, `NotificationTime`) VALUES (%s, 476 AlertValues = (GlobalEmail, flight_summary.data[0].flight, flight_summary.data[0].datetime_takeoff, flight_posit 477 Cursor.execute(AlertSql, AlertValues) 478 #Commits the values to the database under one transaction. 479 db.commit() </pre>				
2	In line 477, I had attempted to pass the attributes as all separate SQL statements. Missing parenthesis.	""	""	<u>FAIL</u>
3	In line 475, a semi-colon is missing.	""	""	<u>PASS</u> but see T14 below.

T14 - Transactions into the “alerts” table within the “flightdeckdb” database (for flight alerts), are not committing the email address attribute.

Test	Suspected Issue / Fix	Input	Expected Output	Output / Notes and Status
1	Will recheck the declaration of the global variable “GlobalEmail”, likely a logic error in initialisation.	Flight number of “BA196,” then check the “Create An Alert For This Flight” checkbox.	The email address attribute should be committed to the “alerts” table within the “flightdeckdb” database.	PASS See below:

```

284 #If no result is found, the credentials are deemed incorrect as they must be correct initially at the
285 #create account stage. The credentials invalid error message is displayed by settings its "visible"
286 #attribute to "True".
287 if not credentials_result:
288     window2['-CREDS_ERR-'].update(visible=True)
289 #If a result is found, the credentials are deemed correct. The sign-in successful message ("-SIGN_MSG-") is
290 #displayed by setting its "visible" attribute to "True". Window3 is executed, window2 is closed.
291 else:
292     window2['-SIGN_MSG-'].update(visible=True)
293     #3000ms window2 timeout to allow the user to read the message before closing.
294     event, sign_values = window2.read(timeout=3000)
295     GlobalEmail = email_address_check

```

Line 295 was initially “GlobalEmail = sign_values[“-EMAIL-”]” and a previous assignment removed above.

T15 - "list index out of range" for "ExistAlertResult3[4]" / "ExistAlertResult3[3]" post-SQL-query.

Test	Suspected Issue / Fix	Input	Expected Output	Output / Notes and Status
1	In the Figure 3.8.2(a) SQL query, I am using the ".fetchall()" function. Unlike the ".fetchone()" function, the ".fetchall()" function returns a list of tuples (each representing one record, for multiple), rather than an individual tuple. See below for fix:	After having created a flight alert, switch to the "Notifications" tab.	The "ExistAlert3" SQL query should be executed and logical operations carried out to later print "TDeparture" or "Test Complete".	<u>PASS</u>
<pre> 586 #If alert creation time < flight departure time 587 if ExistAlertResult3[0][4] < ExistAlertResult3[0][3]: 588 flight_summary, flight_positions = get_searchFLT(SearchEntFlt) 589 590 #Flight departure time - current time = time until departure (via "timestamp" 591 #attribute, under the FlightRadar24 API "flight_positions" endpoint. 592 TDeparture = ExistAlertResult3[0][3] - flight_positions.data[10] 593 print(TDeparture) </pre>				

Evaluation

Post-Development Test Plan (Evidence and Annotations)

Function and robustness, validation, SQL queries, API queries, error handling, etc. Evidence and failure notes located in [Test Plan Evidence Appendix](#). Follow in order.

Test ID	Applies to which Section	Test Data / Action	Requirement	Expected Result	Pass / Fail and applies to which success criteria
1a	Initial Program Execution	Execution of the program executable (via the IDE).	The account creation window should open.	The initial window that opens should be the account creation window.	Pass Figure 4.1(a) N/A
1b		Check Terminal.	Text confirmation (via IDE terminal) that the database connection, via the mysqlconnector library, has been successful.	"DB - Connection Successful" should appear in the terminal.	Pass Figure 4.1(b) N/A
2b	Account Creation	Press the input button or "enter" key, with no values entered into any input opportunities.	An error message should appear, asking you to enter valid inputs.	"Password must be at least 8 characters long." "Passwords must match."	Pass Figure 4.1(c) 1, 1a(ii)
2c		Press the input button or "enter" key, with a password less than 8 characters entered (identically into the password input and re-input)	An error message should appear, asking you to enter valid inputs (at least 8 characters long). Inputs should be also cleared.	"Password must be at least 8 characters long." All inputs cleared.	Pass Figure 4.1(d) 1, 1a(ii)
2d		Press the input button or "enter" key with a password greater than 8 characters (but unidentical password input and re-input).	An error message should appear, asking you to enter valid inputs (password input and re-input identical). Inputs should be cleared.	"Passwords must match." All inputs cleared.	Pass Figure 4.1(e) 1, 1a(ii)
2e		Press the input button or "enter" key with fully valid inputs (password greater than 8 characters and identical password input and re-input).	A message should appear, indicating that validation has been "passed" and your account has been created. The sign-in window should open.	"Account creation successful" or words to that effect. All inputs cleared. The sign-in window	Pass Figure 4.1(f) 1, 1a(i)

				should open.	
3a	Account Sign-In	Press the input button or "enter" key, with no values entered into any input opportunities.	An error message should appear, asking you to enter valid inputs.	"Account does not exist or incorrect credentials" or words to that effect.	Pass Figure 4.1(g) 1, 1a(iii)
3b		Press the input button or "enter" key, with an incorrect password entered.	An error message should appear, asking you to enter valid inputs.	"Account does not exist or incorrect credentials" <i>OWTTE</i> .	Pass Figure 4.1(h) 1, 1a(iii)
3c		Press the input button or "enter" key, with fully valid inputs.	A message should appear, indicating that validation has been "passed" and you are logged in. The main window should open.	"Sign-In successful" <i>OWTTE</i> . All inputs cleared. The main window should open.	Pass Figure 4.1(i) 1, 1a(iii)
4a	"Search For Information" Tab	Enter nothing into the data entry box and press the "Search" button or "enter" key.	A validation statement should be returned, prompting re-entry.	"Flight number cannot be empty" <i>OWTTE</i> .	Fail Figure 4.1(j) 2, 2a(ii) 4, 4a(i)
4ai		Enter a flight number into the data entry box and press the "Search" button or "enter" key.	An API statement should successfully be returned.	Statements of "Flight_Positions" and of "Flight_Summary" returned graphically.	Fail Figure 4.1(k) 2, 2a(ii) 4, 4a(i)
4b		Enter a flight number into the data entry box, check the "Show Stats for Nerds" checkbox, and press the "Search" button or "enter" key.	An API statement (with additional attributes) should successfully be returned.	Statements of "Flight_Positions" and of "Flight_Summary" returned graphically, with additional attributes.	Pass Figure 4.1(l) 2, 2a(ii) 4, 4a(i)
4c		Enter an airport ICAO/IATA code into the data entry box and press the input button or "enter" key.	An API statement should be successfully returned.	One statement of "AirportsFull" returned graphically.	Pass Figure 4.1(m) 2, 2a(ii) 4, 4a(i)
4d		Enter a flight number into the data entry box, check the "Create An Alert For This Flight" checkbox, and press the "Search" button or "enter" key.	The alert record ("alerts" table) should be created and a message should appear, indicating an alert has been created.	An alert with an "AlertID", "EmailAddress", "FlightNumber", "DepartureTime", "CreationTime" and "LastAlerted" attributes should be created.	Partial Pass Figure 4.1(n) 1, 1a(ii) 4, 4a(i)

				The text entry box should automatically clear (after searching) and a message should appear, indicating an alert has been created, detailing the flight number.	
5a	"Manage Alerts" Tab	Select the "Manage Alerts" tab.	An SQL query should be performed and your created alert should be displayed graphically.	An SQL query should be performed and your created alert should be displayed graphically, detailing the flight number, creation time, etc.	Mostly Pass Figure 4.1(o) 2, 2b(i) 2, 2b(ii) 2, 2b(ii.2)
5ai		Enter nothing into the data entry box and press the "Delete" button or "enter" key.	A validation statement should be returned, prompting re-entry.	"Flight number cannot be empty" <i>OWTTE</i> .	Partial Pass Figure 4.1(p) N/A
5aai		Enter a flight number (which an alert has not been created for) into the data entry box and press the "Search" button or "enter" key.	A validation statement should be returned, prompting re-entry.	"You can only delete alerts you have created" <i>OWTTE</i> .	Partial Pass Figure 4.1(q) N/A
5b		Enter the alert flight number (returned previously - see above) into the "delete alert" data entry box and press the "Delete Alert" button or "enter" key.	The alert record ("alerts" table) should be deleted and an indication should appear that the alert has been deleted.	The text entry box should automatically clear (after pressing the "Delete Alert" button or equivalent) and the alert (displayed previously - see above), disappear.	Pass Figure 4.1(r) 2, 2b(i) 2, 2b(ii.2) 3, 3a(i)
6a	"Notifications" Tab	Select the "Notifications" tab.	An SQL query should be performed and your created alerts should be displayed graphically.	An SQL query should be performed and your created alerts should be displayed graphically, with different outputs dependent on whether the flight is prior to departure, how prior to departure, in flight, etc.	Fail Figure 4.1(s) 2, 2b(i) 2, 2b(ii) 3, 3a(i)

Usability Test Plan (Evidence and Annotations)

GUI navigation, layout, feedback messages, etc. Evidence and failure notes located in the [Test Plan Evidence Appendix](#). **Follow in order.**

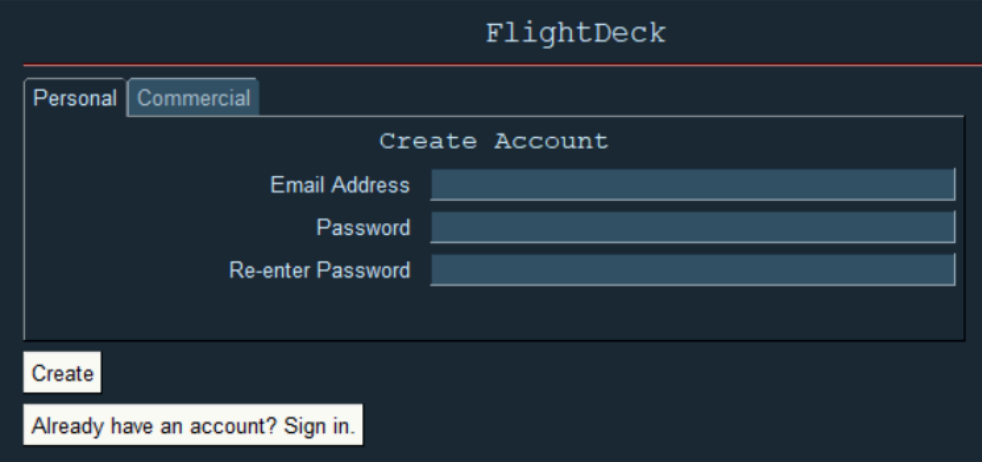
Test ID	Applies to which Section	Test Data / Action	Requirement	Expected Result	Pass / Fail
1a	Initial Program Execution	Execution of the program executable (via the IDE).	The account creation window should open.	The initial window that opens should be the account creation window.	Pass Figure 4.2(a)
1b		Attempt to quit the program using the GUI quit "cross" box (top navigation bar).	The user should be able to quit the program without having to return to the IDE.	<i>Same as requirement</i> , with a confirmation action appearing.	Pass Figure 4.2(b)
1c		Attempt to minimise the program using the GUI minimise box (top navigation bar).	The user should be able to minimise the program window.	<i>Same as requirement.</i>	Pass Figure 4.2(c)
2a	Account Creation	Attempt to select and enter values into any (input) text box elements.	For text data entry elements to be accessible and work as intended.	The cursor indicator should appear within any text box elements, allowing data entry.	Pass Figure 4.2(d)
2b		Attempt to press any button elements.	For button elements to be accessible and work as intended.	All button elements should act as an event trigger, so further development into the program should occur.	Pass Figure 4.2(e)
3a	Account Sign-In	Attempt to select and enter values into any (input) text box elements.	For text data entry elements to be accessible and work as intended.	The cursor indicator should appear within any text box elements, allowing data entry.	Pass Figure 4.2(f)
3b		Attempt to press any button elements.	For button elements to be accessible and work as intended.	All button elements should act as an event trigger, so further development into the program should occur.	Pass Figure 4.2(g)

4a	"Search For Information" Tab	Select the "Search For Information" tab.	The "Search For Information" tab selector should become highlighted.	<i>Same as requirement.</i>	Pass Figure 4.2(h)
4b		Select the "Search For Information" tab.	No text input elements should appear until you select a "Search for Flights / Airports" drop-down option.	<i>Same as requirement.</i>	Fail Figure 4.2(h)
4c		Right click the "Live Flights / Airports" drop-down menu.	Drop-down options appear for different entities to search for, which correspond to different API query functions.	The "Flights" and "Airports" option appears.	Pass Figure 4.2(i)
4d		Select the "Flights" drop-down menu option.	The flight's data entry box and search confirmation button appears. The airport's data entry box and search confirmation button disappears	The data entry box appears alongside "Enter a Flight Number:" and the "Search for Flight" button appears. The airport's data entry box and search confirmation button disappears.	Pass Figure 4.2(j)
4e		Select the "Airports" drop-down menu option.	The airport's data entry box and search confirmation button appears. The flight's data entry box and search confirmation button disappears.	The data entry box appears alongside "Enter an airport ICAO/IATA code:" and the "Search for Airport" button appears. The flight's data entry box and search confirmation button disappears.	Pass Figure 4.2(k)
5a	"Manage Alerts" Tab	Select the "Manage Alerts" tab.	The "Manage Alerts" tab selector should become highlighted.	<i>Same as requirement.</i>	Pass Figure 4.1(o)
6a	"Notifications" Tab	Select the "Notifications" tab.	The "Notifications" tab selector should become highlighted.	<i>Same as requirement.</i>	Pass Figure 4.1(s)
7a	General	Sign-in as a commercial user (may require creating an account).	The company logo should be displayed on the main interface, under any tabs that are interacted with.	<i>Same as requirement.</i>	Pass Figure 4.1 (k)

Test Plan Evidence Appendix

Figures with 4.1 prefix relate to [Post-Development Test Plan \(Evidence and Annotations\)](#). Figures with 4.2 prefix relate to [Usability Test Plan \(Evidence and Annotations\)](#).

For Post-Development Test Plan



FlightDeck

Personal Commercial

Create Account

Email Address

Password

Re-enter Password

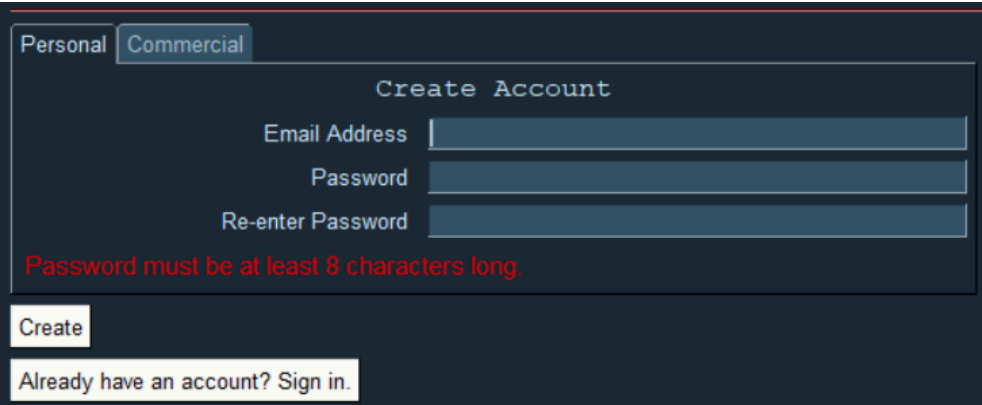
Create

Already have an account? Sign in.

Figure 4.1(a) ▲

DB - Connection Successful

Figure 4.1(b) ▲



Personal Commercial

Create Account

Email Address

Password

Re-enter Password

Password must be at least 8 characters long.

Create

Already have an account? Sign in.

Figure 4.1(c) ▲

Personal Commercial

Create Account

Email Address

Password

Re-enter Password

Password must be at least 8 characters long.

Create

Already have an account? Sign in.

Figure 4.1(d) ▲

Personal Commercial

Create Account

Email Address

Password

Re-enter Password

Create

Already have an account? Sign in.

Password: *jakejake12*

Re-enter Password: *jakejake13*

Then

Personal Commercial

Create Account

Email Address

Password

Re-enter Password

Passwords must match.

Create

Already have an account? Sign in.

Figure 4.1(e) ▲

Personal Commercial

Create Account

Email Address TestEmail@hotmail.com

Password *****

Re-enter Password *****

Create

Already have an account? Sign in.

Password: jakejake12

Re-enter Password: jakejake12

Then

Personal Commercial

Create Account

Email Address

Password

Re-enter Password

Account Creation Successful. Standby..

Create

Already have an account? Sign in.

Then

Flight Information and Management System

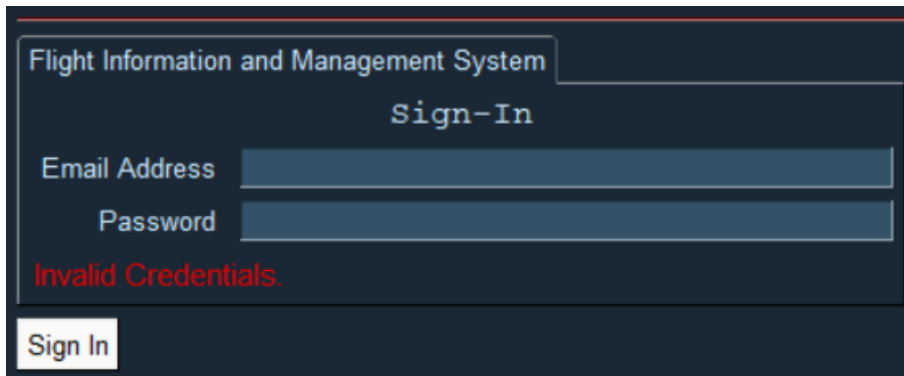
Sign-In

Email Address

Password

Sign In

Figure 4.1(f) ▲



Flight Information and Management System

Sign-In

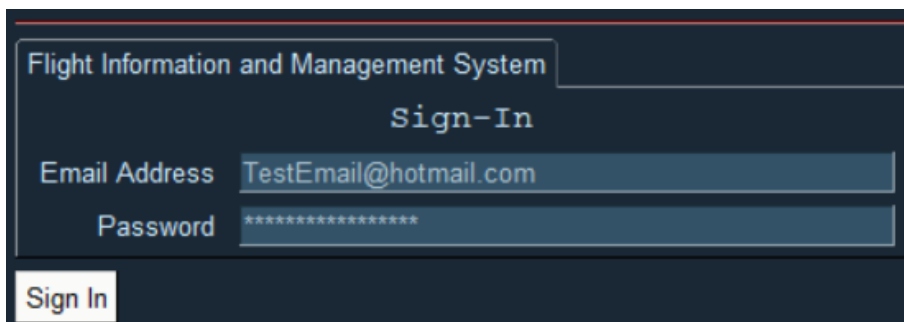
Email Address

Password

Invalid Credentials.

Sign In

Figure 4.1(g) ▲



Flight Information and Management System

Sign-In

Email Address

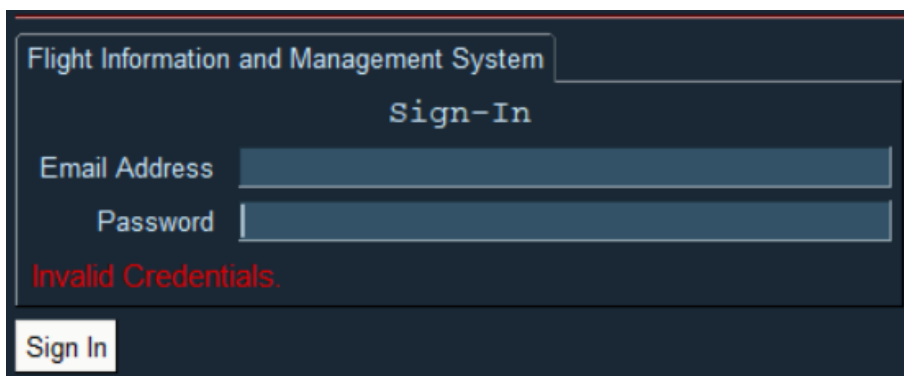
Password

Sign In

Password: *incorrectpassword*

Correct Password: *jakejake12*

Then



Flight Information and Management System

Sign-In

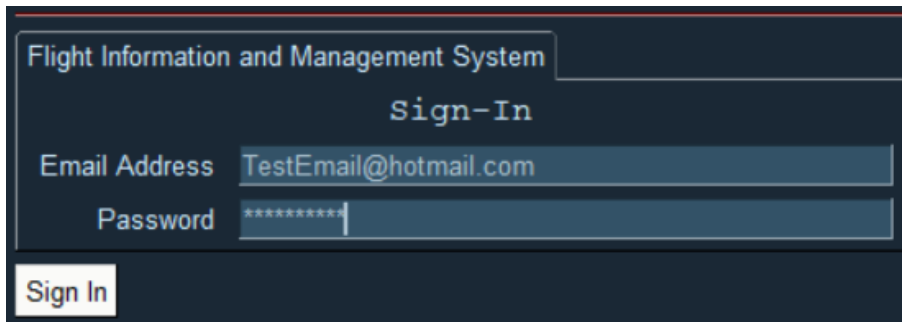
Email Address

Password

Invalid Credentials.

Sign In

Figure 4.1(h) ▲



Flight Information and Management System

Sign-In

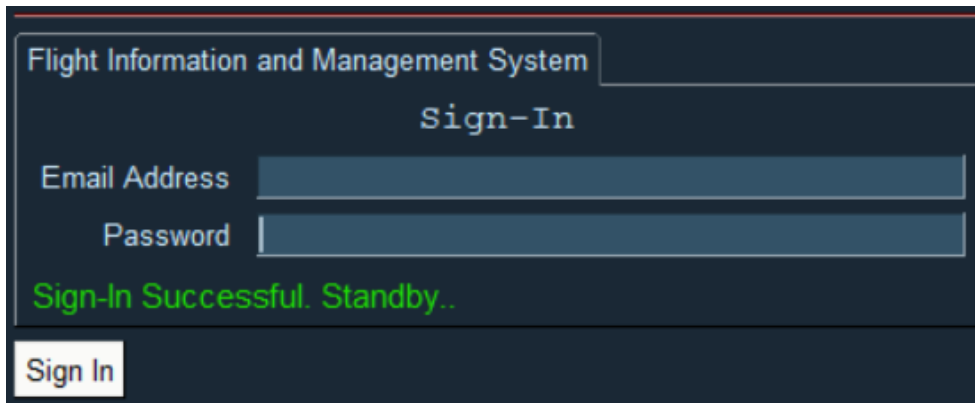
Email Address TestEmail@hotmail.com

Password *****

Sign In

Password: jakejake12

Then



Flight Information and Management System

Sign-In

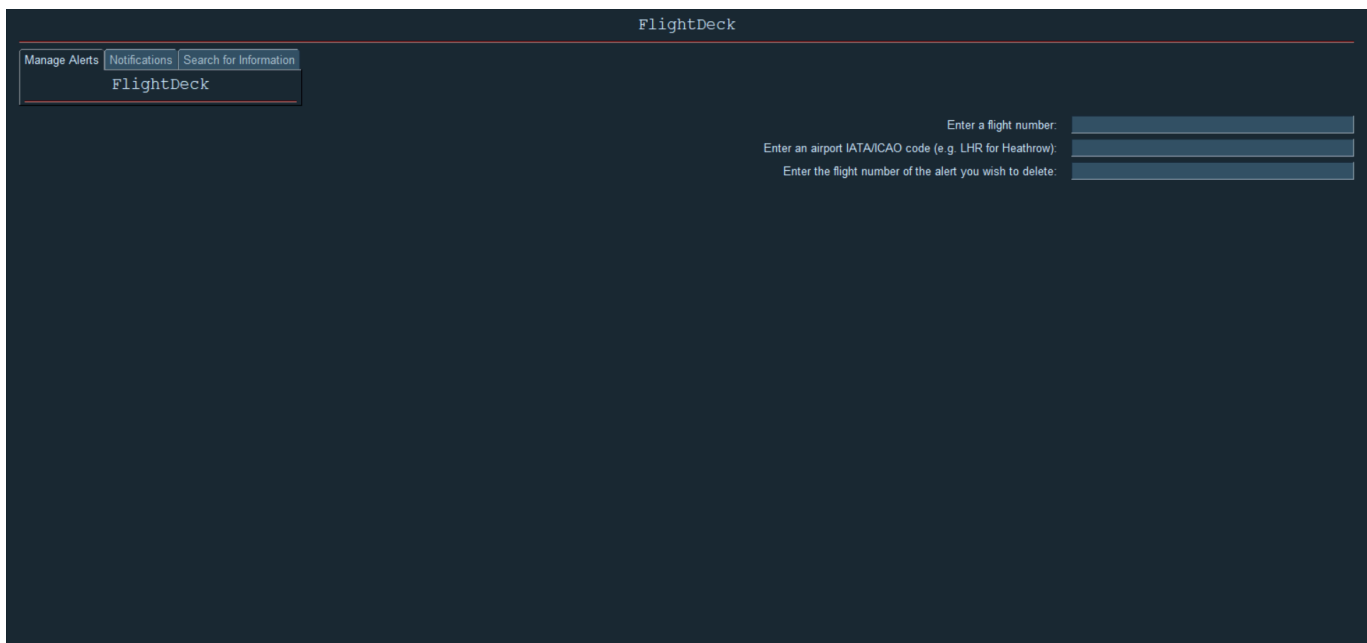
Email Address

Password

Sign-In Successful. Standby..

Sign In

Then



FlightDeck

Manage Alerts Notifications Search for Information

FlightDeck

Enter a flight number:

Enter an airport IATA/ICAO code (e.g. LHR for Heathrow):

Enter the flight number of the alert you wish to delete:

Figure 4.1(i) ▲

```
pydantic_core._pydantic_core.ValidationError: 1 validation error for _LivePositionsParams
flights.0
  String should match pattern '^(?i)(?:[A-Z]{1,3}[0-9]{1,4}[A-Z]?|[0-9][A-Z][0-9]{1,4}|[A-Z]
  For further information visit https://errors.pydantic.dev/2.10/v/string\_pattern\_mismatch
```

Figure 4.1(j) ▲

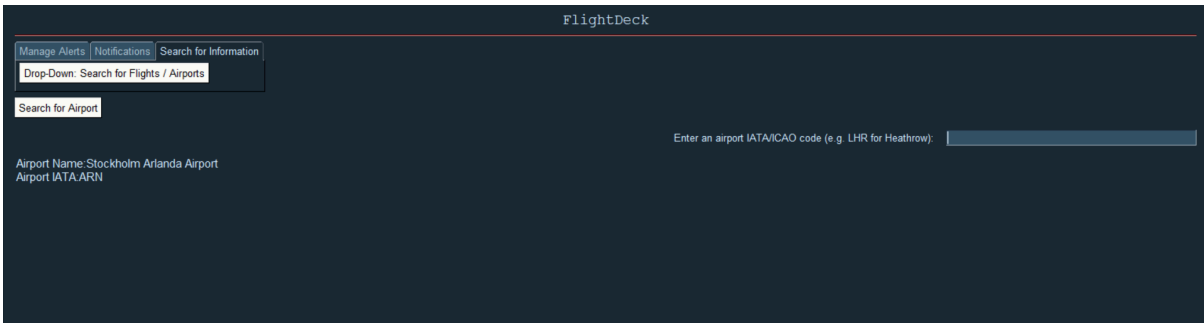


Then



Figure 4.1(k) ▲

This feature has worked in previous program versions (see [3.6 “Search for Information” Tab](#)).



Manage AlertsNotificationsSearch for Information

Drop-Down: Search for Flights / Airports

Search for Flight

☒ Show Stats For Nerds

☐ Create An Alert For This Flight

Enter a flight number:

-- Nerdy Stats --

Tracking and Data Source: ADSB

Squawk Code: 0734

Aircraft Registration: EI-SIN

Figure 4.1(l) ▲

FlightDeck

Manage AlertsNotificationsSearch for Information

Drop-Down: Search for Flights / Airports

Search for Airport

Enter an airport IATA/ICAO code (e.g. LHR for Heathrow):

Airport Name: Stockholm Arlanda Airport

Airport IATA:ARN

Figure 4.1(m) ▲

FlightDeck

Manage AlertsNotificationsSearch for Information

Drop-Down: Search for Flights / Airports

Search for Flight

☐ Show Stats For Nerds

☒ Create An Alert For This Flight

Enter a flight number:

AlertID	EmailAddress	FlightNumber	DepartureTime	CreationTime	LastAlerted
11	TestEmail@hotmail.com	SK1415	2023-01-27 05:15:22.000000	2026-04-11 16:30:36.000000	NULL

Figure 4.1(n) ▲

Partial pass as the alert was successfully created and committed to the “alerts” table within the “flightdeckdb” database, however a confirmation message was not given.

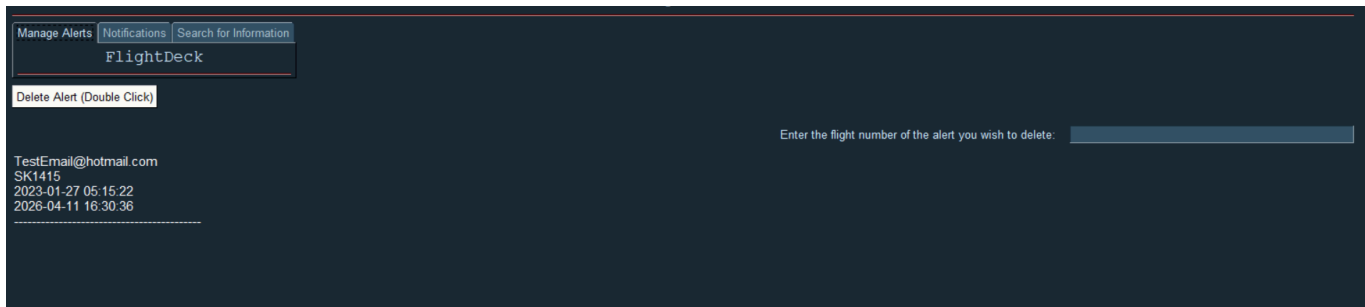


Figure 4.1(o) ▲

Mostly pass as even though all data from the SQL query was returned and displayed accordingly, it would be nice to incorporate labels on the date-times. Although, you could likely work it out yourself if you created the alert, as I can.

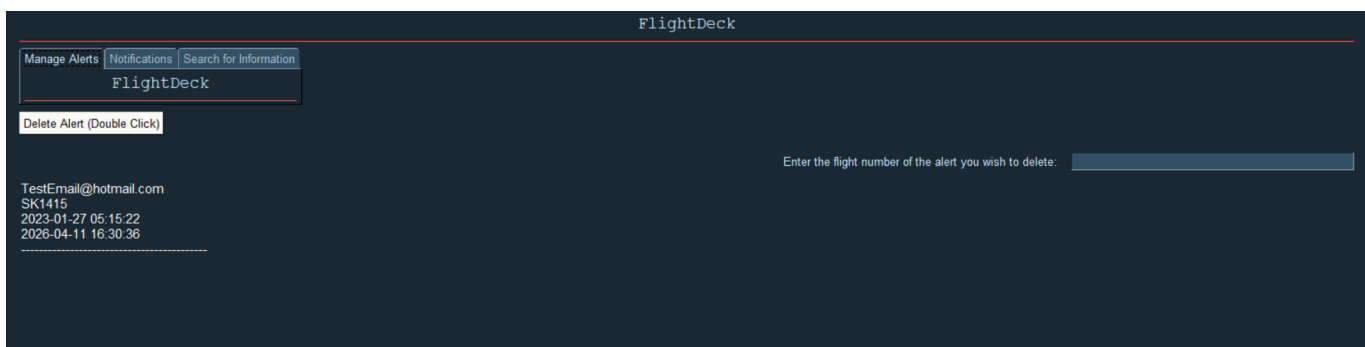
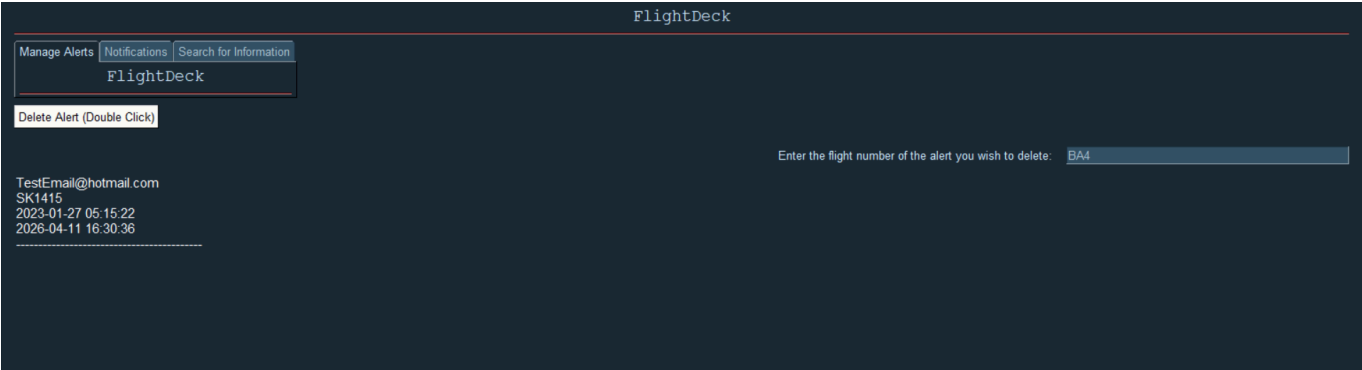


Figure 4.1(p) ▲

*Partial pass as the program does not completely break and close (unlike **Figure 4.1(j) - Test 4a in the Post-Development Test Plan**), but no validation message is displayed to inform the user that the data input box cannot be empty when attempting to delete a flight.*

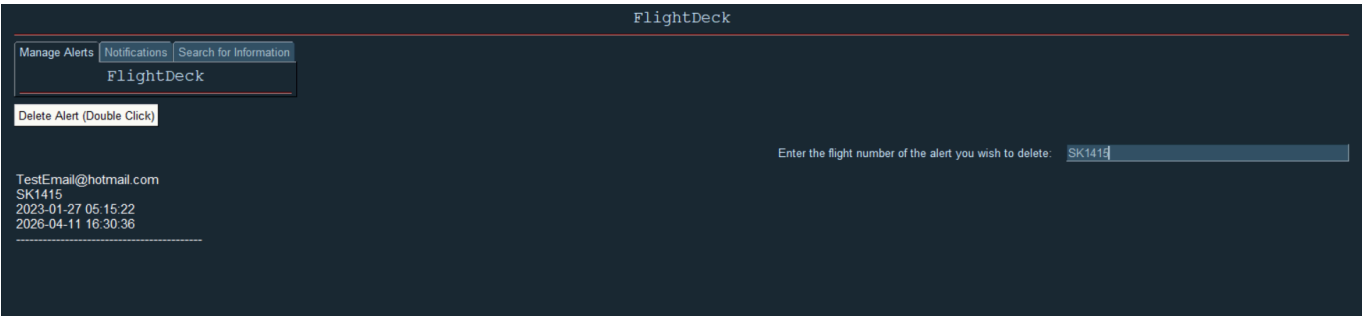


Then

AlertID	EmailAddress	FlightNumber	DepartureTime	CreationTime	LastAlerted
11	TestEmail@hotmail.com	SK1415	2023-01-27 05:15:22.000000	2026-04-11 16:30:36.000000	NULL

Figure 4.1(q) ▲

Partial pass as the program does not completely break and close and the incorrectly identified alert isn't deleted from the "alerts" table, but no validation message is displayed to inform the user they cannot delete an alert which they haven't created.



Then

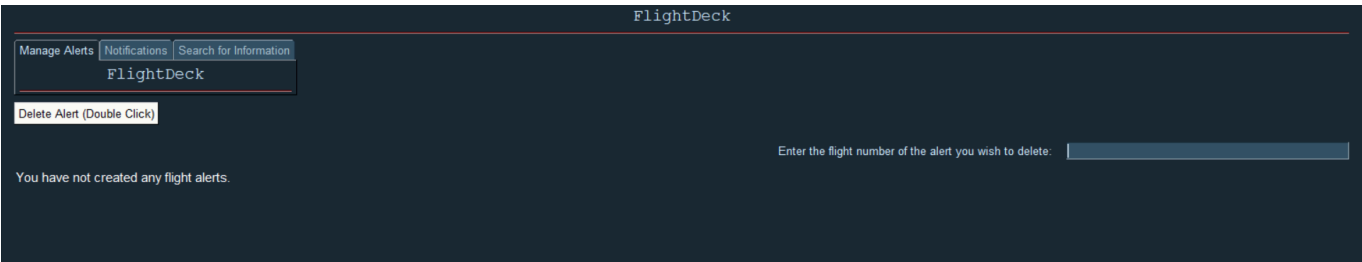


Figure 4.1(r) ▲

AlertID	EmailAddress	FlightNumber	DepartureTime	CreationTime	LastAlerted
12	TestEmail@hotmail.com	SK1415	2023-01-27 05:15:22.000000	2026-04-11 16:45:35.000000	NULL

Then

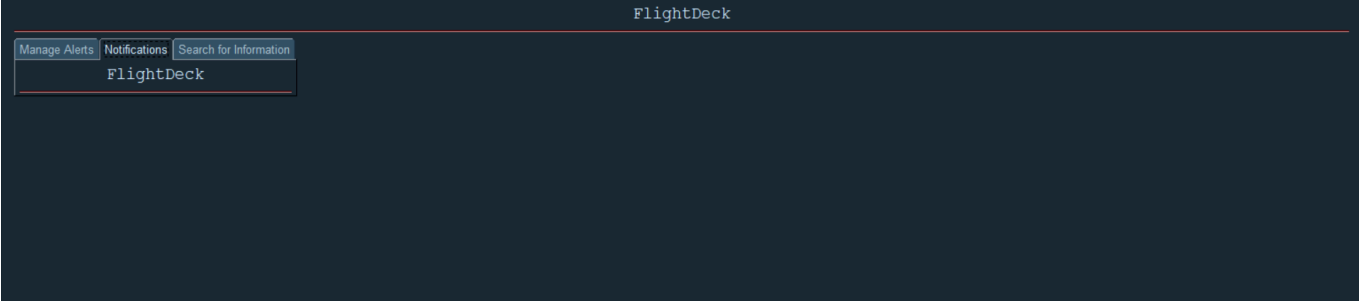
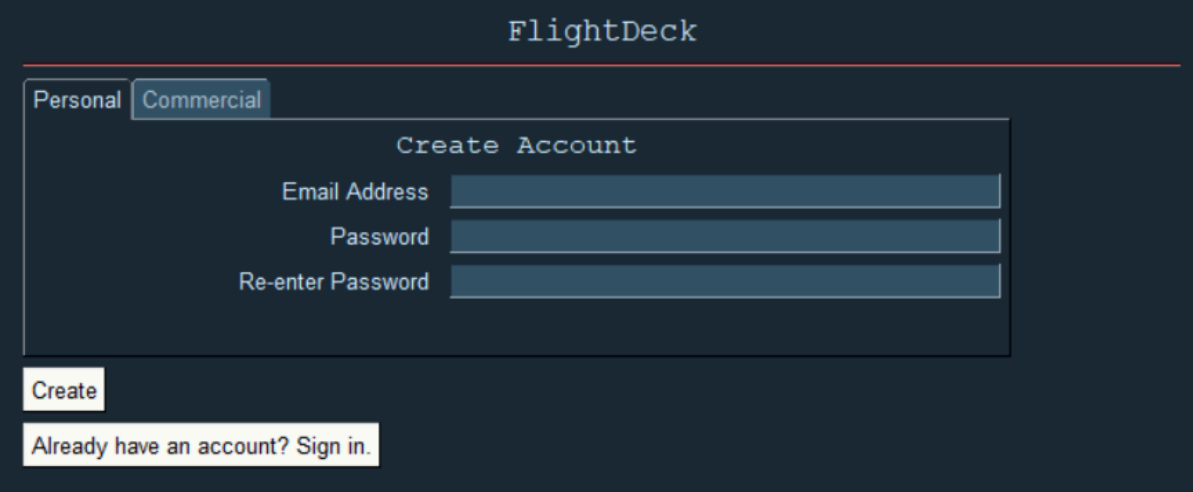


Figure 4.1(s) ▲

For Usability Test Plan



The screenshot shows a web application titled "FlightDeck". At the top, there are two tabs: "Personal" and "Commercial", with "Commercial" being the active tab. Below the tabs is a "Create Account" form. The form contains three input fields: "Email Address", "Password", and "Re-enter Password". Below the form is a "Create" button. At the bottom of the form, there is a link that says "Already have an account? Sign in."

Figure 4.2(a) ▲

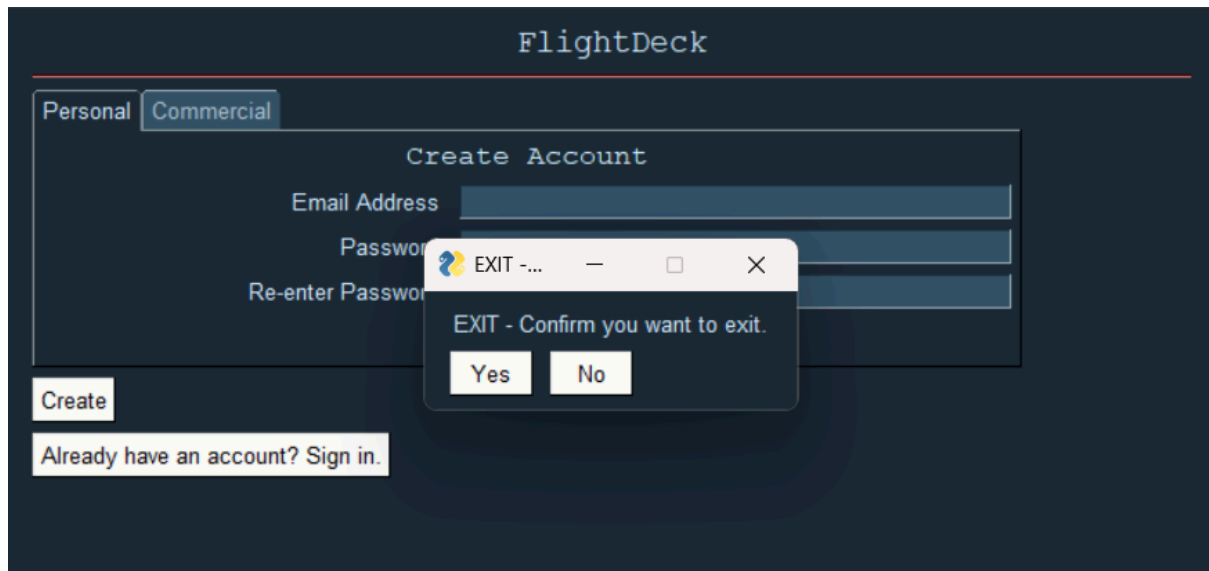
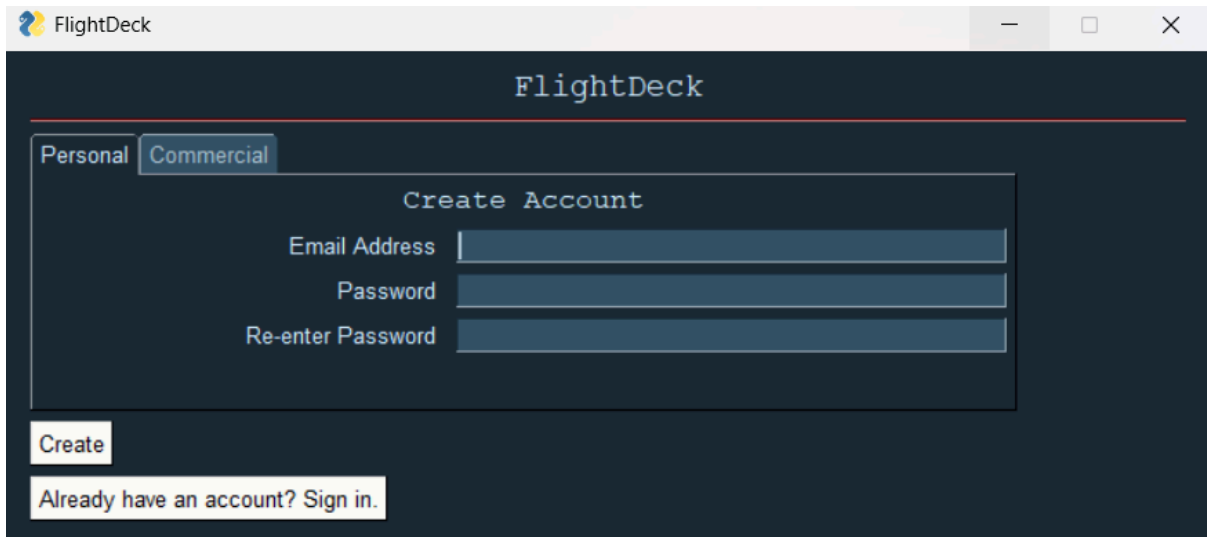


Figure 4.2(b) ▲

Selecting "Yes" was a success. The program closed without having to return to the IDE.



FlightDeck

Personal Commercial

Create Account

Email Address

Password

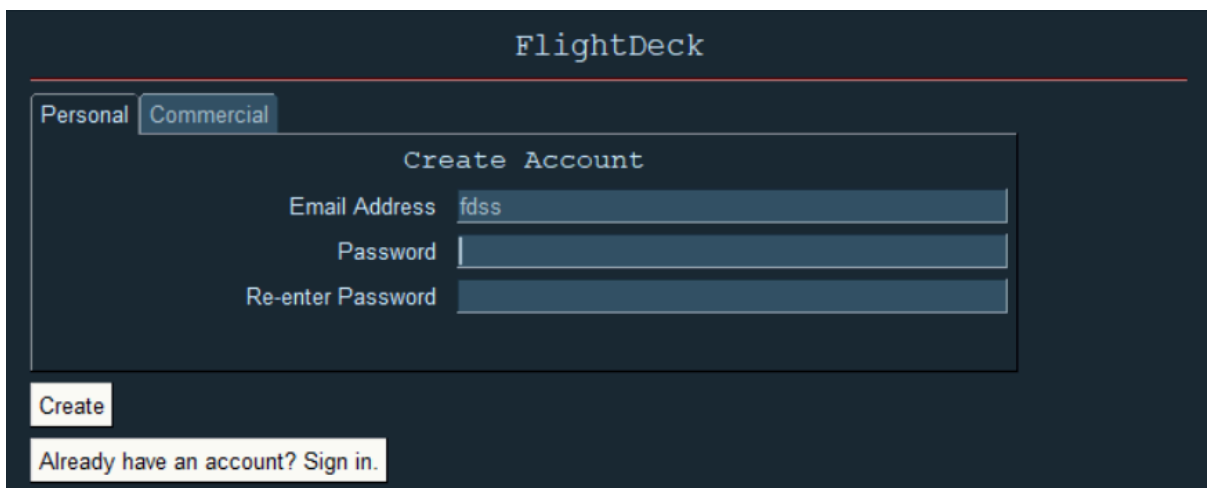
Re-enter Password

Create

Already have an account? Sign in.

Figure 4.2(c) ▲

Selecting the minimise button was a success. The window was minimised.



FlightDeck

Personal Commercial

Create Account

Email Address fdss

Password

Re-enter Password

Create

Already have an account? Sign in.

Figure 4.2(d) ▲

The screenshot shows the 'FlightDeck' login page. At the top, there are two tabs: 'Personal' and 'Commercial', with 'Commercial' selected. Below the tabs is a 'Create Account' form. The form has three input fields: 'Email Address' with the text 'fdss', 'Password', and 'Re-enter Password'. Below the form is a 'Create' button. At the bottom, there is a link that says 'Already have an account? Sign in.'

Figure 4.2(e) ▲

Screenshot taken while clicking the “Already have an account...” button.

The screenshot shows the 'FlightDeck' login page. At the top, there are two tabs: 'Flight Information and Management System' and 'Sign-In', with 'Sign-In' selected. Below the tabs is a 'Sign-In' form. The form has two input fields: 'Email Address' with the text 'test' and 'Password'. Below the form is a 'Sign In' button.

Figure 4.2(f) ▲

This screenshot is identical to the previous one, showing the 'FlightDeck' login page with the 'Sign-In' tab selected and the 'Sign-In' button highlighted.

Figure 4.2(g) ▲

Screenshot taken while clicking the “Sign In” button.

FlightDeck

Manage AlertsNotificationsSearch for Information

Drop-Down: Search for Flights / Airports

Enter a flight number:

Enter an airport IATA/ICAO code (e.g. LHR for Heathrow):

Enter the flight number of the alert you wish to delete:

Figure 4.2(h) ▲

FlightDeck

Manage AlertsNotificationsSearch for Information

Drop-Down: Search for Flights / Airports

FlightsAirports

Enter a flight number:

Enter an airport IATA/ICAO code (e.g. LHR for Heathrow):

Enter the flight number of the alert you wish to delete:

Figure 4.2(i) ▲

FlightDeck

Manage AlertsNotificationsSearch for Information

Drop-Down: Search for Flights / Airports

Search for Flight

☐ Show Stats For Nerds

☐ Create An Alert For This Flight

Enter a flight number:

Figure 4.2(j) ▲

FlightDeck

Manage AlertsNotificationsSearch for Information

Drop-Down: Search for Flights / Airports

Search for Airport

Enter an airport IATA/ICAO code (e.g. LHR for Heathrow):

Figure 4.2(k) ▲

Success-Criteria Testing

See [Success Criteria](#) to see what has been met or not met. Any testing will have been done within the [Post-Development Test Plan \(Evidence and Annotations\)](#), [Usability Test Plan \(Evidence and Annotations\)](#) or [Development of the Solution](#).

Future Address of Partially/Mostly Met and Unmet Success Criteria

From [Success Criteria](#), the main features which were not implemented were commercial features, such as 2c(i) - the ability to create a fleet of aircraft operated, 2c(iii) - the ability to show additional data for efficiency management. This was for a range of different reasons.

The main reason for unmet success criteria was the level of complexity that would be required, within a limited project timeframe (approximately one year), to implement specific features. For example, success criteria 2c(i) would likely involve having to create an SQL statement to create an entire new table, within the database, for each commercial user to store a fleet of aircraft (and for it to be shareable across that company most likely). This would be too complex for this timeframe and would require a level of scalability that at the moment, my project doesn't support (e.g. linking to having to host my database online or on a cloud-based platform).

Similarly, with the idea of success criteria 2c(iii), the FlightRadar24 API does not data attributes, such as fuel efficiency, ect, so I would have to integrate another API service and would add an additional level of complexity (that would also not likely be free of charge).

Finally, some ideas I just didn't seem necessary to implement for specific reasons or the fact that they may become redundant. For example, success criteria 4b(i), involved returning an information message, should the FlightRadar24 API be unavailable (under maintenance, etc), however the API already automatically returns an error message should it be unavailable

Future Address of Partially/Mostly Met and Unmet Functionality

From my [Post-Development Test Plan \(Evidence and Annotations\)](#) and [Usability Test Plan \(Evidence and Annotations\)](#), I can see that all API and SQL queries work as intended, which is the important part.

The majority of unexpected issues are caused by validation methods being very strong in the account creation and sign-in window but missed in the main window. The majority fell to not including a GUI displayed message to inform the user of the unsatisfied validation criteria or invalid user inputs that didn't cause major issues, unlike the small number actually crashing the program. With many of the invalid user inputs, I had not thought to implement validation against, until the final testing stage was carried out and I could see other people's difficulties in usability.

In the future, particularly since I would continue my development focussed on the commercial use and functionality side of things, I would ensure that validation is a priority to implement across features that I hadn't previously thought about.

The other key failures of testing that I noticed revolved around the complexity of nested selection statements that extend across the entirety of my program code (particularly since all GUI elements sit within while loops in order to keep them actively refreshing and prevent freezing). Specifically with branches that involved updating GUI elements, such as altering the “visible” element. Hence why during final testing, some tests (like [Post-Development Test 4ai](#)) remain unmet, as I found it very difficult to keep track of which elements were being made visible or being hidden, at different times and under different circumstances.

In the future, it would likely be a good idea to develop a flow chart of the nested selection statements, their pre-requisites and what GUI element attributes are altered within them. This way, I would have a visual representation to reference and would have likely aided me in solving this issue, as throughout development, I could return SQL and API queries to the IDE terminal, just not always to the user interface.

Maintenance Issues and General Limitations

Maintenance Limitations

Libraries And Other Import Limitations

One key maintenance limitation follows the fact that during initial development, I had intended to make use of an open-source GUI library called PySimpleGUI, however I found that it had been discontinued without purchase of a license. Therefore, I came across a previous open-source version of PySimpleGUI that someone had branded as FreeSimpleGUI.

The use of this previous library version, to my knowledge, doesn't support developer updates and maintenance (with the latest versions of PySimpleGUI now being closed-source). Pre-coded libraries should be well tested but any missed bugs or adaptations to features, for example to improve code efficiency, will have to be made on the users own behalf. Likely requiring high skill.

Database Management Limitations

After completion of my NEA project, database management will no longer be a priority and therefore, maintenance will be limited if at all. This means that database backups will no longer be supported and so, data loss could occur, in the event of corruption, etc.

Additionally, because of this, should use of my application continue afterwards and the database expand in size, its time-space complexity will increase, making searches using SQL slow. I am also unsure if PhpMyAdmin includes any performance or database size limitations, which would never be a current issue, but could become an issue in this scenario.

General Limitations

API Sandbox and Scalability Limitations

In order to make my program development feasible (economically), I used the free sandbox environment, which allowed me to make an unlimited number of API queries, without having to purchase a subscription plan (\$9/month for 30,000 credits - lowest tier). However, in order for FlightRadar24 to justify this free development environment, all query returns are static with default returns.

This meant that during testing of API reliant stages, it was impossible to alter test data to meet specific selection conditions, in order to fully test functionality. This was clear during development of [3.8 “Notifications” Tab](#), where multiple nested selection statements were used and during testing, I realised from my “alerts” table within my “flightdeckdb” database, that my “test” record consisted of date-time attributes dating back to the year 2023. In some cases, making no pre-conditions achievable.

Additionally, the API service enforces a query limit in order to maintain performance across the platform. This means that, should the project be expanded, I could encounter issues with these limits in the future.

Database Scalability Limitations

My database, used to store user credentials, alerts and company domains and logos for commercial use, is a MySQL database that is stored locally on my device. This also presents a scalability issue, as anyone who would want to use my program on a different client, would have to install XAMPP (a tool used to access the database), as well as python libraries, to access data within the database.

Development Complexity / Time Limitations

Unfortunately, due to development complexity and time limitations, some aspects of my program which I initially intended to implement (see [Success Criteria](#)) have not been implemented, or have been adapted or implemented at a smaller scale. This mainly consisted of commercial features. For example, commercial features, such as, company aircraft fleet management (see [Success Criteria](#) - 2a(i)).

User Feedback / Use

To an extent in any program, a user's interaction with specific features (mostly GUI elements) plays a role in limitations, as each individual processes assumptions / biases on how to navigate a system differently.

Future Developments To Meet Limitations

API Sandbox and Scalability Limitations

In order to overcome API scalability limitations, I would have to take a more efficient programming approach (e.g. making multiple queries within individual statements **or** creating my own flight / airport search limit for users), in order to steer clear of the query limit.

However, I would imagine that with the API solution being designed with a commercial over personal use aspect in mind, the query limit is most likely quite high and wouldn't be an issue for the relatively small scale size of my project if it were to be expanded further.

In the future, I would also have to transition from a sandbox environment to a paid environment, which would not only allow me to return live data to users, but also complete some post-development tests that are otherwise impossible to carry out with static historical data returns.

Database Scalability Limitations

To allow my program to access data within my database (via SQL queries), when being executed on separate clients, I would have to transition my locally hosted database to one hosted online or a cloud-based solution. This would make data accessible but costly and presumably with increased difficulty, in terms of integration into a Python environment.

Development Complexity / Time Limitations

Simply, a long-term development timeframe would make development complexity negligible, to a large extent. The only development complexities that wouldn't be overcome by an extended timeframe would be those relating to resource or economic requirements, such as, database capacity and search / sorting time complexity, API service costs, etc.

User Feedback / Use

To mitigate incorrect user interaction with GUI elements, the layout and simplicity where possible has been well thought out (see [Usability Features](#)). Additionally, I could have included a tutorial / instructions / help tab as part of my development plan, where labelled images of my interface would aid users in how to correctly navigate and use different functions.

Overall Conclusion

From my development, I have had the most success in development of API and SQL queries, which is an achievement to say that I had no previous experience, other than basic SQL from the A level specification.

Additionally, even though there were some logic issues as stated in [Future Address of Partially/Mostly Met and Unmet Functionality](#), I was successful in structuring the GUI elements and handling the different attributes that could accompany the different elements, and I managed to update and create them successfully (mostly).

In summary, for future development of my project, I would work on the commercial aspects and improve functionality in terms of validation.

Code Appendix

```
#Imports the library FreeSimpleGUI and assigns the alias of "GUI" for faster coding.
#E.g. "GUI." is equivalent to "FreeSimpleGUI."
import FreeSimpleGUI as GUI
GUI.theme('DarkBlue')

#Imports the FlightRadar24 API and SDK.
from fr24sdk.client import Client
#Takes the API token from a pre-defined environmental variable, via Windows PowerShell.
client =
Client(api_token="0198c22c-876f-718e-bca1-a360872c8820|Y6pXXBQN250dYwViBPrqGsPWVr7q8AoPlykCzQBA832cb895")

#Other imports
import mysql.connector
import hashlib
from datetime import datetime, timedelta

#These two variables are used later on, in the "get_SearchFLT" function, as this API query
#requires a date-time range to search within and became inefficient having to change the date
#to meet the pre-requisites, this now does this for me and meets the 14-day max range.
TimeFrom = datetime.now()
TimeTo = TimeFrom + timedelta(days=14)

#Uses the MySQL connector in order to integrate the flightdeck database to my program.
db = mysql.connector.connect(
    host = "localhost",
    user = "root",
    password = "",
    database = "flightdeckdb"
)

#Quick reference messages for successful or unsuccessful database connection on start.
if db.is_connected():
    print("DB - Connection Successful")
else:
    print("DB - Connection Issue")

#Opens the cursor (a tool used to run MySQL queries and retrieve data).
Cursor = db.cursor()
```

Key Function: *get_title1()*

```
#get_title1 getter procedure for title text.
def get_title1():
    return [
        [GUI.Text(text="FlightDeck", key="-TITLE-", expand_x=True,
justification='c', font=("Courier New", 14))],
        [GUI.HorizontalSeparator(color = "maroon")]
    ]
```

Key Function: *get_basic_create()*

```
#get_basic_create getter procedure for basic account creation layout.
#Includes all error message variants for the account creation page.
def get_basic_create():
    return [
        [GUI.Text(text="Create Account", expand_x=True, justification='c',
font=("Courier New", 12))],

        [GUI.Push(), GUI.Text('Email Address'), GUI.Input(do_not_clear=False,
key='-EMAIL_PER-')],
        [GUI.Push(), GUI.Text('Password'), GUI.Input(do_not_clear=False,
password_char='*', key='-PASS_ENT_PER-')],
        [GUI.Push(), GUI.Text('Re-enter Password'),
GUI.Input(do_not_clear=False, password_char='*', key='-PASS_REENT_PER-')],

        [GUI.Text(text="Password must be at least 8 characters long.",
expand_x=True, justification='l', font=("Arial", 11), text_color=("#de0000"),
key='-8CHAR_ERR_PER-', visible=False)],
        [GUI.Text(text="Passwords must match.", expand_x=True,
justification='l', font=("Arial", 11), text_color=("#de0000"),
key='-PASS_MATCH_ERR_PER-', visible=False)],
        [GUI.Text(text="Account Creation Successful. Standby..",
expand_x=True, justification='l', font=("Arial", 11), text_color=("#1ede00"),
key='-CREATED_MSG_PER-', visible=False)],
        [GUI.Text(text="This email address is already associated with an
account.", expand_x=True, justification='l', font=("Arial", 11),
text_color=("#de0000"), key='-EXIST_ERR_PER-', visible=False)]

    ]
```

Key Function: get_commerce_create()

```

#get_commerce_sign getter procedure for basic sign in integrated into
commercial sign in layout.
def get_commerce_create():
    return [
        [GUI.Text(text="Create Account", expand_x=True, justification='c',
font=("Courier New", 12))],

        [GUI.Push(), GUI.Text('Company Domain (e.g. company.com)'),
GUI.Input(do_not_clear=False, key='-COMPANY-')],
        [GUI.Push(), GUI.Text('Email Address'), GUI.Input(do_not_clear=False,
key='-EMAIL_CO-')],
        [GUI.Push(), GUI.Text('Password'), GUI.Input(do_not_clear=False,
password_char='*', key='-PASS_ENT_CO-')],
        [GUI.Push(), GUI.Text('Re-enter Password'),
GUI.Input(do_not_clear=False, password_char='*', key='-PASS_REENT_CO-')],

        [GUI.Text(text="Password must be at least 8 characters long.",
expand_x=True, justification='l', font=("Arial", 11), text_color=("#de0000"),
key='-8CHAR_ERR_CO-', visible=False)],
        [GUI.Text(text="Passwords must match.", expand_x=True,
justification='l', font=("Arial", 11), text_color=("#de0000"),
key='-PASS_MATCH_ERR_CO-', visible=False)],
        [GUI.Text(text="Account Creation Successful. Standby..",
expand_x=True, justification='l', font=("Arial", 11), text_color=("#1ede00"),
key='-CREATED_MSG_CO-', visible=False)],
        [GUI.Text(text="This email address is already associated with an
account.", expand_x=True, justification='l', font=("Arial", 11),
text_color=("#de0000"), key='-EXIST_ERR_CO-', visible=False)]
    ]

#Integrates the basic creation interface and the commercial creation
interface into one variable.
#Allows me to use the "enable_events" feature to track what account creation
tab the user is actively using
#Via the "-TAB_GROUP-" key identifier.
create_tabs = [
    [GUI.TabGroup(
        [[
            GUI.Tab("Personal", get_basic_create()),
            GUI.Tab("Commercial", get_commerce_create())
        ]],
        key="-TAB_GROUP-",
        enable_events=True
    )]
]

```

```
#Create buttons with specific unique keys for the event-triggered dictionary inputs.
create_buttons = [
    [GUI.Button('Create', bind_return_key=True)],
    #The "Already have an account? Sign in." button has been assigned with the
    event identifier "-GO_SIGN-" to skip validation
    #and to close "window1" and create "window2", the same as if validation
    were to be met when creating an account.
    [GUI.Button('Already have an account? Sign in.', bind_return_key=True,
key="-GO_SIGN-")]
]

#Combines all the sub-layouts into one main layout for the window1_run()
function.
create_FLayout = get_title1() + create_tabs + create_buttons
```

Key Function: hash_password(password)

```
#Password hashing algorithm.
def hash_password(password):
    #Uses the hashlib SHA512 hash function to hash the entered password.
    #uft-8 converts the password string into bytes as hashing functions cannot
    operate on high-level text (only machine level).
    return hashlib.sha256(password.encode('utf-8')).hexdigest()
```

Key Function: window1_run()

```
def window1_run():
    #Any functions / procedures regarding physical and device specific changes
    to the window can only be made after
    #it has been finalised using the procedure ".Finalize()"
    window1 = GUI.Window('FlightDeck', create_FLayout, size = (700, 400),
enable_close_attempted_event = True).Finalize()

    #Infinite loop that keeps the window event actively running.
    while True:
        #"create_values" is a dictionary created that contains inputs.
        event, create_values = window1.read()
        print(event, create_values)
        active_tab = create_values['-TAB_GROUP-'] if "-TAB_GROUP-" in
create_values else None

        #If an event is triggered by the user switching tabs, this condition
        ensures false error messages aren't returned
        #without an invalid input.
        if event == '-TAB_GROUP-':
            window1['-8CHAR_ERR_PER-'].update(visible=False)
            window1['-PASS_MATCH_ERR_PER-'].update(visible=False)
            window1['-8CHAR_ERR_CO-'].update(visible=False)
            window1['-PASS_MATCH_ERR_CO-'].update(visible=False)
            continue

        #If the user attempts to close the window, a popup will appear (center
        screen) to confirm their decision.
        if (event == GUI.WINDOW_CLOSE_ATTEMPTED_EVENT or event == 'Exit') and
GUI.popup_yes_no('EXIT - Confirm you want to exit.') == 'Yes':
            Cursor.close()
            db.close()
            break

        #If the event identifier is equal to "-GO_SIGN-", which is set from
        the "Already have an account? Sign in."
        #button push on the interface, "window1" is closed and "window2" is
        created.
        if event == "-GO_SIGN-":
            window1.close()
            window2_run()

        #Assigns an index of the output dictionary "create_values" to
        different equivalent variables using keys.
        #One variable may have multiple keys, if the first key index is blank,
        it uses the second.
        password_per = create_values["-PASS_ENT_PER-"]
        password_co = create_values["-PASS_ENT_CO-"]
        password_reinput_per = create_values["-PASS_REENT_PER-"]
        password_reinput_co = create_values["-PASS_REENT_CO-"]
        email_address_per = create_values["-EMAIL_PER-"]
        email_address_co = create_values["-EMAIL_CO-"]
        company_domain = create_values["-COMPANY-"]
```

```
#Sets all validation variables / markers to "True", so that they can
be flipped if needed.
valid_8char_per = True
valid_8char_co = True
valid_passmatch_per = True
valid_passmatch_co = True
valid_domain = True

#Validates that the original password is longer than 8 characters.
if active_tab == 'Personal':
    if len(password_per) < 8:
        window1['-8CHAR_ERR_PER-'].update(visible=True)
        valid_8char_per = False
    if password_per != password_reinput_per:
        window1['-PASS_MATCH_ERR_PER-'].update(visible=True)
        valid_passmatch_per = False

#If the user is currently on the "Commercial" tab (create account
window).
elif active_tab == 'Commercial':

    #Selects the top record (only consisting of the CompanyDomain
attribute) from the
    #"domains" table under the "flightdeckdb" database, where the
"CompanyDomain" is equal to
    #the user-entered domain.
    domain_check = "SELECT `CompanyDomain` FROM domains WHERE
`CompanyDomain` = %s"
    Cursor.execute(domain_check, (company_domain,))
    domain_result = Cursor.fetchone()

    if len(password_co) < 8:
        window1['-8CHAR_ERR_CO-'].update(visible=True)
        valid_8char_co = False
    if password_co != password_reinput_co:
        window1['-PASS_MATCH_ERR_CO-'].update(visible=True)
        valid_passmatch_co = False

    #If a result is returned from the domain_check SQL query, the
company domain entered
    #by the user must exist. Verification passed.
    if domain_result:
        event, create_values = window1.read(timeout=3000)
        print("COMPANY DOMAIN CHECK SUCCESS")

        valid_domain = True

    else:
        GlobalDomain = False

    if (active_tab == "Personal" and valid_8char_per and
valid_passmatch_per) or (active_tab == "Commercial" and valid_8char_co and
valid_passmatch_co and valid_domain):
```

```
#If password validation is met (minimum 8 characters and both
entries matching)
#the password is hashed via the
#hash_password(password) function defined above.
hashed_password = hash_password(password_per or password_co)

# Inserts the email address, password and company name from the
above inputs into the column names
exists_check = "SELECT `EmailAddress` FROM credentials WHERE
`EmailAddress` = %s"
Cursor.execute(exists_check, (email_address_per or
email_address_co,))
exists_result = Cursor.fetchone()

# stored in "values". The cursor then executes that operation with
"sql1" and "values" and is committed
#Checks if an account has already been created with the same
email, if so, the program will close.
if company_domain == "":
    company_domain = None

if not exists_result:
    sql1 = "INSERT INTO Credentials (`EmailAddress`, `Password`,
`CompanyName`) VALUES (%s, %s, %s)"
    values = ((email_address_per or email_address_co),
hashed_password, company_domain)
    Cursor.execute(sql1, values)
    db.commit()
    window1['-CREATED_MSG_PER-'].update(visible=True)
    window1['-CREATED_MSG_CO-'].update(visible=True)
    event, create_values = window1.read(timeout=3000)

    window1.close()
    window2_run()

else:
    window1['-EXIST_ERR_PER-'].update(visible=True)
    window1['-EXIST_ERR_CO-'].update(visible=True)
    event, create_values = window1.read(timeout=2500)

    window1.close()
    break
```

Key Function: get_sign()

```

#get_sign getter procedure for sign in layout.
#Includes all error message variants for the sign-in page.
def get_sign():
    return [
        [GUI.Text(text="Sign-In", expand_x=True, justification='c',
font=("Courier New", 12))],

        [GUI.Push(), GUI.Text('Email Address'), GUI.Input(do_not_clear=False,
key='-EMAIL-')],
        [GUI.Push(), GUI.Text('Password'), GUI.Input(do_not_clear=False,
password_char='*', key='-PASS-')],

        [GUI.Text(text="Invalid Credentials.", expand_x=True,
justification='l', font=("Arial", 11), text_color=("#de0000"),
key='-CREDS_ERR-', visible=False)],
        [GUI.Text(text="Sign-In Successful. Standby..", expand_x=True,
justification='l', font=("Arial", 11), text_color=("#1ede00"),
key='-SIGN_MSG-', visible=False)]
    ]

#Create buttons with specific unique keys for the event-triggered dictionary
inputs.
sign_buttons = [
    [GUI.Button('Sign In', bind_return_key=True)]
]

#A singular tab has been used to group the get_sign() elements as I couldn't
figure out how to
#make them all group to the left-hand side of the window... this saves
development time...
sign_tab = [
    [GUI.TabGroup(
        [[GUI.Tab("Flight Information and Management System", get_sign())]]
    )]
]

sign_FLayout = get_title1() + sign_tab + sign_buttons

```

Key Function: window2_run()

```

def window2_run():
    #Declares the value of sign_values["-EMAIL-"] as a
    #global variable for later use (in "window3_run").
    global GlobalEmail

    window2 = GUI.Window('FlightDeck', sign_FLayout, size = (700, 400),
enable_close_attempted_event = True).Finalize()
    while True:
        event, sign_values = window2.read()
        print(event, sign_values)

        if (event == GUI.WINDOW_CLOSE_ATTEMPTED_EVENT or event == 'Exit') and
GUI.popup_yes_no('EXIT - Confirm you want to exit.') == 'Yes':
            Cursor.close()
            db.close()
            break

        #Assigns an index of the output dictionary "sign_values" to different
equivalent variables using
        #the equivalent keys.
        password_check = sign_values["-PASS-"]
        email_address_check = sign_values["-EMAIL-"]
        #This executes a hash algorithm on the password entered on the sign-in
page to compare later on
        #and since the original password has been hashed, it cannot be
reverted to its original state due to
        #hashing being a one-way function.
        hashed_password_check = hash_password(password_check)

        #The "credentials_check" statement selects all records from the table
"credentials" where the email address
        #and password are equal, respectively, to the "email_address_check"
and "hashed_password_check".
        credentials_check = "SELECT * FROM `credentials` WHERE `EmailAddress`
= %s AND `Password` = %s"
        Cursor.execute(credentials_check, (email_address_check,
hashed_password_check))
        credentials_result = Cursor.fetchone()

        #If no result is found, the credentials are deemed incorrect as they
must be correct initially at the
        #create account stage. The credentials invalid error message is
displayed by settings its "visible"
        #attribute to "True".
        if not credentials_result:
            window2['-CREDS_ERR-'].update(visible=True)
        #If a result is found, the credentials are deemed correct. The sign-in
successful message ("-SIGN_MSG-") is
        #displayed by setting its "visible" attribute to "True". Window3 is
executed, window2 is closed.
        else:
            window2['-SIGN_MSG-'].update(visible=True)

```

```
        #3000ms window2 timeout to allow the user to read the message
before closing.
        event, sign_values = window2.read(timeout=3000)
        GlobalEmail = email_address_check

        window2.close()
        window3_run()
```

Key Function: *get_searchAPT(SearchEntApt)*

```
#Defines the "get_searchAPT()" function which searches for an airport.
def get_searchAPT(SearchEntApt):
    #Uses "client" as the instance of the FlightRadar24 API key.
    with
Client(api_token="0198c22c-876f-718e-bca1-a360872c8820|Y6pXXBQN250dYwViBPrqGs
PWVr7q8AoPlYkCzQBA832cb895") as client:

        #Declares the variable "airport" and uses the API key instance to
return a desired collection of
        #attributes under the "get_light" pre-determined function, with the
airport ICAO code as a parameter.
        airport = client.airports.get_light(code = SearchEntApt)

    return airport
```

Key Function: *get_searchFLT(SearchEntFlt)*

```
#Defines the "get_searchFLT()" function which searches for flight instances.
def get_searchFLT(SearchEntFlt):
    with
Client(api_token="0198c22c-876f-718e-bca1-a360872c8820|Y6pXXBQN250dYwViBPrqGs
PWVr7q8AoPlYkCzQBA832cb895") as client:

        #Performs an API query using the "flight_positions.get_full()"
function. This query uses
        #the user input, identifier "SearchEntFlt", as the value for the
"flights" parameter.
        flight_positions = client.live.flight_positions.get_full(flights =
[SearchEntFlt])

        #Performs an API query using the "flight_summary.get_full()" function.
This query uses the
        #same user input as the value for the "flights" parameter.
        flight_summary = client.flight_summary.get_full(
            flights = [SearchEntFlt],
            #The "TimeFrom" and "TimeTo" functions, part of the "datetime"
library
            #are used to automate the date/time range parameters.
            flight_datetime_from = TimeFrom.isoformat(timespec='seconds'),
            flight_datetime_to = TimeTo.isoformat(timespec='seconds'))

    return flight_summary, flight_positions
```

```

#Groups the drop-down menu, under "Search for Information"
#into the respective search menus.
search_menus = [
    [GUI.ButtonMenu('Drop-Down: Search for Flights / Airports',
                    ['Menu', ['Flights::-FLT-', 'Airports::-APT-']],
                    key="-MENU_OPT-")]
]

#A list of elements that appear within the "Search for Information" tab. This
allows me to group them all
#under one collective list that can be used as a (GUI layout) parameter for
the "window3_run" function.
SearchElements = [
    #Buttons
    [GUI.Button('Search for Flight', key='-SEARCHBT_FLT-',
bind_return_key=True, visible=False)],
    [GUI.Button('Search for Airport', key='-SEARCHBT_APT-',
bind_return_key=True, visible=False)],
    [GUI.Button('Delete Alert (Double Click)', key='-ALERTDELETEBT-',
bind_return_key=True, visible=False)],
    #Tickboxes
    [GUI.Checkbox('Show Stats For Nerds', default=False, key="-NERDCHK_FLT-",
visible=False)],
    [GUI.Checkbox('Create An Alert For This Flight', default=False,
key="-ALERTCHK_FLT-", visible=False)],

    #Text and Inputs (each with a unique identifier).
    [GUI.Push(), GUI.Text('Enter a flight number: ', key =
"-SEARCHENT_FLT_TEXT-"), GUI.Input(justification = "l", do_not_clear=False,
key='-SEARCHENT_FLT-')],
    [GUI.Push(), GUI.Text('Enter an airport IATA/ICAO code (e.g. LHR for
Heathrow): ', key = "-SEARCHENT_APT_TEXT-"), GUI.Input(justification = "l",
do_not_clear=False, key='-SEARCHENT_APT-')],
    [GUI.Push(), GUI.Text('Enter the flight number of the alert you wish to
delete: ', key = "-ALERTDELETE_FLT_TEXT-"), GUI.Input(justification="l",
do_not_clear=False, key='-ALERTDELETE_FLT-')],
]

#A list of tabs that appear within Window3, each taking the user
#to the respective place.
MainTabs = [
    [GUI.TabGroup(
        [[
            #Tabs can include functions and other GUI elements
            #within them (e.g. "get_title1()").
            GUI.Tab("Manage Alerts", get_title1()),
            GUI.Tab("Notifications", get_title1()),
            GUI.Tab("Search for Information", search_menus)
        ]],
        #Allows the changing of tabs to act as an event trigger,
        #with the identifier "-TAB_GROUP-"
        key="-TAB_GROUP-",

```

```

        enable_events=True
    ])
]

#These elements will display returns of API queries within respective tabs.
Base64 = ""
OutputElements= [
    [GUI.Text(expand_x=True, justification='l', font=("Arial", 11),
key="-AIRPORT-", visible=False)],
    [GUI.Text(expand_x=True, justification='l', font=("Arial", 11),
key="-FLIGHTSUMMARY-", visible=False)],
    [GUI.Text(expand_x=True, justification='l', font=("Arial", 11),
key="-FLIGHTPOSITIONS-", visible=False)],
    [GUI.Text(expand_x=True, justification='l', font=("Arial", 11),
key="-NERD-", visible=False)],
    [GUI.Text(text="Alert Creation Successful", expand_x=True,
justification='l', font=("Arial", 11), text_color="#1ede00",
key='-ALERTCREATED-', visible=False)],
    [GUI.Text(text="This email address is already associated with an alert for
this flight.", expand_x=True, justification='l', font=("Arial", 11),
text_color="#de0000", key='-ALERTEXISTS-', visible=False)],
    [GUI.Text(text="", expand_x=True, justification='l', font=("Arial", 11),
key='-NOTIFICATION-', visible=True)],
    [GUI.Image(Base64, key="-LOGO-", visible=False)]
]

#Concatenates individual functions and element groups into one variable (see
>window3_run()).
main_FLayout = get_title1() + MainTabs + SearchElements + OutputElements

```

Key Function: window3_run()

```

def window3_run():
    window3 = GUI.Window('FlightDeck', main_FLayout, size = (700, 400),
enable_close_attempted_event = True).Finalize()
    window3.maximize()

    while True:
        event, main_values = window3.read()
        print(event, main_values)
        active_tab = main_values['-TAB_GROUP-'] if "-TAB_GROUP-" in
main_values else None

        #Assigns the "-SEARCHENT_FLT-" and "-SEARCHENT_APT-" identifier,
within the "main_values" dictionary,
        #to the respective variables, as identifiers cannot be passed into
functions as parameters.
        SearchEntFlt = main_values['-SEARCHENT_FLT-']
        SearchEntApt = main_values['-SEARCHENT_APT-']

        if (event == GUI.WINDOW_CLOSE_ATTEMPTED_EVENT or event == 'Exit') and
GUI.popup_yes_no('EXIT - Confirm you want to exit.') == 'Yes':
            Cursor.close()
            db.close()
            break

        CompanyCheck = "SELECT `CompanyName` FROM credentials WHERE
`EmailAddress` = %s AND `CompanyName` IS NOT NULL"
        Cursor.execute(CompanyCheck, (GlobalEmail,))
        # Fetches one (the first) value.
        CompanyCheckResult = Cursor.fetchone()

        if CompanyCheckResult:
            Logo = "SELECT `Logo` FROM domains WHERE `CompanyDomain` LIKE %s"
            Cursor.execute(Logo, (CompanyCheckResult[0],))
            #Fetches one (the first) value.
            LogoResult = Cursor.fetchone()
            Base64 = LogoResult[0]

            window3["-LOGO-"].update(data=Base64, visible=True, subsample=3)

        if active_tab == "Search for Information":

            #Displays all GUI elements required to form an API query, to
search for an airport.
            if main_values["-MENU_OPT-"] == "Airports::-APT-":
                window3['-SEARCHENT_FLT_TEXT-'].update(visible=False)
                window3['-SEARCHENT_FLT-'].update(visible=False)
                window3['-SEARCHBT_FLT-'].update(visible=False)
                window3["-NERDCHK_FLT-"].update(visible=False)
                window3["-ALERTCHK_FLT-"].update(visible=False)

                window3['-SEARCHENT_APT_TEXT-'].update(visible=True)
                window3['-SEARCHENT_APT-'].update(visible=True)

```

```

        window3['-SEARCHBT_APT-'].update(visible=True)
        window3['-ALERTDELETE_FLT_TEXT-'].update(visible=False)
        window3['-ALERTDELETE_FLT-'].update(visible=False)
        window3['-ALERTDELETEBT-'].update(visible=False)

        if event == '-SEARCHBT_APT-':
            #Triggers the "get_searchAPT" API query function with the
user input assigned to
            #"SearchEntApt", assigned to a dictionary identifier,
being passed as a parameter.
            airport = get_searchAPT(SearchEntApt)
            #Updates the "-AIRPORT-" GUI element with the data stored
in "airport".

            window3["-AIRPORT-"].update(str("Airport Name:" +
airport.name + "\nAirport IATA:" + airport.iata), visible=True)

        #Displays all GUI elements required to form an API query, to
search for a flight.
        if main_values["-MENU_OPT-"] == "Flights::-FLT-":
            window3['-SEARCHENT_FLT_TEXT-'].update(visible=True)
            window3['-SEARCHENT_FLT-'].update(visible=True)
            window3['-SEARCHBT_FLT-'].update(visible=True)
            window3["-NERDCHK_FLT-"].update(visible=True)
            window3["-ALERTCHK_FLT-"].update(visible=True)

            window3['-SEARCHENT_APT_TEXT-'].update(visible=False)
            window3['-SEARCHENT_APT-'].update(visible=False)
            window3['-SEARCHBT_APT-'].update(visible=False)
            window3['-ALERTDELETE_FLT_TEXT-'].update(visible=False)
            window3['-ALERTDELETE_FLT-'].update(visible=False)
            window3['-ALERTDELETEBT-'].update(visible=False)

            window3["-FLIGHTSUMMARY-"].update(visible=True)
            window3["-FLIGHTPOSITIONS-"].update(visible=True)

        if event == '-SEARCHBT_FLT-':
            #Triggers the "get_searchFLT" API query function with the
user input
            #assigned to "SearchEntFlt", being passed as a parameter.
            flight_summary, flight_positions =
get_searchFLT(SearchEntFlt)

            # Ln 441-447: allows me to calculate a rough idea of
whether a flight is early / late.
            ExpectedDuration = (flight_summary.data[0].circle_distance
/ 852) + 0.25

            # By default, "Z" is included within the date/time string,
dictating UTC time.
            # "Z" cannot be included for mathematical calculations, so
is replaced with a blank space.
            ActualDuration = datetime.fromisoformat(

```

```

        flight_positions.data[0].eta.replace("Z", "")) -
datetime.fromisoformat(
        flight_summary.data[0].datetime_takeoff)

    # the ".total_seconds()" / 3600 function and operation
converts
    # seconds into hours, for standardised units for
calculation.
    if (ActualDuration.total_seconds() / 3600) <
ExpectedDuration:
        status = "Likely Early"
    if (ActualDuration.total_seconds() / 3600) >
ExpectedDuration:
        status = "Likely Late"

    #Updates the respective GUI elements with the data from
the API query and sets their
    #visibility to "True". Additional text has also been
provided. Labels, headings, etc.
    #BROKEN: This doesn't appear, potential issue with
visibility after version restoration.
    window3["-FLIGHTSUMMARY-"].update(str("-- Flight Info --"
+ "\nAirline: " + flight_summary.data[0].operating_as + " // Flight: " +
flight_summary.data[0].flight + "\nAircraft: " + flight_summary.data[0].type
+ " // Callsign: " + flight_summary.data[0].callsign + "\nDeparture Airport:
" + flight_summary.data[0].orig_iata + " // Departure Time: " +
flight_summary.data[0].datetime_takeoff), visible=True)
    window3["-FLIGHTPOSITIONS-"].update(str("\nETA: " +
flight_positions.data[0].eta + "\n" + status), visible=True)

    #If the "Show Stats For Nerds" checkbox is checked, then
the following additional
    #attributes will be displayed, after a flight is searched
for.
    if main_values["-NERDCHK_FLT-"]:
        window3["-NERD-"].update(str("\n-- Nerdy Stats --" +
"\nTracking and Data Source: " + flight_positions.data[0].source + "\nSquawk
Code: " + flight_positions.data[0].squawk + "\nAircraft Registration: " +
flight_positions.data[0].reg), visible=True)
    if not main_values["-NERDCHK_FLT-"]:
        window3["-NERD-"].update(visible=False)

    #If the "Create An Alert For This Flight" checkbox has
been selected...
    if main_values["-ALERTCHK_FLT-"]:
        ExistAlert1 = "SELECT `EmailAddress`, `FlightNumber`
FROM alerts WHERE `EmailAddress` = %s AND `FlightNumber` = %s"
        #Executes the above SQL statement "ExistAlert1" and
passes the values of "email_address_per" /
        #"email_address_co" and the flight number of the
queried flight into the respective parameters above.
        Cursor.execute(ExistAlert1, (GlobalEmail,
flight_summary.data[0].flight))

```

```

        #Fetches one (the first) value.
        ExistAlertResult1 = Cursor.fetchone()

        if not ExistAlertResult1:
            AlertSql = "INSERT INTO alerts (`EmailAddress`,
            `FlightNumber`, `DepartureTime`, `CreationTime`) VALUES (%s, %s, %s, %s)"
            AlertValues = (GlobalEmail,
            flight_summary.data[0].flight, flight_summary.data[0].datetime_takeoff,
            flight_positions.data[0].timestamp)
            Cursor.execute(AlertSql, AlertValues)
            #Commits the values to the database under one
transaction.

            db.commit()
            Output = "Alert Creation Successful"
            window3["-ALERTCREATED-"].update(Output,
            visible=True, text_color="#1ede00"))
        else:
            window3["-ALERTEXISTS-"].update(visible=True)

#Redundancy to ensure that respective GUI elements are hidden,
#should the user leave the tab.
else:
    window3['-SEARCHENT_FLT_TEXT-'].update(visible=False)
    window3['-SEARCHENT_FLT-'].update(visible=False)
    window3['-SEARCHBT_FLT-'].update(visible=False)
    window3["-NERDCHK_FLT-"].update(visible=False)
    window3["-ALERTCHK_FLT-"].update(visible=False)

    window3['-SEARCHENT_APT_TEXT-'].update(visible=False)
    window3['-SEARCHENT_APT-'].update(visible=False)
    window3['-SEARCHBT_APT-'].update(visible=False)
    window3['-ALERTDELETE_FLT_TEXT-'].update(visible=False)
    window3['-ALERTDELETE_FLT-'].update(visible=False)
    window3['-ALERTDELETEBT-'].update(visible=False)

if active_tab == "Manage Alerts":
    window3['-SEARCHENT_FLT_TEXT-'].update(visible=False)
    window3['-SEARCHENT_FLT-'].update(visible=False)
    window3['-SEARCHBT_FLT-'].update(visible=False)
    window3["-NERDCHK_FLT-"].update(visible=False)
    window3["-ALERTCHK_FLT-"].update(visible=False)

    window3['-SEARCHENT_APT_TEXT-'].update(visible=False)
    window3['-SEARCHENT_APT-'].update(visible=False)
    window3['-SEARCHBT_APT-'].update(visible=False)
    window3['-ALERTDELETE_FLT_TEXT-'].update(visible=True)
    window3['-ALERTDELETE_FLT-'].update(visible=True)
    window3['-ALERTDELETEBT-'].update(visible=True)

    ExistAlert2 = "SELECT * FROM alerts WHERE `EmailAddress` = %s"
    Cursor.execute(ExistAlert2, (GlobalEmail,))
    # Fetches one (the first) value.
    ExistAlertResult2 = Cursor.fetchone()

```

```

if ExistAlertResult2:
    Separator = ""
    # "len()" function determines the number of chars' within
    # "ExistAlertResult2" to create a line break.
    l = len(ExistAlertResult2[1]) * 2
    for i in range(l):
        Separator = Separator + "-"

    Output = ""
    # Returns the 2nd, 3rd and 4th attribute within
    # the returned "ExistAlertResult2" list.
    for i in range(1, 5):
        # Concatenates the previous value of "Output" with the
value
        # at index "i" from "ExistAlertResult2", plus a line break.
        Output = Output + str(ExistAlertResult2[i]) + "\n"
    Output = Output + Separator
    # Passes the "Output" as a parameter into the ".update()"
function.
    window3["-ALERTCREATED-"].update(Output, text_color="#FFFFFF",
visible=True)

    else:
        Output = "You have not created any flight alerts."
        window3["-ALERTCREATED-"].update(Output, text_color="#FFFFFF",
visible=True)

    if event == "-ALERTDELETEBT-":
        SQLDelete = "DELETE FROM alerts WHERE `EmailAddress` = %s AND
`FlightNumber` = %s"
        Cursor.execute(SQLDelete, (GlobalEmail,
main_values["-ALERTDELETE_FLT-"]))
        db.commit()

    else:
        window3["-ALERTCREATED-"].update(visible=False)
        window3["-FLIGHTSUMMARY-"].update(visible=False)
        window3["-FLIGHTPOSITIONS-"].update(visible=False)

    if active_tab == "Notifications":
        ExistAlert3 = "SELECT * FROM alerts WHERE `EmailAddress` = %s"
        Cursor.execute(ExistAlert3, (GlobalEmail,))
        # Fetches one (the first) value.
        ExistAlertResult3 = Cursor.fetchall()

        SearchEntFlt = ExistAlertResult3[0][2]
        flight_summary, flight_positions = get_searchFLT(SearchEntFlt)

        if flight_summary.data[0].flight_ended:
            window3["-NOTIFICATION-"].update("This flight has ended.",
visible=True)

        return

```

```

        #Ensures the two date-times are of the same format, for
mathematical operations.
        TDeparture =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][3]))

        #If current time is prior to the flight departure time (BEFORE
DEPARTURE)...
        if
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) <
ExistAlertResult3[0][3]:

            #If the time until departure is less than 4 hours...
            if TDeparture < timedelta(hours=4):

                #If no alert has been previously "pushed" for this
flight...
                if ExistAlertResult3[0][5] is None:
                    #Flight departure time - current time = time until
departure (via "timestamp"
                    #attribute, under the FlightRadar24 API
                    "flight_positions" endpoint).
                    window3["-NOTIFICATION-"].update("Time Until
Departure: " + TDeparture, visible=True)
                    TAlert = None

                    #Updates the "LastAlerted" attribute with the current
timestamp.
                    UpdateAlertSql = "UPDATE alerts SET `LastAlerted` = %s
WHERE `EmailAddress` = %s"
                    UpdateAlertValues =
(datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")),
GlobalEmail)
                    Cursor.execute(UpdateAlertSql, UpdateAlertValues)
                    db.commit()

                #Else (if an alert has been previously "pushed" for this
flight)...
                else:
                    #TAlert (time since last alert) is calculated.
                    TAlert =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][5]))

                    #If the time since the last alert (TAlert) is >= 20
minutes...
                    if TAlert >= timedelta(minutes=20):

                        #An up-to-date API query is made.
                        flight_summary, flight_positions =
get_searchFLT(SearchEntFlt)

```

```

#Calculates an up-to-date "TDeparture" and ensures
the two date-times
#are of the same format, for mathematical
operations.

TDeparture =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][3]))

#Outputs "TDeparture"
window3["-NOTIFICATION-"].update("Time Until
Departure: " + TDeparture, visible=True)

#Updates the "LastAlerted" attribute with the
current timestamp.
UpdateAlertSql = "UPDATE alerts SET `LastAlerted`
= %s WHERE `EmailAddress` = %s"
UpdateAlertValues =
(datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")),
GlobalEmail)
Cursor.execute(UpdateAlertSql, UpdateAlertValues)
db.commit()

#If the time until departure is greater than 4 hours...
if TDeparture > timedelta(hours=4):
    #Output statement.
    window3["-NOTIFICATION-"].update("Alerts only appear 4
hours in advance to departure" + "(" + TDeparture + ")", visible=True)

#Else (if current time is beyond the flight departure time (AFTER
DEPARTURE))...
else:
    if ExistAlertResult3[0][5] is None:
        TAlert = None
    else:
        TAlert =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][5]))

    if TAlert is not None and TAlert >= timedelta(minutes=20):

        #An up-to-date API query is made.
        flight_summary, flight_positions =
get_searchFLT(SearchEntFlt)
        #Calculates an up-to-date "TDeparture" and ensures the two
date-times
        #are of the same format, for mathematical operations.
        TDeparture =
datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")) -
datetime.fromisoformat(str(ExistAlertResult3[0][3]))

        #Converts elapsed flight time into hours.
        ElapsedTime = (flight_summary.data[0].flight_time / 60 /
60)

```

```
InFlightInfo = "-- Journey Data --" + "\nDeparture Time: "
+ flight_summary.data[0].datetime_takeoff + "\nETA: " +
flight_positions.data[0].eta + "\nElapsed Time: " + str(ElapsedTime) + " hrs"
+ "\nElapsed Distance: " + str(flight_summary.data[0].actual_distance) +
"\n\n-- Position / Navigation Data --" + "\nAltitude: " +
str(flight_positions.data[0].alt) + " ft" + "\nHeading: " +
str(flight_positions.data[0].track) + "\nIndicated Airspeed: " +
str(flight_positions.data[0].vspeed) + " kts" + "\nLatitude/Longitude: " +
str(flight_positions.data[0].lat) + " / " + str(flight_positions.data[0].lon)
+ "\n\nTo Date As Of " + flight_positions.data[0].timestamp
window3["-NOTIFICATION-"].update(InFlightInfo,
visible=True)

#Updates the "LastAlerted" attribute with the current
timestamp.
UpdateAlertSql = "UPDATE alerts SET `LastAlerted` = %s
WHERE `EmailAddress` = %s"
UpdateAlertValues =
(datetime.fromisoformat(flight_positions.data[0].timestamp.replace("Z", "")),
GlobalEmail)
Cursor.execute(UpdateAlertSql, UpdateAlertValues)
db.commit()

else:
    window3["-NOTIFICATION-"].update(visible=False)

window1_run()
```

References

Code, Turtle. "Multiple Windows with Python PySimpleGUI - Tutorial 5." YouTube, 1 Jan. 2024, youtu.be/NUznl-yb3Qc.

Accessed 18 Sept. 2025.

FlightRadar24. "Flightradar24 API - Getting Started." Flightradar24.com, 1 Oct. 2024, fr24api.flightradar24.com/docs/getting-started.

Accessed 25 Sept. 2025.

PySimpleGUI. "PySimpleGUI Documentation." Pysimplegui.com, 10 July 2018, docs.pysimplegui.com/en/latest/documentation/module/elements/.

Accessed 18 Sept. 2025.

Szumski, Damian. "Flightradar24 Python SDK." Pypi.org, 25 Nov. 2025, pypi.org/project/fr24sdk/.

Accessed 25 Sept. 2025.